

REDE NACIONAL DE DADOS DE TRANSPORTE

RNDT

Data Space de Transporte do Brasil

Documento Técnico de Referência — Arquitetura, Componentes e Guia de Implementação do MVP

"Soberania de Dados, Confiança Descentralizada e Interoperabilidade para a Mobilidade Nacional."

Versão	1.0 — MVP
Data	Junho de 2026
Classificação	Público / Uso Interno
Elaboração	Ministério dos Transportes do Brasil
Parceiros Estratégicos	PRF, DNIT, ANTT, SENATRAN, Serpro
Base Arquitetural	IDS RAM 4.0 (International Data Spaces Association) European Mobility Data Space (EMDS)

MINISTÉRIO DOS TRANSPORTES

Brasília — Distrito Federal

2026

O presente documento consolida os fundamentos teóricos, os componentes tecnológicos open-source, os macrofluxos operacionais e o roteiro de implementação prática para o estabelecimento da Rede Nacional de Dados de Transporte (RNDT), alinhada às melhores práticas internacionais de espaços de dados soberanos.

Índice

- **PARTE I - FUNDAMENTAÇÃO TEÓRICA E CONCEITUAL**
 - **1. INTRODUÇÃO AOS DATA SPACES**
 - 1.1 O Que São Data Spaces
 - 1.2 A Estratégia Europeia de Dados e o Contexto Global
 - **2. O MODELO IDS RAM**
 - 2.1 Visão Geral do IDS RAM
 - 2.2 As 5 Camadas da Arquitetura IDS RAM
 - 2.3 Os Papéis no IDS RAM
 - 2.4 O Papel do IDS RAM no EMDS
 - **3. O EMDS (EUROPEAN MOBILITY DATA SPACE)**
 - 3.1 Visão Geral do EMDS
 - 3.2 Padrões de Mobilidade no EMDS
 - 3.3 Formatos de Serialização no EMDS
 - 3.4 Modelos de Comunicação no EMDS
- **PARTE II - COMPONENTES E MACROFLUXOS**
 - **4. COMPONENTES DO DATA SPACE**
 - 4.1 O Conector de Dados
 - Eclipse Dataspace Components (EDC)
 - FIWARE True Connector
 - FIWARE Data Space Connector (DSC)
 - Distribuições Prontas e Templates de Código
 - Ferramentas Open-Source para Interfaces e Painéis Visuais
 - Motores de Políticas Semânticas e Validação
 - Orquestradores e Pipelines de Dados
 - EDC Connector Launcher / Postman Ecosystem
 - Visual Policy Editors

- Node-RED (Para Prototipagem Rápida e Ingestão)
- Resumo da Estratégia de Adoção
- 4.2 O Gestor de Identidade (DAPS/IAM)
 - Keycloak
 - Omejdn DAPS
 - FIWARE IdM (Keyrock)
 - Funcionamento no MVP com ICP-Brasil
- 4.3 O Metadata Broker (Catálogo Central)
 - CKAN
 - TrueGg/Dataspace Broker
 - Exemplo Real: A Configuração de Colheita no CKAN para o MVP
- 4.4 A Clearing House (Livro de Registro)
 - IDS Clearing House (Fraunhofer ISST)
 - Hyperledger Fabric / Besu
 - Exemplo Real: Como a Clearing House Opera na Fase 4 (MVP)
- 4.5 A App Store (Catálogo de Serviços)
 - Conceito de "Bring Algorithms to the Data"
 - Como funciona a Engrenagem Técnica?
 - 3 Exemplos Reais Aplicados ao Cenário de Transportes
 - Ferramentas e Implementações para Acelerar o MVP
 - Desenhando o componente de forma "Low-Code" para o MVP
 - Recomendação Pragmática para o MVP da RNDT
 - O Fluxo Real: Do Código ao Bit de Retorno
- 5. MACROFLUXO DO PROVEDOR DE DADOS
 - 5.1 Visão Geral do Macrofluxo
 - 5.2 Fase 1: Preparação e Ingestão dos Dados
 - 5.2.1 O Propósito da Fase 1
 - 5.2.2 Arquitetura da Fase 1: Do Sistema Legado ao Conector
 - 5.2.3 O Data Plane: O Canal de Transferência
 - 5.2.4 Checklist de Conclusão da Fase 1
 - 5.3 Fase 2: Publicação e Catalogação
 - 5.3.1 O Propósito da Fase 2
 - 5.3.2 O Protocolo DSP na Fase 2
 - 5.3.3 Colheita pelo Metadata Broker (CKAN)
 - 5.3.4 A Experiência do Consumidor na Descoberta
 - 5.3.5 Checklist de Conclusão da Fase 2
 - 5.4 Fase 3: Negociação de Contratos
 - 5.4.1 O Propósito da Fase 3
 - 5.4.2 O Fluxo de Negociação DSP
 - 5.4.3 Passo 1: Catalog Request
 - 5.4.4 Passo 2: Catalog Response (Oferta + Políticas)
 - 5.4.5 Passo 3: Contract Request (Aceite pelo Consumidor)
 - 5.4.6 Passo 4: Contract Agreement (Contrato Assinado)

- 5.4.7 O Papel do DAPS/Omejdn na Fase 3
- 5.4.8 Checklist de Conclusão da Fase 3
- 5.5 Fase 4: Transferência e Auditoria
 - 5.5.1 O Propósito da Fase 4
 - 5.5.2 O Fluxo de Transferência DSP
 - 5.5.3 Passo 1: Transfer Request
 - 5.5.4 Passo 2: Validação e Preparação pelo Provedor
 - 5.5.5 Passo 3: Transferência do Dado Bruto
 - 5.5.6 Passo 4: Geração do Recibo de Transferência
 - 5.5.7 Passo 5: Registro na Clearing House
 - 5.5.8 O Papel da App Store na Fase 4 Avançada
 - 5.5.9 Checklist de Conclusão da Fase 4
- 5.6 Síntese do Macrofluxo: O Ciclo Fechado de Soberania
- 5.7 Diagrama Consolidado do Macrofluxo
- 6. MACROFLUXO DO CONSUMIDOR DE DADOS
 - 6.1 Visão Geral do Macrofluxo do Consumidor
 - 6.2 Fase 1: Descoberta e Seleção de Ativos
 - 6.2.1 O Ponto de Entrada: O Portal RNDT
 - 6.2.2 AAPI de Descoberta para Integração Programática
 - 6.2.3 O Conector do Consumidor: Configuração Inicial
 - 6.2.4 Checklist de Conclusão da Fase 1
 - 6.3 Fase 2: Due Diligence e Qualificação
 - 6.3.1 Análise de Metadados e Proveniência
 - 6.3.2 Análise de Políticas ODRL
 - 6.3.3 Verificação Técnica de Compatibilidade
 - 6.3.4 Checklist de Conclusão da Fase 2
 - 6.4 Fase 3: Negociação e Contratação
 - 6.4.1 Iniciando a Negociação via DSP
 - 6.4.2 Avaliação da Oferta e Tomada de Decisão
 - 6.4.3 Contra-Proposta (Opcional)
 - 6.4.4 Armazenamento do Contrato Assinado
 - 6.4.5 Checklist de Conclusão da Fase 3
 - 6.5 Fase 4: Consumo e Valorização
 - 6.5.1 Iniciando a Transferência
 - 6.5.2 Recebimento e Processamento do Dado
 - 6.5.3 Geração de Valor: Produtos e Serviços
 - 6.5.4 Compliance Contínuo e Revogação
 - 6.5.5 Checklist de Conclusão da Fase 4
 - 6.6 Ferramentas para o Consumidor
 - 6.6.1 Para as Fases 1 e 2: Automatização de Busca e Negociação
 - 6.6.2 Para a Fase 3: Criação de Data Apps para a App Store
 - 6.6.3 Para a Fase 4: Recepção, Ingestão e Inteligência Analítica
 - 6.6.4 Resumo da Arquitetura do Consumidor para o MVP

- 7. IDENTIDADE, SEGURANÇA E CONFIABILIDADE
 - 7.1 Estratégia de Identidade no MVP
 - 7.1.1 Como funciona o MVP baseado apenas em ICP-Brasil?
 - 7.1.2 Vantagens dessa abordagem para o MVP
 - 7.1.3 O Gargalo (Por que o IDS RAM prefere DIDs a longo prazo?)
 - 7.1.4 Recomendação de Arquitetura para o seu Roadmap
 - 7.2 Credenciais Verificáveis (VCs)
 - 7.2.1 Elas são mandatórias segundo o IDS RAM?
 - 7.2.2 Exemplo Real de Aplicação de VCs no Ecossistema de Transportes
 - 7.2.3 Como o JSON-LD da Credencial Verificável se parece no Conector
 - 7.3 Assinaturas Criptográficas
 - 7.3.1 Cenário A: A Assinatura no MVP (Centralizada ou Rígida)
 - 7.3.2 Cenário B: A Assinatura na Produção (Descentralizada com VCs)
 - 7.3.3 Resumo para a Tomada de Decisão
 - 7.4 Remote Attestation (Atestação Remota)
 - 7.4.1 Como se dá o fluxo real? (O Princípio Técnico)
 - 7.4.2 Exemplo Real no Cenário da RNDT
 - 7.4.3 Ferramentas Open-Source Utilizadas para Implementar
 - 7.4.4 Como aplicar no Roadmap do seu MVP?
 - 7.5 Validação de Acesso no MVP
 - 7.5.1 O Cenário Prático
 - 7.5.2 Exemplo Real do Algoritmo de Validação no Conector do Provedor
 - 7.5.3 O que muda desse modelo para a Produção (Com Remote Attestation)?
- PARTE III - EXEMPLOS PRÁTICOS E APLICAÇÕES REAIS
 - 8. CASOS DE USO REAIS
 - 8.1 Base de Dados de Pavimentos do DNIT
 - 8.2 Dados de Sinistros da PRF
 - 8.3 Dados de Telemetria de Carga (ANTT)
 - 8.4 Dados de Estacionamento e Recarga Elétrica
 - 8.5 App Store no Transporte
 - 9. FERRAMENTAS E IMPLEMENTAÇÕES OPEN-SOURCE
 - 9.1 Frameworks de Conectores
 - 9.2 Motores de Transformação e Integração
 - 9.3 Gestores de Identidade
 - 9.4 Catálogos e Brokers
 - 9.5 Clearing House
 - 9.6 App Store e Orquestração
 - 9.7 Ferramentas para Consumidor
 - 10. GARANTIA DE CUMPRIMENTO DE REGRAS (POLICY ENFORCEMENT)
 - 10.1 Barreira Estática (Plano de Controle)
 - 10.2 Execução Dinâmica (Plano de Dados)
 - 10.3 Auditoria e Confiança Conectada
 - 11. COMPONENTES COMPARTILHADOS: FLUXO INTEGRADO

- 11.1 Visão Sistêmica da RNDT
- 11.2 Interação entre Componentes
- 11.3 Onboarding de Participantes
- PARTE IV — ARQUITETURA OPERACIONAL DO MVP DA RNDT: INTEGRAÇÃO, GOVERNANÇA E ROTEIRO DE IMPLEMENTAÇÃO
 - Introdução
 - 1. Arquitetura de Referência do MVP da RNDT
 - 1.1 Visão Geral da Arquitetura em Camadas
 - 1.2 Decisões Arquiteturais do MVP
 - 2. Roteiro de Implementação em 90 Dias
 - Fase 1 — Dias 1-30: Fundação e Onboarding dos Primeiros Provedores
 - Fase 2 — Dias 31-60: Integração e Primeiros Consumidores
 - Fase 3 — Dias 61-90: App Store e Computação Próxima ao Dado
 - 3. Matriz de Responsabilidades (RACI)
 - 4. Estratégia de Dados para o MVP
 - 4.1 Conjuntos de Dados Prioritários
 - 4.2 Pipeline de Transformação (Apache Camel)
 - 5. Modelo de Governança e Precificação do MVP
 - 5.1 Tipos de Acesso
 - 5.2 Precificação do MVP
 - 6. Segurança e Conformidade no MVP
 - 6.1 Níveis de Segurança (Baseados no IDS RAM)
 - 6.2 Conformidade Legal
 - 7. Indicadores de Sucesso do MVP (KPIs)
 - 8. Riscos e Mitigações
 - 9. Próximos Passos Pós-MVP
 - 9.1 Evolução para Produção (6-12 meses)
 - 9.2 Visão de Longo Prazo (3-5 anos)
 - 10. Glossário Rápido do MVP
 - Conclusão da Parte IV

PARTE I - FUNDAMENTAÇÃO TEÓRICA E CONCEITUAL

1. INTRODUÇÃO AOS DATA SPACES

1.1 O Que São Data Spaces

Data Spaces são ecossistemas federados de compartilhamento de dados que garantem a soberania dos dados, a confiança descentralizada e a interoperabilidade entre participantes que não necessariamente se conhecem. Diferentemente de arquiteturas tradicionais de integração, onde os dados são centralizados em um grande data lake ou abertos irrestritamente em portais, os Data Spaces mantêm a custódia dos dados com o provedor original, permitindo que ele defina as regras de uso mesmo após o dado ter sido compartilhado.

Os princípios fundamentais incluem:

- **Soberania de Dados (*Data Sovereignty*):** O dono do dado determina as regras de uso, retenção, finalidade e precificação, mesmo quando o dado está sendo consumido por terceiros.
- **Confiança Descentralizada:** A confiança não depende de uma relação prévia entre as partes, mas de certificados criptográficos, contratos digitais (ODRL) e validação automatizada de identidades.
- **Interoperabilidade:** Os dados trafegam sob padrões semânticos e técnicos internacionais (como DATEX II, GTFS, mobilityDCAT-ap), garantindo que qualquer participante da rede compreenda o significado e a estrutura da informação sem necessidade de integrações ad-hoc.

A diferença crucial entre Data Spaces e arquiteturas tradicionais de integração (como ETLs, APIs REST abertas ou portais de dados abertos) é que, nas arquiteturas tradicionais, o controle sobre o dado é perdido no momento da cópia. No Data Space, o controle persiste através de contratos digitais vinculativos e mecanismos de aplicação forçada de políticas (*Policy Enforcement*).

1.2 A Estratégia Europeia de Dados e o Contexto Global

A **Estratégia Europeia de Dados** (European Data Strategy), lançada em 2020, estabeleceu a visão de que os dados devem fluir livremente dentro da União Europeia, mas sob regras de soberania, privacidade e competitividade. Para viabilizar essa visão, a Comissão Europeia propôs a criação de **Common European Data Spaces** — espaços de dados setoriais que unem participantes públicos e privados sob uma governança comum.

O **European Mobility Data Space (EMDS)** é um desses espaços setoriais, focado especificamente em transportes, logística e mobilidade urbana. Ele foi concebido para permitir que operadoras de transporte público, concessionárias de rodovias, plataformas de navegação, seguradoras e órgãos governamentais compartilhem dados de tráfego, infraestrutura, frotas e sinistros de forma segura e interoperável.

Para a **RNDT (Rede Nacional de Dados de Transporte)** no Brasil, o EMDS serve como espelho arquitetural e operacional. A RNDT pode adotar os mesmos padrões técnicos (IDS RAM, EDC, DATEX II, mobilityDCAT-ap), adaptando-os ao contexto jurídico brasileiro (LGPD, ICP-Brasil, Marco Legal de Garantias) e às especificidades da malha viária e do transporte nacional. A relevância é direta: sem um

modelo de Data Space, o Brasil continuará com bases de transporte fragmentadas em silos (DNIT, PRF, ANTT, estados, municípios), impedindo a geração de inteligência integrada para políticas públicas e inovação no setor.

2. O MODELO IDS RAM (INTERNATIONAL DATA SPACES REFERENCE ARCHITECTURE MODEL)

2.1 Visão Geral do IDS RAM

O **IDS RAM** (*International Data Spaces Reference Architecture Model*) foi desenvolvido pela **IDSA** (*International Data Spaces Association*), um consórcio global de empresas, institutos de pesquisa e governos fundado na Alemanha em 2016. O modelo nasceu da constatação de que a economia digital precisava de uma arquitetura de referência que garantisse a soberania dos dados em ecossistemas federados, especialmente em setores industriais sensíveis como automotivo, saúde, energia e mobilidade.

O propósito do IDS RAM é fornecer um **blueprint conceitual, técnico e de governança** que permita a qualquer organização construir ou participar de um Data Space sem reinventar os pilares de segurança, semântica e interoperabilidade. Ele é agnóstico de setor, mas foi projetado para ser especializado por domínio — como ocorre no EMDS para mobilidade.

A relação com o EMDS é de **fundamento para aplicação**: o IDS RAM fornece as camadas abstratas de arquitetura (negócio, dados, funcionalidade, processo, sistema), enquanto o EMDS traduz essas camadas para o vocabulário específico de transportes (GTFS, DATEX II, mobilityDCAT-ap). Para a RNDT, essa relação é igualmente válida: a RNDT será a aplicação setorial brasileira construída sobre os alicerces do IDS RAM.

2.2 As 5 Camadas da Arquitetura IDS RAM

O IDS RAM organiza um Data Space em cinco camadas hierárquicas e interdependentes:

Camada de Negócio (*Business Layer*)

Define os modelos de governança, os papéis dos participantes (Data Owner, Provider, Consumer, Broker, Subject, Collector), os aspectos legais dos contratos de dados e as regras de precificação e licenciamento. É nesta camada que se decide, por exemplo, que uma concessionária de rodovias pode vender dados de tráfego, mas um órgão público deve disponibilizá-los gratuitamente com restrições de uso.

Camada de Dados (*Data Layer*)

Especifica o vocabulário, as ontologias e os metadados que garantem a semântica comum. No contexto de transporte, esta camada abriga os padrões **mobilityDCAT-ap** (para catalogação), **DATEX II** (para

infraestrutura e trânsito), **GTFS/GTFS-RT** (para transporte público), **NeTEx** (para redes e tarifas) e **SIRI** (para informações em tempo real entre operadores).

Camada de Funcionalidade (*Functional Layer*)

Estabelece os requisitos técnicos que os componentes de software devem cumprir. Define, por exemplo, que um conector deve separar o **Control Plane** (gestão de contratos e identidades) do **Data Plane** (tráfego de dados), e que deve suportar o **Dataspace Protocol (DSP)** para negociação automatizada.

Camada de Processo (*Process Layer*)

Descreve os fluxos de interação entre os atores. É nesta camada que se mapeiam as **Fases 1 a 4** do macrofluxo do Provedor (preparação, publicação, negociação, transferência) e as fases espelhadas do Consumidor.

Camada de Sistema (*System Layer*)

Representa a infraestrutura real de hardware, redes, contêineres Docker, servidores Kubernetes, certificados digitais e protocolos de comunicação. É onde o **Eclipse Dataspace Connector (EDC)**, o **Apache Camel**, o **Keycloak** e o **CKAN** ganham vida como processos em execução.

2.3 Os Papéis no IDS RAM

O IDS RAM define uma taxonomia rigorosa de papéis para evitar ambiguidades jurídicas e técnicas:

- **Data Owner (Dono do Dado):** A entidade que detém o controle legal e estratégico sobre o dado. No caso da PRF, o Data Owner é a própria instituição, que decide se os dados de sinistros serão públicos, restritos ou comerciais.
- **Data Provider (Provedor de Dados):** A entidade que disponibiliza tecnicamente o dado na rede através de um conector. Pode ser o mesmo que o Data Owner (como na PRF) ou uma entidade delegada.
- **Data Consumer (Consumidor de Dados):** A entidade que solicita e utiliza os dados. Pode ser o Ministério dos Transportes, uma concessionária, uma startup de logística ou um instituto de pesquisa.
- **Data Broker (Intermediário de Metadados):** O componente que centraliza apenas os metadados (e não os dados brutos), auxiliando os consumidores a descobrir quais provedores possuem os dados de seu interesse. Na RNDT, este papel é desempenhado pelo **Metadata Broker (CKAN)**.
- **Data Subject (Titular dos Dados):** A pessoa física a quem os dados se referem. No contexto de sinistros, é o condutor ou passageiro cujos dados pessoais estão protegidos pela LGPD.
- **Data Collector (Coletor de Dados):** A entidade que coleta dados brutos de sensores, sistemas legados ou formulários e os estrutura para ingestão no Data Space. No caso de dados de pavimento, o Data Collector pode ser o DNIT ou uma empresa contratada de varredura rodoviária.

2.4 O Papel do IDS RAM no EMDS

O IDS RAM funciona como o **projeto de engenharia e arquitetura conceitual de base**, enquanto o EMDS é a **aplicação prática e setorial** desse modelo no universo dos transportes e da mobilidade na Europa. Sem o IDS RAM, o EMDS seria apenas um conjunto isolado de APIs e repositórios de dados sem interoperabilidade jurídica ou técnica de segurança.

O IDS RAM confere ao EMDS o enquadramento de **confiança descentralizada**, permitindo que concorrentes comerciais e órgãos públicos compartilhem dados de transporte de forma segura, auditável e juridicamente vinculativa. Os blocos de construção (*building blocks*) do IDS RAM são materializados no EMDS da seguinte forma:

- **No Conector:** Quando o EMDS adota o **Eclipse Dataspace Components (EDC)** ou o **FIWARE Data Space Connector**, ele está implementando um software cujo design obedece rigorosamente às especificações de conector do IDS RAM.
- **Nos Metadados:** O IDS RAM exige um vocabulário de metadados comum para a rede (*Information Model*). O EMDS adotou e expandiu isso criando o **mobilityDCAT-ap** para o catálogo de transportes.
- **Nos Contratos:** A capacidade que uma prefeitura europeia tem no EMDS de estipular que *"meus dados de sensores de tráfego em tempo real só podem ser acessados por parceiros homologados e expiram em 5 minutos"* baseia-se diretamente nas especificações de controle de uso do IDS RAM.

Para a concepção da **RNDT** no Brasil, o entendimento é o mesmo: se a RNDT se espelhar no EMDS europeu para a implementação prática, ela estará, por consequência, importando os fundamentos conceituais e de segurança descritos no **IDS RAM**.

3. O EMDS (EUROPEAN MOBILITY DATA SPACE)

3.1 Visão Geral do EMDS

O **EMDS** (*European Mobility Data Space*) é um dos espaços comuns de dados europeus criados sob a Estratégia Europeia de Dados. Ele foi concebido para responder a uma das maiores demandas de arquitetura atuais: a necessidade de unificar os ecossistemas de mobilidade (transporte público, infraestrutura viária, logística de cargas, veículos conectados) sob uma governança federada que garanta a soberania dos dados.

Os objetivos do EMDS incluem:

- Eliminar silos de dados entre operadoras de transporte, concessionárias de rodovias, fabricantes de veículos e órgãos públicos.
- Permitir a criação de novos produtos e serviços (como roteirização multimodal, otimização de frotas, previsão de congestionamentos) através da combinação segura de dados de múltiplas fontes.
- Garantir que os dados pessoais e sensíveis (como trajetos individuais, placas de veículos) sejam protegidos por padrão, em conformidade com o RGPD.

A relação com o IDS RAM é de **especialização setorial**: o EMDS pega os conceitos abstratos do IDS RAM e os adapta para os desafios logísticos e de tráfego. O consórcio europeu que desenhou o EMDS (o *PrepDSpace4Mobility*) utilizou os blocos de construção (*building blocks*) do IDS RAM para garantir que os operadores de ônibus, trens e rodovias pudessem trocar dados sob regras idênticas.

3.2 Padrões de Mobilidade no EMDS

Para que os conectores conversem entre si no EMDS, eles precisam adotar padrões consolidados internacionalmente, divididos por nichos de transporte:

GTFS (General Transit Feed Specification) e GTFS-RT (Realtime)

O padrão global para horários, rotas e posicionamento geográfico de frotas de ônibus, trens e metrô em tempo real. O GTFS Estático é uma coleção de arquivos **CSV** compactados em um arquivo `.zip` (ex: `routes.txt`, `trips.txt`). O GTFS-RT, para transmissão de dados de GPS e alertas em tempo real, utiliza **Protocol Buffers (Protobuf)** da Google — um formato binário compacto que reduz drasticamente o tamanho do payload na rede.

NeTEx (Network Exchange)

Padrão XML europeu altamente detalhado para dados de infraestrutura de transporte público: tarifas complexas, redes de paradas, acessibilidade, calendários de operação. É mantido pelo CEN (*Comité Européen de Normalização*).

SIRI (Service Interface for Realtime Information)

Padrão XML focado na troca de informações de transporte público em tempo real entre operadores: previsão de chegada na parada, cancelamentos de viagem, desvios de rota.

DATEX II

O padrão definitivo europeu para gerenciamento de tráfego, condições de estradas, sensores de velocidade em rodovias, pedágios e eventos de trânsito (obras, acidentes, condições meteorológicas). Serializado em **XML** ou **JSON**.

mobilityDCAT-ap

Este não é um formato para o dado que roda no ônibus, mas sim o padrão de *metadados* exigido pelo IDS RAM e pelo EMDS. É o formato que descreve o conjunto de dados (ex: *"Este arquivo contém os dados de GTFS do município X, atualizados diariamente, sob a licença Y"*). É o padrão que o *Broker* lê para indexar a busca. Utiliza **JSON-LD** ou **Turtle** (`.ttl`).

FTI (Freight Transportation Information)

Modelos voltados para rastreamento de cadeias de suprimento e fretes intermodais. No Brasil, o equivalente estratégico estruturado que busca essa interoperabilidade é o ecossistema do **DT-e (Documento Eletrônico de Transporte)**.

3.3 Formatos de Serialização no EMDS

Os atores da rede utilizam quatro formatos principais de serialização, a depender do nicho de transporte e do objetivo do dado:

Protocol Buffers (Protobuf)

- **Onde é usado:** No padrão **GTFS-RT** para o posicionamento de frotas de ônibus/trens por GPS e alertas de viagem em tempo real.
- **Características:** É um formato binário desenvolvido pela Google. Em vez de transmitir texto puro (como JSON ou XML), o dado é compactado em uma estrutura binária altamente otimizada na Fase 1. Isso reduz drasticamente o tamanho do payload e o consumo de banda de rede do conector, sendo ideal para streaming de milhares de veículos simultâneos.

XML (Extensible Markup Language)

- **Onde é usado:** Nos padrões **DATEX II**, **NeTEx** e **SIRI**.
- **Características:** O XML utiliza esquemas de validação rígidos (arquivos `.xsd`). Ele é historicamente o formato padrão das especificações do CEN (*Comité Européen de Normalização*), garantindo que estruturas hierárquicas altamente complexas e com muitas referências cruzadas sejam validadas rigidamente nas portas de entrada e saída dos conectores.

JSON-LD (JavaScript Object Notation for Linked Data)

- **Onde é usado:** No catálogo de metadados sob o perfil **mobilityDCAT-ap** e na negociação de contratos e termos do próprio conector (como o *Eclipse Dataspace Connector - EDC via Dataspace Protocol*).
- **Características:** É uma extensão do JSON tradicional que adiciona a propriedade `@context`. Ele transforma dados textuais comuns em grafos interconectados (Web Semântica). Ele serve para que o *Metadata Broker* central indexe e compreenda o significado exato de conceitos abstratos (como políticas de privacidade baseadas no RGPD ou termos de uso), permitindo buscas inteligentes por humanos ou agentes de IA.

CSV (Comma-Separated Values)

- **Onde é usado:** No **GTFS Estático** (tabelas de horários planejados de linhas, calendários e rotas).
- **Características:** Arquivos de texto simples delimitados por vírgula, compactados dentro de um arquivo `.zip`. É o formato padrão consumido por planejadores de rota globais (como Google Maps).

3.4 Modelos de Comunicação no EMDS

Para transferir esses formatos de serialização de forma segura e soberana, os conectores de espaço de dados utilizam diferentes modelos de arquitetura de rede:

Modelo de Request-Response Tradicional (REST / HTTP)

- **Como funciona:** O conector do Consumidor faz uma chamada síncrona `GET` para o endpoint do conector do Provedor (ex: solicitando o arquivo `.zip` do GTFS Estático ou um snapshot atual de obras na rodovia via DATEX II).
- **Protocolo:** Baseado em **HTTP/1.1** ou **HTTP/2** protegido com TLS mútuo (mTLS), onde os conectores autenticam suas identidades digitais antes de liberar o payload.

Modelo Orientado a Eventos e Mensageria (Publish-Subscribe)

- **Como funciona:** Essencial para os dados em tempo real da Fase 1 e Fase 4 (como telemetria GPS e alertas SIRI). O Provedor não espera um pedido; ele publica atualizações à medida que os eventos acontecem. Os consumidores interessados se inscrevem em "tópicos" específicos do canal e recebem os pacotes continuamente.
- **Protocolos e Brokers de Código Aberto:** Internamente (Fase 1), os órgãos utilizam **Apache Kafka**, **RabbitMQ** ou **MQTT**. Na transmissão externa de conector para conector (Fase 4), os barramentos de dados utilizam canais como **WebSockets** persistentes ou **gRPC** (muito utilizado para trafegar os binários do Protobuf devido à alta performance).

Modelo de Server-Sent Events (SSE)

- **Como funciona:** Uma variação unidirecional usada para streaming HTTP. O conector do consumidor abre uma conexão de longa duração com o conector do provedor. O provedor mantém essa linha aberta e envia as atualizações de transporte (como previsões de chegada SIRI) em forma de fluxo contínuo de texto conforme elas ocorrem, fechando o canal apenas em caso de falha de rede.

PARTE II - COMPONENTES E MACROFLUXOS

4. COMPONENTES DO DATA SPACE

4.1 O Conector de Dados (Data Connector)

Eclipse Dataspace Components (EDC)

O **Eclipse Dataspace Components (EDC)** é o projeto de código aberto mais ativo e amplamente adotado no mundo para a criação de conectores de *Data Spaces*. Ele é utilizado massivamente na iniciativa automotiva *Catena-X* e no *European Mobility Data Space*.

- **O que ele faz:** Fornece um framework modular e extensível para implementar a negociação de contratos de dados, aplicação de políticas (ex: restrição de uso por tempo ou finalidade) e a transferência segura de dados de ponta a ponta.
- **Arquitetura:** Ele separa o componente em **Control Plane** (gerenciamento de contratos e protocolos) e **Data Plane** (responsável pelo tráfego pesado do dado em si, seja via HTTP, S3, SQL, etc.).
- **Linguagem principal:** Java (baseado em Eclipse).
- **Onde encontrar:** [GitHub do Eclipse-EDC](#)

FIWARE True Connector

Para ecossistemas que já utilizam a arquitetura FIWARE (muito comum em cidades inteligentes e mobilidade urbana), o **True Connector** é a solução de código aberto padrão.

- **O que ele faz:** Ele integra a arquitetura de Context Brokers do FIWARE com as especificações de segurança e soberania de dados da IDSA (International Data Spaces Association). Ele atua como uma casca segura para expor APIs de contexto interno para o mundo externo do data space.
- **Onde encontrar:** [GitHub do FIWARE - True Connector](#)

FIWARE Data Space Connector (DSC)

O projeto **FIWARE/data-space-connector** (FIWARE DSC) é uma iniciativa de código aberto oficial da **FIWARE Foundation**. Ele foi concebido especificamente para responder a uma das maiores demandas de arquitetura atuais: a necessidade de unificar os ecossistemas do **FIWARE** (líder em Cidades Inteligentes e IoT) com as diretrizes e componentes do **Eclipse Dataspace Components (EDC)** e as recomendações técnicas da **DSBA** (*Data Spaces Business Alliance*, que reúne Gaia-X, IDSA, FIWARE e BDVA).

O repositório oficial funciona como um guia de arquitetura e um *Helm Umbrella-Chart*, empacotando múltiplos componentes de forma que qualquer organização possa implantar rapidamente seu próprio nó (seja como provedor, consumidor ou ambos) em um ecossistema federado.

As principais características técnicas e módulos integrados nesta solução incluem:

Governança e Autenticação Descentralizada (OID4VC)

O conector baseia-se fortemente no padrão **OID4VC** (*OpenID for Verifiable Credentials*) e em identidades descentralizadas baseadas em **DIDs** (W3C).

- Ele valida credenciais verificáveis (*Verifiable Credentials* - VCs) de quem tenta acessar os dados. Ele é projetado para ser compatível com especificações europeias (como o EBSI) e recomendações da iniciativa **Gaia-X**.
- **M2M e H2M**: Suporta nativamente tanto interações de máquina para máquina (sistemas integrados/IIA) quanto humanos para máquina (usando carteiras digitais — *digital wallets* — para autenticação).

Framework de Autorização Baseado em Políticas (ODRL e OPA)

O FIWARE DSC adota uma arquitetura rigorosa de aplicação de políticas de soberania de dados usando o padrão W3C **ODRL** (*Open Digital Rights Language*).

- Ele utiliza o **APISIX** como API Gateway atuando como PEP (*Policy Enforcement Point*).
- O mecanismo de decisão de políticas (PDP) é implementado através do **Open Policy Agent (OPA)**, que avalia dinamicamente se o consumidor autenticado possui as credenciais exigidas para ler o dado solicitado.

Catálogo de Produtos e Contratos (Padrão TM Forum)

Diferente de conectores puros que lidam apenas com o canal técnico de transmissão, o FIWARE DSC traz componentes de negócio.

- Ele integra APIs baseadas nos padrões do **TM Forum** para gerenciar especificações de produtos, ofertas de dados, ordens de serviço e inventários de contratos.
- Oferece um **Marketplace Portal** integrado (derivado do *FIWARE Business API Ecosystem* — BAE), fornecendo uma interface web amigável para que o usuário possa publicar ofertas ou negociar contratos de dados visualmente.

Integração com o Ecossistema EDC (Dataspace Protocol)

O projeto implementa nativamente o **Dataspace Protocol (DSP)** do projeto Eclipse/IDSA para orquestrar o acesso ao catálogo, a negociação de contratos formais e o controle do processo de transferência segura de dados.

Camada de Dados Interoperável

Embora o conector ofereça compatibilidade nativa imediata com o padrão **ETSI NGSI-LD** (através do componente *Scorpio Context Broker*, nativo do FIWARE), ele funciona como um mediador universal capaz de proteger e trafegar dados vindos de quase qualquer interface baseada em HTTP, incluindo:

- APIs REST convencionais
- Buckets S3 (Object Storage)
- Interfaces NGSIv2 legadas

Como isso se aplica ao contexto de um Gestor de TIC (como na RNDT)?

Se um órgão brasileiro decidir utilizar a arquitetura FIWARE para sua smart city ou infraestrutura e desejar conectá-la a uma rede federada nacional de transportes (RNDT), o FIWARE/data-space-connector é a peça de engenharia ideal. Ele evita a necessidade de "reinventar a roda" em termos de segurança, tokens de contratos e conformidade com o ecossistema europeu, permitindo implantar todo o arcabouço tecnológico em clusters **Kubernetes** corporativos utilizando pacotes Helm prontos.

Distribuições Prontas e Templates de Código (Boilerplates)

Como o repositório principal do Eclipse EDC fornece apenas as peças de engenharia básicas (o *framework*), a comunidade criou distribuições abertas com conectores pré-configurados e prontos para produção:

- **EDC Connector Framework (O Core):** É o próprio projeto principal sob a licença Eclipse Public License 2.0. Ele é modularizado em Java (usando Gradle). Para construir o seu, você cria um repositório próprio e importa os módulos necessários (como extensões de banco de dados PostgreSQL, suporte a nuvens específicas, etc.).
- **Código:** github.com/eclipse-edc/Connector
- **EDC Samples:** O repositório oficial de exemplos interativos em código aberto. É a melhor ferramenta para desenvolvedores começarem a rodar conectores locais em minutos (via Gradle e Docker), mostrando cenários reais de transferência de dados e negociação de contratos básicos.
- **Código:** github.com/eclipse-edc/Samples
- **AOS / Tractus-X Eclipse EDC (Catena-X):** A indústria automotiva europeia criou uma distribuição altamente robusta e open-source do EDC especificamente voltada para ecossistemas industriais complexos. Se você quer ver uma implementação real de produção com segurança avançada baseada no EDC, este repositório é uma das maiores referências de código aberto do mundo.
- **Código:** github.com/eclipse-tractusx/tractusx-edc

Ferramentas Open-Source para Interfaces e Painéis Visuais (UIs)

Por padrão, o EDC é controlado puramente via APIs REST. Para gerenciar os nós de forma gráfica, você precisa de uma interface interativa (UI). Existem excelentes projetos de código aberto focados nisso:

- **EDC DataSpace Authority (DSA) UI / Amora UI:** Projetos open-source mantidos por membros do consórcio Eclipse para fornecer um painel administrativo web para o EDC. Eles permitem que você gerencie visualmente ativos de dados (*Assets*), crie políticas de uso (*Policies*), defina ofertas de contrato (*Contract Definitions*) e monitore as transferências ativas.
- **Prometheus-X Portal & UI Components:** Um ecossistema de ferramentas 100% open-source voltado para a construção de Data Spaces. Eles fornecem componentes de interface web (construídos em frameworks como Vue.js ou React) prontos para serem acoplados ao lado de conectores EDC, facilitando a vida do gestor na hora de criar catálogos e gerenciar consentimentos.
- **Código:** github.com/prometheus-x

Motores de Políticas Semânticas e Validação (Open Source)

O conector EDC precisa avaliar se uma requisição cumpre as regras do contrato usando a linguagem ODRL. Para construir e testar essas regras programaticamente, utilizam-se estas ferramentas abertas:

- **Open Policy Agent (OPA):** Embora o EDC possua seu próprio motor interno de avaliação de políticas em Java, muitas arquiteturas estendem o conector delegando decisões complexas para o OPA. É o motor open-source de controle de políticas em nuvem mais famoso do mercado, rodando através de uma linguagem declarativa chamada Rego.
- *Código:* github.com/open-policy-agent/opa

Orquestradores e Pipelines de Dados para o *Data Plane* do EDC

O EDC possui um componente chamado *Data Plane*, que é o responsável por buscar o dado no seu sistema legado e transmiti-lo. Para "construir" e customizar esse comportamento de forma flexível e aberta:

- **Apache Camel:** Uma das ferramentas open-source de integração mais poderosas do mercado. Existe um esforço contínuo da comunidade para integrar rotas do Apache Camel dentro do Data Plane do EDC. Isso permite que seu conector EDC se conecte nativamente de forma visual ou declarativa a centenas de formatos e protocolos legados (como SAP, arquivos locais, filas AMQP, etc.) apenas configurando rotas Camel.
- **Node-RED:** Excelente para a fase de prototipagem e desenvolvimento ágil do conector. Você pode construir um fluxo visual no Node-RED que coleta os dados dos seus arquivos/bancos, faz o tratamento e expõe um endpoint HTTP no próprio Node-RED para empurrar o dado diretamente para o *Data Plane* do seu conector EDC.

EDC Connector Launcher / Postman Ecosystem

Como o EDC opera fortemente através de APIs REST para configurar contratos, o uso de coleções interativas no **Postman** ou **Insomnia** associadas a ambientes locais de teste é a prática padrão do mercado.

- A comunidade EDC disponibiliza repositórios com o "EDC Launcher" configurado via **Docker Compose**. Com um único comando, você sobe de forma interativa dois conectores locais (geralmente chamados de *Company A* e *Company B*) simulando uma rede completa na sua máquina para testar o envio de dados.

Visual Policy Editors (Editores Visuais de Políticas)

Uma das partes mais complexas de um conector é definir o contrato de uso (ex: "*Este dado só pode ser lido por órgãos públicos e expira em 30 dias*"). Escrever isso puramente em formatos como JSON-LD (ODRL — *Open Digital Rights Language*) é propício a erros.

- **Prometheus-X Policy Editor:** É uma ferramenta interativa visual baseada na web criada por consórcios de data spaces europeus que permite construir regras de uso arrastando blocos e gerando o código de conformidade ODRL que o seu conector vai interpretar nativamente.

Node-RED (Para Prototipagem Rápida e Ingestão)

Muitas vezes, o conector precisa buscar dados de sistemas legados de forma ágil (arquivos locais, correias de transmissão MQTT, bancos de dados antigos). O **Node-RED** é uma ferramenta de desenvolvimento visual baseada em fluxos altamente interativa.

- Muitos arquitetos utilizam o Node-RED para criar o pipeline de transformação semântica (convertendo dados internos para formatos como DATEX II ou GTFS) e, em seguida, usam um nó HTTP no próprio Node-RED para empurrar o dado diretamente para o *Data Plane* do conector EDC.

Resumo da Estratégia de Adoção

Para um órgão público ou empresa que deseja se estruturar rapidamente:

1. **A Base do Conector:** Baixe o código do **Eclipse Dataspace Components (EDC)**.
2. **A Infraestrutura:** Utilize **Docker/Kubernetes** para orquestrar e subir o conector.
3. **A Ferramenta Interativa de Teste:** Configure o ambiente local com o repositório de amostras (*samples*) do EDC e utilize o **Postman** para simular as negociações de contratos e visualizar o fluxo de metadados funcionando em tempo real.

4.2 O Gestor de Identidade (DAPS/IAM)

Keycloak (A Solução Mais Rápida e Flexível)

O **Keycloak** (mantido pela Red Hat/CNCF) é a ferramenta open-source de gerenciamento de identidade e acesso (IAM) mais popular do mercado mundial. Embora não tenha sido criado especificamente para a iniciativa de *Data Spaces* europeia, ele possui todas as engrenagens necessárias para atuar como o DAPS do IDS RAM no seu MVP.

Por que ele acelera o MVP?

- **Suporte Nativo a X.509 (mTLS):** O Keycloak possui um mecanismo nativo onde ele consegue interceptar a chamada do conector, ler o e-CNPJ/Certificado ICP-Brasil da instituição e extrair os dados de identidade diretamente dele.
- **Mapeamento de Atributos:** Você pode cadastrar em tabelas simples na interface do Keycloak quais atributos (ex: `rndtStatus: "Homologado"`, `orgao: "PRF"`) pertencem a cada CNPJ.
- **Emissão de Tokens JWT:** Ao validar a assinatura do órgão, ele gera o token de acesso estruturado automaticamente.

Exemplo Real de Configuração no Keycloak

Na interface do Keycloak, você cria um *Realm* chamado `RNDT-MVP` e configura o fluxo de autenticação como **"X509/Validate Username Form"**.

No mapeamento do Token (Mappers), você define que o campo `cn` (Common Name) do certificado da

ICP-Brasil será mapeado para o campo `sub` (Subject) do token. Quando o conector da PRF se autentica, o Keycloak gera o token de segurança que servirá de passaporte na rede.

Omejdn DAPS (A Solução Nativa IDS RAM)

O **Omejdn** (desenvolvido pelo instituto de pesquisa alemão *Fraunhofer ISST*) é um servidor de autenticação open-source construído especificamente para atuar como um **DAPS nativo** dentro de ecossistemas que utilizam o Eclipse Dataspace Connector (EDC).

Por que ele acelera o MVP?

- **Feito para o EDC:** Ele já fala nativamente o dialeto que o conector EDC espera. Você não precisa configurar fluxos complexos de mapeamento de tokens, pois o Omejdn já cospe o token com as claims exatas exigidas pela arquitetura de *Data Spaces* (como o `securityProfile`).
- **Autenticação Baseada em Chaves Simples:** Ele permite cadastrar os componentes da rede associando seus IDs diretamente às suas chaves públicas (como as chaves públicas extraídas dos certificados ICP-Brasil do Ministério e dos órgãos).

Exemplo Real de Configuração no Omejdn

O Omejdn é configurado via arquivos YAML simples (infraestrutura como código), o que facilita o deploy automatizado via Docker pela equipe de TI. O cadastro de um participante como o DNIT no arquivo `clients.yml` do Omejdn fica assim:

```
- client_id: "urn:govbr:cnpj:04898488000177" # CNPJ do DNIT
  name: "Departamento Nacional de Infraestrutura de Transportes"
  token_endpoint_auth_method: "private_key_jwt"
  # Chave pública extraída do e-CNPJ ICP-Brasil do DNIT
  public_key: |
    -----BEGIN PUBLIC KEY-----
    MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA0X...
    -----END PUBLIC KEY-----
  attributes:
    rndtStatus: "Homologado"
    securityClearance: "Nivel_2"
```

Quando o conector do DNIT envia uma requisição assinada para o Omejdn, o servidor bate o hash contra essa chave pública cadastrada, valida o nó e devolve o token pronto para a Fase 3.

FIWARE IdM (Keyrock)

Para ecossistemas baseados em Cidades Inteligentes e Mobilidade Urbana, o **Keyrock** é o componente de gestão de identidades da fundação open-source **FIWARE** (altamente conectada com as diretrizes do Gaia-X).

Por que ele acelera o MVP?

- **Alinhamento com APIs de Transporte:** Se o MVP for consumir dados que venham de APIs no padrão NGSI (comum em sensores de tráfego urbanos e frotas de ônibus), o Keyrock já possui integração nativa para gerenciar quais políticas de acesso se aplicam a esses sensores.
- **Interface Pronta para APIs:** Possui uma API REST rica para automação de novos cadastros de frotas e concessionárias parceiras.

Funcionamento no MVP com ICP-Brasil

No desenho de um Produto Mínimo Viável (MVP) para a **RNDT**, é uma estratégia arquitetural perfeitamente válida e pragmática postergar a implementação de DIDs (*Decentralized Identifiers*) e **utilizar exclusivamente a infraestrutura de chaves públicas da ICP-Brasil** para a autenticação e segurança.

O **IDS RAM (Reference Architecture Model)** é agnóstico em relação à tecnologia de criptografia de base, desde que os pilares de **confiança mútua, autenticidade e não-repúdio** sejam garantidos.

Como funciona o MVP baseado apenas em ICP-Brasil?

No ecossistema IDS clássico, o conector utiliza um componente chamado **DAPS (Dynamic Attribute Provisioning Service)** para validar o token de identidade do outro conector. Para o MVP, a validação de identidade máquina-a-máquina (mTLS) pode ser simplificada substituindo a tecnologia de credenciais verificáveis (DIDs) por certificados digitais tradicionais:

1. **Autenticação de Rede (mTLS):** Cada órgão cooperante (PRF, DNIT, ANTT) e cada concessionária privada utiliza um **Certificado de Equipamento/Aplicação (A3 ou SSL/TLS EV)** emitido por uma Autoridade Certificadora homologada pela ICP-Brasil.
2. **Assinatura de Contratos (Fase 3):** No momento em que os conectores fecham o *Contract Agreement*, o payload JSON-LD é assinado digitalmente utilizando a chave privada do certificado ICP-Brasil do órgão.
3. **Validação da Confiança:** Em vez de consultar uma rede *blockchain* ou um barramento de identidades descentralizadas para resolver um DID, o conector do Provedor simplesmente valida a cadeia de certificação do Consumidor contra a **LCR (Lista de Certificados Revogados)** oficial do Instituto Nacional de Tecnologia da Informação (ITI). Se o certificado for ICP-Brasil válido, a máquina assume que o participante é confiável.

Vantagens dessa abordagem para o MVP

- **Aproveitamento Jurídico Legal:** Os certificados ICP-Brasil possuem validade jurídica inquestionável e presunção de veracidade legal no Brasil (segundo a MP nº 2.200-2). Para um ambiente governamental, isso elimina barreiras de compliance jurídico logo no início do projeto.
- **Redução drástica da complexidade técnica:** Implementar DIDs exige configurar resolvedores de identidade (*Universal Resolvers*), gerenciar chaves descentralizadas e, muitas vezes, integrar com redes de registro distribuído (DLT). Utilizar ICP-Brasil reduz o esforço da Fase 2 e 3 para protocolos de segurança web tradicionais (X.509).
- **Velocidade de Entrega:** A equipe de desenvolvimento foca no que gera valor imediato para o negócio do transporte: as esteiras do Apache Camel (Fase 1), o mapeamento DATEX II e a negociação de

contratos de dados (Fase 3/4).

O Gargalo (Por que o IDS RAM prefere DIDs a longo prazo?)

Embora funcione perfeitamente para o MVP, é importante mapear o que o ecossistema perde ao não usar DIDs, para planejar a evolução pós-MVP:

- **Granularidade de Atributos Dinâmicos:** O certificado ICP-Brasil prova *quem* o órgão é (ex: "Polícia Rodoviária Federal"), mas ele não consegue provar, de forma automatizada e dinâmica, *atributos temporais de infraestrutura* (ex: "Este conector específico possui a homologação de segurança nível 2 da RNDT válida até dezembro"). Com DIDs e *Verifiable Credentials* (VCs), a emissão e revogação desses sub-atributos são instantâneas.
- **Custo de Escala:** Gerar e renovar certificados ICP-Brasil para centenas de sistemas, servidores e aplicações de terceiros que desejem se plugar à RNDT pode gerar um custo financeiro e administrativo recorrente elevado. Os DIDs possuem custo de geração e manutenção virtualmente zero.

Recomendação de Arquitetura para o seu Roadmap

Para avançar com segurança:

1. **No MVP:** Configure o plano de controle dos conectores (como o Eclipse EDC) para autenticar os participantes usando certificados **ICP-Brasil (X.509 v3)** injetados no protocolo de comunicação.
2. **Design "Forward-Compatible":** Deixe o campo de identificação de nós estruturado de forma que, no futuro, a string do CNPJ/Certificado possa ser substituída por uma URI de DID (ex: `did:govbr:cnpj:00000000000100`) sem quebrar o banco de dados das Definições de Contrato.

4.3 O Metadata Broker (Catálogo Central)

CKAN (Comprehensive Knowledge Archive Network)

O **CKAN** é a plataforma open-source líder mundial para portais de dados (usada no dados.gov.br e no *European Data Portal*).

Por que acelera o MVP?

- **O que ele faz:** O provedor de dados ganha uma interface web amigável (um painel administrativo). Em vez de escrever códigos em grafos RDF, o usuário preenche um formulário comum de cadastro de conjunto de dados (Título, Descrição, Frequência de atualização, etc.).
- **A facilidade:** Com o plugin de automação semântica ativado, o CKAN converte o formulário preenchido em uma URL pública contendo os padrões **DCAT-ap** e **mobilityDCAT-ap** de forma totalmente automática.

TrueGg/Dataspace Broker (Comunidade IDS)

Se você preferir uma ferramenta nativa desenvolvida especificamente dentro dos repositórios do ecossistema IDS europeu:

- **Por que acelera o MVP:** Existem implementações de referência em contêineres Docker (baseadas na arquitetura do *Fraunhofer* ou da *IDSA*) construídas sobre bancos de dados de grafos (**Triple Stores** como o *Apache Jena* ou *GraphDB*). Ele é projetado para ler diretamente o protocolo DSP (*Dataspace Protocol*) que os conectores EDC usam na Fase 2.

Exemplo Real: A Configuração de Colheita (*Harvesting*) no CKAN para o MVP

Vamos ver na prática como o **CKAN** (atuando como o Metadata Broker da RNDT) é configurado para colher os metadados que a PRF disponibilizou.

Passo 1: O Cadastro do Nó Alvo no Broker

O administrador do Ministério dos Transportes entra na interface administrativa do CKAN do Broker central e cria uma **Fonte de Colheita (Harvest Source)** para a PRF.

A configuração no banco de dados do CKAN fica exatamente assim:

- **Título:** Fonte de Metadados da Polícia Rodoviária Federal
- **URL do Nó:** `https://edc.prf.gov.br/api/v1/dsp/catalog` (*A URL que a PRF informou no onboarding*)
- **Tipo de Colhedor (Harvester Type):** `dcat_jsonld` (*Instrui o CKAN a ler e parsear JSON-LD*)
- **Frequência:** `cron: "0 2 * * *"` (*Todo dia às 2 horas da manhã*)

Passo 2: O Payload JSON-LD Armazenado no Broker

Durante a madrugada, o robô do CKAN acessa a URL da PRF (utilizando o token do Keycloak que vimos na resposta anterior). O conector da PRF responde entregando o catálogo.

O Broker valida a sintaxe e o Apache Solr indexa o seguinte nó semântico **mobilityDCAT-ap** dentro do banco do portal central:

```
{
  "@context": "http://w3id.org/transport/mobilityDCAT-ap/context.jsonld",
  "@id": "https://rndt.gov.br/catalog/prf-sinistros-2026",
  "@type": "dcat:Dataset",
  "dct:title": "Sinistros e Ocorrências Rodoviárias Consolidadas (2026)",
  "dct:description": "Dados geocodificados e corrigidos de acidentes nas rodovias federais.",
  "dct:publisher": "Polícia Rodoviária Federal",
  "dcat:keyword": ["sinistros", "acidentes", "segurança viária", "datex2"],
  "dcat:theme": "http://publications.europa.eu/resource/authority/data-theme/TRAN",
  "dcat:distribution": {
    "@type": "dcat:Distribution",
    "dct:format": "application/xml (DATEX II)",
    "dcat:accessService": {
      "@type": "dcat:DataService",
      "dcat:endpointUrl": "https://edc.prf.gov.br/api/v1/dsp",
      "edc:assetId": "asset-prf-sinistros-2026-datex2"
    }
  }
}
```

Passo 3: A Experiência do Usuário Consumidor

Na manhã seguinte, um analista do DNIT ou uma equipe técnica de uma concessionária entra na URL pública do Broker central da RNDT (<https://rndt.gov.br>) e digita na barra de busca: **"sinistros datex2"**.

1. O motor do Apache Solr do CKAN varre os índices e encontra o registro acima.
2. A tela exibe um botão visual chamado **"Solicitar Acesso via Conector"**.
3. Ao clicar, a interface gráfica do CKAN (ou o conector local do consumidor via API) captura o bloco `dcat:accessService` do JSON-LD: ele descobre que precisa disparar seu conector para falar com <https://edc.prf.gov.br/api/v1/dsp> buscando o Ativo `asset-prf-sinistros-2026-datex2` .

O fluxo se fecha perfeitamente. O Broker atuou como a vitrine inteligente: ele indexou o metadado sem nunca tocar na base de dados sigilosa ou pesada de sinistros da PRF, que permanece segura dentro da infraestrutura deles até que o contrato seja formalmente assinado.

4.4 A Clearing House (Livro de Registro)

IDS Clearing House (Fraunhofer ISST)

Esta é a implementação de referência oficial de código aberto desenvolvida pelo *Fraunhofer Institute*.

- **Por que acelera o MVP:** Ela é escrita em Rust e projetada em contêineres Docker de fácil implantação. Ela foi desenhada especificamente para receber mensagens do protocolo DSP (*Dataspace Protocol*) vindas do Eclipse EDC.

- **A Tecnologia de Fundo:** Para garantir a imutabilidade exigida pelo IDS RAM, ela utiliza internamente uma tecnologia de banco de dados de registro distribuído (DLT) leve ou uma estrutura de árvore criptográfica (como uma *Merkle Tree*) em cima de um banco de dados convencional, eliminando a necessidade de subir uma rede blockchain complexa no MVP.

Hyperledger Fabric / Besu (Abordagem DLT Customizada)

Se a governança da RNDT preferir utilizar uma infraestrutura de Ledger que o governo brasileiro já domina (como a rede do Serpro/Dataprev ou o ecossistema do DREX):

- **Por que acelera o MVP:** Pode-se expor uma API REST simples à frente de um canal privado do **Hyperledger Fabric**. Toda vez que o conector conclui uma transferência, ele faz um `post` nessa API, registrando o log na blockchain governamental.

Exemplo Real: Como a Clearing House Opera na Fase 4 (MVP)

Vamos ver o fluxo real de como o conector do Provedor (ex: **DNIT**) registra a entrega de uma base de pavimentação para uma **Concessionária** sem vaziar o dado para a Clearing House.

Passo 1: A Transferência Ocorre

O conector do DNIT envia o arquivo de pavimentação direto para a Concessionária via mTLS. Assim que o último bloco (*chunk*) de dados é recebido com sucesso pela Concessionária, o conector do DNIT calcula o **Hash SHA-256** do arquivo enviado.

Passo 2: O Envio do Recibo de Entrega (Payload JSON-LD)

O *Data Plane* do conector do DNIT monta uma mensagem de log e a envia para a URL da **Clearing House central da RNDT**. O payload que entra no livro de registros é puramente metadado criptográfico:

```
{
  "@context": [
    "https://w3id.org/idsa/core/context.jsonld",
    "https://w3id.org/transport/rndt/context.jsonld"
  ],
  "@type": "ids:LogMessage",
  "id": "urn:rndt:log:tx-987654321",
  "issuerConnector": "urn:govbr:cnpj:04898488000177", // DNIT
  "issued": "2026-06-20T10:25:00Z",

  "correlationMessage": "urn:uuid:contract-agreement-dnit-ccr-001", // ID do Contrato da Fase 3

  "payloadDescription": {
    "recipientConnector": "urn:govbr:cnpj:12345678000199", // Concessionária
    "assetId": "asset-dnit-pavimentacao-2026",
    "transferStatus": "COMPLETED",
    "bytesTransferred": 1073741824, // 1 GB
    // O Hash do dado bruto (Garante integridade sem revelar o conteúdo)
    "dataHashSHA256": "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855"
  },

  "proof": {
    "type": "JsonWebSignature2020",
    "verificationMethod": "https://rndt.gov.br/certs/dnit.crt", // Assinado com e-CNPJ do DNIT
    "jws": "eyJ...assinatura_digital_do_dnit_provando_o_envio..."
  }
}
```

Passo 3: O Registro Imutável e a Auditoria

A Clearing House recebe esse JSON-LD, valida que a assinatura pertence ao conector oficial do DNIT (usando a validação ICP-Brasil integrada ao Keycloak) e carimba o tempo na transação (*Timestamping*). Ela salva esse bloco na sua estrutura imutável e devolve um recibo.

Por que a Clearing House é vital para o Ministério dos Transportes?

Imagine que, três meses após o MVP rodar, a Concessionária processe o DNIT ou reclame na ANTT alegando que: *"O DNIT nunca nos entregou a base de pavimentação de 2026, por isso não conseguimos cumprir a meta contratual de reparo da BR"*.

1. O auditor do Ministério dos Transportes acessa a **Clearing House**.
2. Ele busca pelo ID do Contrato (urn:uuid:contract-agreement-dnit-ccr-001).
3. O sistema exibe o log imutável, assinado pelo próprio conector do DNIT, confirmando que 1 GB de dados foi transmitido com sucesso no dia 20 de junho de 2026 às 10:25h, batendo com o Hash do arquivo original.

A disputa jurídica é encerrada em minutos por meio de **prova matemática**.

Recomendação para o MVP

Suba a imagem Docker da **IDS Clearing House do Fraunhofer ISST** em modo simplificado (usando persistência local segura com criptografia de disco). Isso resolve o requisito de não-repúdio do IDS RAM com esforço de codificação zero, dando à RNDDT o nível de auditoria exigido pelas melhores práticas de governança digital do setor público.

4.5 A App Store (Catálogo de Serviços)

Conceito de "Bring Algorithms to the Data"

O conceito de **App Store (ou Catálogo de Serviços)** dentro do modelo IDS RAM resolve um dos maiores dilemas da governança pública e da segurança da informação: **como gerar inteligência sobre dados altamente sensíveis ou volumosos sem precisar distribuí-los ou centralizá-los em um único grande data lake?**

Em arquiteturas tradicionais, se o Ministério dos Transportes ou uma concessionária quisesse fazer uma análise complexa cruzando dados da PRF, ANTT e DNIT, a solução seria extrair cópias dessas bases de dados (gerando arquivos imensos) e enviá-las para um ambiente terceiro. Isso quebra a soberania e abre margem para vazamentos.

O Catálogo de Serviços inverte essa lógica através do paradigma da **Computação Próxima ao Dado** (*Bring Algorithms to the Data*). Em vez de mover o dado até o código, **move-se o código até o dado**.

Como funciona a Engrenagem Técnica?

A App Store da RNDDT funciona como um repositório seguro e centralizado de **imagens de contêineres Docker homologadas**.

1. **A Publicação do App:** Um cientista de dados do Ministério dos Transportes escreve um script em **Python (Pandas, Scikit-Learn)** ou **R** para fazer um cálculo de previsão de desgaste asfáltico ou severidade de sinistros. Ele empacota esse código em um contêiner Docker e o envia para o Catálogo de Serviços (App Store) da rede.
2. **A Homologação:** O comitê de segurança da RNDDT analisa o código para garantir que ele faz *apenas* o cálculo prometido (evitando códigos maliciosos que tentem roubar senhas ou exportar a base bruta). O app é assinado digitalmente pelo Ministério como "Seguro".
3. **A Execução Distribuída (Fase 4 Avançada):** Quando uma Concessionária deseja rodar aquela análise na base restrita da PRF, o conector da Concessionária faz o download do contêiner da App Store e o envia para ser executado **dentro da infraestrutura isolada** do conector da PRF.

O aplicativo roda na memória do servidor da PRF, consome as tabelas brutas locais, gera o resultado consolidado e envia de volta para a Concessionária **apenas o indicador ou o gráfico final**, destruindo o contêiner logo em seguida. O dado bruto nunca saiu do prédio da PRF.

3 Exemplos Reais Aplicados ao Cenário de Transportes

Exemplo 1: Algoritmo de Classificação de Severidade de Sinistros (Machine Learning)

- **O Problema:** A base detalhada de sinistros da PRF possui dados sensíveis (placas, nomes de condutores, CPFs) protegidos pela LGPD, mas pesquisadores precisam entender os fatores que causam acidentes fatais.
- **O App na Vitrine:** Um app chamado `rndt-sinistros-classifier:v1`. Ele carrega um modelo de árvore de decisão treinado para ler o histórico, identificar os fatores críticos (clima, tipo de BR, velocidade) e gerar uma matriz de correlação.
- **A Execução:** O app entra no ambiente da PRF, roda o modelo direto no banco de dados local da polícia, gera os pesos estatísticos de cada causa e cospe apenas o resultado limpo (ex: "*Fator Climático Chuva + Curva Acentuada aumenta em 42% a chance de óbito*"). O pesquisador recebe o indicador científico, e os CPFs permanecem intocados.

Exemplo 2: Pipeline de Anonimização e LGPD On-The-Fly

- **O Problema:** Uma startup de logística quer consumir dados de fluxo de veículos nas praças de pedágio da ANTT para prever tráfego, mas a ANTT não pode expor a identificação das tags dos veículos.
- **O App na Vitrine:** Um app de processamento chamado `rndt-anonymizer-streaming`.
- **A Execução:** Ele atua acoplado ao *Data Plane* da ANTT na Fase 4. À medida que os dados de passagem dos eixos saem do banco da ANTT, o contêiner intercepta o fluxo, aplica uma função de *hashing* criptográfico irreversível nas IDs das tags e remove colunas sensíveis. O dado sai "sanitizado" do conector da ANTT e chega limpo ao conector da startup.

Exemplo 3: Cálculo do Índice de Condição da Manutenção (ICM) Automatizado

- **O Problema:** O DNIT possui imagens brutas e dados de sensores de pavimentação colhidos por carros de varredura. São terabytes de dados brutos de engenharia rodoviária, inviáveis de serem baixados pela internet por uma agência reguladora regional.
- **O App na Vitrine:** Um app de visão computacional ou processamento pesado chamado `dnit-icm-calculator`.
- **A Execução:** O app é disparado diretamente no Storage do DNIT onde estão os terabytes de imagens de rachaduras de asfalto. O contêiner roda o processamento pesado localmente usando o hardware do DNIT e devolve para o solicitante apenas uma tabela CSV consolidada de poucas linhas com a nota do ICM de cada rodovia (ex: "BR-101 KM 50: Nota 8"). O tráfego de rede caiu de terabytes de imagens para poucos kilobytes de texto.

Ferramentas e Implementações para Acelerar o MVP

Eclipse EDC Data App Orchestrator (O Caminho Nativo)

Como você já planejou o uso do **Eclipse Dataspace Components (EDC)** para os conectores, a comunidade do Eclipse mantém extensões e padrões de referência voltados especificamente para a execução de **Data Apps**.

- **Como acelera o MVP:** O *Data Plane* do EDC possui uma arquitetura extensível que pode se conectar diretamente ao **Docker Daemon** ou a um microsserviço local de orquestração.
- **A Implementação Prática:** Em vez de criar um portal complexo, a sua "App Store" no MVP pode ser simplesmente um **Docker Registry Privado** (usando o **Harbor** ou o próprio **Docker Registry open-source**) controlado pelo Ministério dos Transportes.

O Fluxo no MVP:

1. O cientista de dados envia a imagem do script Python para o Harbor do Ministério (`registry.rndt.gov.br/apps/calculador-icm:v1`).
2. Na Fase 3, o conector do Consumidor envia um contrato solicitando a execução do app.
3. O conector EDC do Provedor aceita, faz o `docker pull` da imagem diretamente do Harbor do Ministério, instancia o contêiner localmente, passa os parâmetros, coleta o JSON de retorno e destrói o contêiner.

FIWARE Marketplace & Camara APIs (Foco em Ecossistemas de Dados)

Se você busca uma solução que já traga uma interface visual pronta de "vitrine de aplicativos" integrada ao catálogo de dados, a fundação **FIWARE** possui componentes específicos para isso (muito utilizados em projetos europeus de transporte e cidades inteligentes).

- **Keyrock + Business API Ecosystem (BAE):** É uma suíte open-source que cria uma "loja virtual" onde desenvolvedores podem cadastrar tanto conjuntos de dados quanto serviços/aplicativos analíticos.
- **Por que acelera:** Ela já vem com o gerenciamento de direitos digitais pronto. O analista consegue enxergar o aplicativo em uma interface web amigável, clicar em "Adquirir" e disparar os tokens de autorização para o conector executar o contêiner correspondente.

Desenhando o componente de forma "Low-Code" para o MVP

Se você quiser colocar essa engrenagem de pé em poucos dias com ferramentas que a equipe de TIC já domina amplamente, a melhor estratégia é acoplar o **Apache Camel** (que já está na Fase 1) como o controlador do Data App no lado do Provedor.

Em vez de criar códigos complexos de infraestrutura, você usa uma rota do Apache Camel dentro do conector do Provedor para gerenciar a vida do script Python:

```
# Rota do Apache Camel no Provedor para Executar o App da Store
- from: "direct:executar-data-app"
  steps:
    - to: "http://registry.rndt.gov.br/apps/script_analise" # Puxa o script/parâmetros
    - to: "exec:python?args=/tmp/script.py" # Roda o script localmente no sandbox
    - marshal: { json: {} } # Transforma o output impresso em JSON limpo
    - to: "direct:enviar-de-volta-ao-consumidor" # Devolve para a Fase 4 do EDC
```

Recomendação Pragmática para o MVP da RNDT

Para o MVP, **não gaste energia construindo uma interface gráfica de "App Store"**. Adote a abordagem de infraestrutura simplificada:

1. Suba um servidor **Harbor (Open-Source Container Registry)** na nuvem do Ministério dos Transportes. Ele será a sua App Store física, onde ficam guardadas as imagens Docker homologadas e assinadas pelo governo.
2. Configure o conector EDC do Provedor para aceitar comandos de execução baseados nessas imagens específicas.

Isso reduz o desenvolvimento a **zero linhas de código de interface**, resolve o problema de repositório utilizando uma ferramenta padrão de mercado (Harbor) e valida o modelo de computação soberana na Fase 4 com total segurança e velocidade de entrega.

O Fluxo Real: Do Código ao Bit de Retorno

Imagine o cenário do **Exemplo 1**: O Ministério dos Transportes (Consumidor) enviou o aplicativo `rndt-sinistros-classifier` para rodar dentro da infraestrutura da PRF (Provedora) para calcular a matriz de correlação de acidentes fatais.

Passo 1: O Isolamento e a Execução (Lado Provedor)

1. O conector EDC da PRF instancia o contêiner Docker do Data App em uma rede interna isolada (*sandbox*).
2. O conector da PRF atua como um "Proxy". Ele puxa os dados brutos de sinistros do banco de dados interno da PRF e os injeta em uma porta local do contêiner (geralmente via uma API REST local ou montagem de volume em memória).
3. O script Python dentro do contêiner lê o dado bruto, processa a matriz de correlação e gera o resultado.

Passo 2: A Formatação do Retorno pelo Script

Para que o conector entenda o que o script gerou, o código do aplicativo deve salvar o resultado em um formato padrão combinado (geralmente **JSON** ou **CSV**).

O script Python finaliza sua execução fazendo exatamente isto:

```
# Fim do script Python dentro do container
resultado_matriz = matriz_correlacao.to_json()
```

```
# O script entrega o resultado na porta de saída que o conector EDC está escutando
return_to_connector(resultado_matriz, content_type="application/json")
```

Passo 3: O Conector do Provedor "Embala" o Resultado

Assim que o contêiner Docker termina o cálculo e entrega o JSON na porta de saída, o conector da PRF intercepta esse payload.

Aqui ocorre a **mágica da soberania**: o conector da PRF pega esse JSON gerado pelo script, joga na memória do seu *Data Plane* e o transforma em um **Payload de Resposta Temporário**. Ele aplica:

- A criptografia de canal (mTLS).
- A assinatura digital do e-CNPJ da PRF.
- A vinculação com o ID do contrato original da Fase 3 (para provar que aquele resultado pertence àquela requisição autorizada).

Passo 4: A Transferência Segura de Volta

O *Data Plane* da PRF faz o streaming desse JSON criptografado direto para o *Data Plane* do conector do Ministério dos Transportes (Consumidor). Assim que o envio termina, a PRF **destrói o contêiner Docker** e apaga qualquer cache na memória local.

Passo 5: O Recebimento no Consumidor

O conector do Ministério dos Transportes recebe o pacote embalado, valida a assinatura da PRF, remove a casca criptográfica e entrega o JSON purificado direto na tela do analista ou no sistema de BI (como um painel do PowerBI ou Superset) do Ministério.

Resumo do Fluxo de Entrada e Saída

Podemos resumir o fluxo de dados em uma linha de transmissão clara:

```
[Consumidor] --(Envia o App Docker)--> [Conector Provedor]
                                     |
                                     (Roda no Sandbox)
                                     v
[Consumidor] <--(JSON Criptografado)-- [Retorno em JSON] <== (Dado Bruto + Script)
```

O contêiner funciona como um "cozinheiro trancado na dispensa": ele entra, usa os ingredientes sigilosos da PRF, monta o prato pronto (JSON de resultado) e passa o prato pela fresta da porta para o conector. O conector embala o prato em uma caixa térmica lacrada (criptografia) e entrega para o Ministério dos Transportes. Os ingredientes brutos nunca deixaram a dispensa da PRF.

5. MACROFLUXO DO PROVEDOR DE DADOS

5.1 Visão Geral do Macrofluxo

O **Macrofluxo do Provedor de Dados** descreve o ciclo de vida completo pelo qual uma instituição brasileira — seja órgão público (PRF, DNIT, ANTT, SENATRAN) ou concessionária privada — transforma seus dados internos em ativos negociáveis dentro da RNDT. Este fluxo não é uma mera sequência técnica de upload e download: é um processo de **governança digital** que materializa os princípios de soberania de dados, confiança descentralizada e interoperabilidade semântica definidos pelo IDS RAM.

O Macrofluxo é dividido em **quatro fases interdependentes**, cada uma correspondendo a uma camada funcional do IDS RAM:

Fase	Nome	Objetivo Estratégico	Camada IDS RAM
Fase 1	Preparação e Ingestão	Transformar dados brutos em ativos semânticos prontos para circulação	Camada de Dados + Sistema
Fase 2	Publicação e Catalogação	Anunciar a existência do ativo na rede sem expor seu conteúdo	Camada de Funcionalidade + Processo
Fase 3	Negociação de Contratos	Estabelecer regras jurídicas e técnicas de uso vinculativas	Camada de Negócio + Processo
Fase 4	Transferência e Auditoria	Executar a entrega sob controle de políticas e registrar prova de entrega	Camada de Sistema + Processo

A diferença crucial entre este fluxo e um pipeline tradicional de ETL ou API REST é que **o controle sobre o dado não se extingue na entrega**. O Provedor mantém a soberania através de contratos digitais ODRL, aplicação forçada de políticas (Policy Enforcement) e registros imutáveis na Clearing House.

5.2 Fase 1: Preparação e Ingestão dos Dados

5.2.1 O Propósito da Fase 1

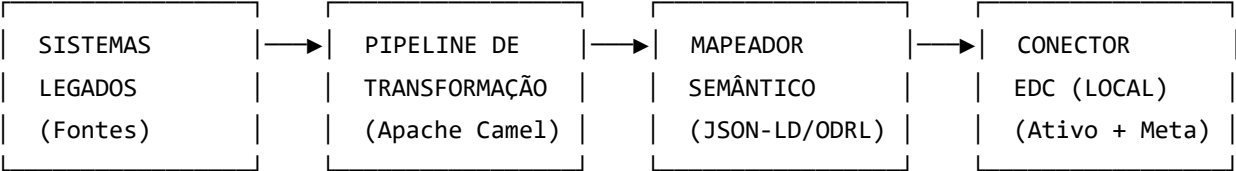
A Fase 1 é onde o dado deixa de ser um arquivo estático numa pasta de servidor ou num banco legado e se torna um **Ativo de Dados (Data Asset)** — uma entidade semântica identificável, descritível e transferível dentro do ecossistema da RNDT. Esta fase ocorre inteiramente dentro da infraestrutura do Provedor, antes que qualquer comunicação externa aconteça.

Os objetivos desta fase são:

- **Extrair** dados de sistemas legados (SIGTAP, SIGM, bancos Oracle, arquivos CSV históricos, sensores IoT de pavimentação).
- **Transformar** os dados para padrões semânticos internacionais (DATEX II, GTFS, mobilityDCAT-ap).
- **Catalogar internamente** o ativo no conector local, associando metadados ricos e políticas de uso.
- **Preparar o canal de transferência** configurando o *Data Plane* do conector para servir aquele ativo sob demanda.

5.2.2 Arquitetura da Fase 1: Do Sistema Legado ao Conector

A arquitetura da Fase 1 pode ser visualizada como uma esteira de processamento com quatro estações:



Estação 1: Sistemas Legados (Fontes de Dados)

Os Provedores da RNDT possuem bases heterogêneas. Exemplos reais:

Instituição	Sistema Legado	Tipo de Dado	Formato Bruto
PRF	Sistema de Registro de Acidentes	Sinistros rodoviários	Banco Oracle / XML proprietário
DNIT	SIGM / Sistema de Pavimentação	Condição de infraestrutura	Shapefiles / CSV de varredura
ANTT	Sistema de Autorizações	Fretes e concessões	Banco SQL / PDFs
SENATRAN	RENACH / RENAVAM	Condutores e veículos	Banco legado / API SOAP

Estação 2: Pipeline de Transformação (Apache Camel)

O **Apache Camel** atua como o orquestrador de integração na Fase 1. Ele é configurado para:

- **Consumir** dados das fontes legadas via conectores nativos (JDBC para Oracle, SFTP para arquivos, MQTT para sensores).
- **Roteiar** os dados através de transformações (DataWeave, XSLT, scripts Python).
- **Produzir** o dado transformado em um endpoint que o *Data Plane* do EDC consome.

Exemplo Real: Rota Camel para Dados de Sinistros da PRF

```
# Rota Apache Camel para transformar XML da PRF em DATEX II
- from: "sql:SELECT * FROM sinistros WHERE data_ocorrencia > CURRENT_DATE - INTERVAL '1' DAY"
  steps:
    - to: "bean:prfSinistroTransformer" # Converte campos internos para DATEX II
    - marshal:
      json:
        library: Jackson
        prettyPrint: true
    - to: "file:/rndt/staging/prf/sinistros?fileName=sinistros_${date:now:yyyyMMdd}.json"
    - to: "direct:registrar-asset-edc"
```

Estação 3: Mapeador Semântico (JSON-LD / ODRL)

Após a transformação técnica, o dado precisa de uma **camada de significado**. O Mapeador Semântico converte o arquivo transformado em um **Ativo de Dados** descrito por metadados DCAT e regras ODRL.

Exemplo: Metadados do Ativo "Sinistros PRF 2026"

```

{
  "@context": [
    "https://w3id.org/idsa/core/context.jsonld",
    "http://w3id.org/transport/mobilityDCAT-ap/context.jsonld"
  ],
  "@id": "urn:rndt:asset:prf-sinistros-2026",
  "@type": "ids:DataAsset",
  "dct:title": "Sinistros e Ocorrências Rodoviárias Consolidadas (2026)",
  "dct:description": "Dados geocodificados de acidentes em rodovias federais, atualizados diariamente",
  "dct:publisher": {
    "@id": "urn:govbr:cnpj:00394502000144",
    "foaf:name": "Polícia Rodoviária Federal"
  },
  "dcat:theme": "http://publications.europa.eu/resource/authority/data-theme/TRAN",
  "dcat:keyword": ["sinistros", "acidentes", "segurança viária", "datex2", "lgpd"],
  "ids:representation": {
    "@type": "ids:DataRepresentation",
    "ids:instance": {
      "@type": "ids:Artifact",
      "ids:fileName": "sinistros_20260620.json",
      "ids:format": "application/json"
    }
  },
  "odrl:hasPolicy": {
    "@type": "odrl:Agreement",
    "odrl:target": "urn:rndt:asset:prf-sinistros-2026",
    "odrl:assigner": "urn:govbr:cnpj:00394502000144",
    "odrl:permission": [
      {
        "odrl:action": "odrl:use",
        "odrl:constraint": [
          {
            "odrl:leftOperand": "odrl:purpose",
            "odrl:operator": "odrl:eq",
            "odrl:rightOperand": "pesquisa-cientifica"
          },
          {
            "odrl:leftOperand": "odrl:elapsedTime",
            "odrl:operator": "odrl:leq",
            "odrl:rightOperand": "P30D"
          }
        ]
      }
    ],
    "odrl:prohibition": [
      {
        "odrl:action": "odrl:derive",
        "odrl:constraint": {

```

```

        "odrl:leftOperand": "odrl:recipient",
        "odrl:operator": "odrl:neq",
        "odrl:rightOperand": "orgao-publico"
    }
}
]
}
}

```

O que este JSON-LD expressa:

- **Identidade:** O ativo possui um URI único (urn:rndt:asset:prf-sinistros-2026).
- **Proveniência:** A PRF é a publicadora oficial (via CNPJ).
- **Semântica:** Os temas e palavras-chave seguem vocabulários controlados (mobilityDCAT-ap).
- **Política de Uso (ODRL):**
 - **Permissão:** Uso permitido para fins de pesquisa científica, por no máximo 30 dias.
 - **Proibição:** É proibido derivar novos dados se o consumidor não for um órgão público.

Estação 4: Registro no Conector EDC (Local)

O conector EDC do Provedor recebe o metadado do ativo e o armazena em seu **Asset Index** interno (geralmente um banco PostgreSQL ou um cache em memória). A partir deste momento, o ativo está pronto para ser exposto ao mundo externo, mas ainda não está visível na rede.

API REST do EDC para Registro de Ativo:

POST /api/v1/data/assets

Content-Type: application/json

```

{
  "asset": {
    "id": "asset-prf-sinistros-2026",
    "properties": {
      "name": "Sinistros PRF 2026",
      "description": "Dados de acidentes em rodovias federais",
      "contentType": "application/json"
    }
  },
  "dataAddress": {
    "type": "HttpData",
    "baseUrl": "http://internal-prf-api:8080/sinistros/2026",
    "proxyPath": "true"
  }
}

```

5.2.3 O Data Plane: O Canal de Transferência

O *Data Plane* do EDC é o componente responsável por **servir o dado bruto** quando uma transferência autorizada ocorre na Fase 4. Na Fase 1, ele é configurado para:

- **Apontar** para a fonte de dados transformada (HTTP interno, bucket S3, banco SQL).
- **Aplicar** criptografia de canal (TLS 1.3) e assinatura digital (e-CNPJ ICP-Brasil).
- **Registrar** métricas de transferência para a Clearing House.

Configuração do Data Plane para o Ativo da PRF:

```
# Configuração do Data Plane no EDC
dataPlane:
  transferTypes:
    - type: "HttpProxy"
      sourceTypes: ["HttpData"]
      sinkTypes: ["HttpProxy"]
  publicApi:
    baseUrl: "https://edc.prf.gov.br/api/v1/public"
  security:
    mtls:
      enabled: true
      keystore: "/etc/ssl/certs/prf-ecnpj.p12"
      truststore: "/etc/ssl/certs/icp-brasil-truststore.jks"
```

5.2.4 Checklist de Conclusão da Fase 1

Antes de avançar para a Fase 2, o Provedor deve garantir:

- ☐ O dado bruto foi extraído do sistema legado com sucesso.
- ☐ A transformação para o padrão semântico (DATEX II, GTFS, etc.) foi validada.
- ☐ O metadado DCAT/mobilityDCAT-ap foi gerado e está sintaticamente correto.
- ☐ A política ODRL foi revisada pelo jurídico da instituição e reflete as regras de LGPD.
- ☐ O Data Plane do conector EDC está configurado e respondendo na URL pública.
- ☐ O ativo foi registrado no Asset Index local do conector.

5.3 Fase 2: Publicação e Catalogação

5.3.1 O Propósito da Fase 2

A Fase 2 é o momento em que o Provedor **torna o ativo descobrível** na RNDT sem, contudo, transferir o dado em si. É a fase da **vitrine inteligente**: o Metadata Broker central (CKAN) colhe os metadados do conector do Provedor e os indexa para busca por Consumidores.

Os objetivos desta fase são:

- **Publicar** o catálogo de metadados do conector Provedor em um endpoint DSP (*Dataspace Protocol*).
- **Permitir que o Broker central (CKAN)** colha (*harvest*) esses metadados periodicamente.
- **Garantir** que o Consumidor consiga descobrir o ativo, compreender seu significado e saber como solicitá-lo — tudo isso sem nunca ter acesso ao conteúdo bruto.

5.3.2 O Protocolo DSP na Fase 2

O **Dataspace Protocol (DSP)** é o idioma que os conectores utilizam para conversar sobre catálogos e contratos. Na Fase 2, o Provedor expõe um endpoint DSP que responde a requisições de catálogo.

Endpoint DSP do Provedor:

```
GET https://edc.prf.gov.br/api/v1/dsp/catalog
Headers:
Authorization: Bearer <token_JWT_do_Keycloak>
```

Resposta DSP (JSON-LD):

```
{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@id": "https://edc.prf.gov.br/catalog",
  "@type": "dcat:Catalog",
  "dct:title": "Catálogo de Dados da PRF na RNDT",
  "dct:description": "Conjuntos de dados de sinistros, operações e infraestrutura rodoviária federal",
  "dcat:dataset": [
    {
      "@id": "urn:rndt:asset:prf-sinistros-2026",
      "@type": "dcat:Dataset",
      "dct:title": "Sinistros e Ocorrências Rodoviárias Consolidadas (2026)",
      "dcat:distribution": {
        "@type": "dcat:Distribution",
        "dct:format": "application/json",
        "dcat:accessService": {
          "@type": "dcat:DataService",
          "dcat:endpointUrl": "https://edc.prf.gov.br/api/v1/dsp",
          "dct:conformsTo": "https://w3id.org/dspace/v0.8"
        }
      }
    }
  ]
}
```

5.3.3 Colheita pelo Metadata Broker (CKAN)

O CKAN atua como o **Broker central** da RNDT. Ele não armazena dados brutos — apenas metadados. Sua função na Fase 2 é:

1. **Agendar** colheitas periódicas dos conectores dos Provedores.
2. **Parsear** as respostas DSP (JSON-LD) e convertê-las para o formato interno do CKAN.
3. **Indexar** os metadados no Apache Solr para busca full-text.
4. **Exibir** uma interface web onde Consumidores possam pesquisar e descobrir ativos.

Configuração da Fonte de Colheita no CKAN:

```
# Configuração interna do CKAN (RNDT Central)
harvest_source:
  id: "prf-catalogo"
  title: "Catálogo de Metadados da PRF"
  url: "https://edc.prf.gov.br/api/v1/dsp/catalog"
  type: "dcat_jsonld"
  frequency: "DAILY"
  config:
    default_tags: ["sinistros", "prf", "rodovias-federais"]
    default_groups: ["seguranca-viaria"]
    validator_profiles: ["mobilityDCAT_AP"]
```

5.3.4 A Experiência do Consumidor na Descoberta

Um analista do DNIT ou uma startup de logística acessa o portal <https://rndt.gov.br> e digita na busca: **"sinistros BR-101"**.

1. O Solr do CKAN retorna o ativo `urn:rndt:asset:prf-sinistros-2026`.
2. A interface exibe:
 - **Título:** Sinistros e Ocorrências Rodoviárias Consolidadas (2026)
 - **Publicador:** Polícia Rodoviária Federal (CNPJ: 00.394.502/0001-44)
 - **Formato:** JSON (DATEX II)
 - **Frequência:** Diária
 - **Política:** Uso restrito a pesquisa científica, expira em 30 dias.
3. Um botão **"Solicitar Acesso via Conector"** captura o `dcat:accessService` e inicia a Fase 3 no conector do Consumidor.

Ponto Crítico: Até este momento, nenhum byte de dado de sinistros saiu da infraestrutura da PRF. O Consumidor sabe que o dado existe, o que ele significa e como pedi-lo — mas não o possui.

5.3.5 Checklist de Conclusão da Fase 2

- ☐ O endpoint DSP do conector Provedor está acessível publicamente via HTTPS.
- ☐ O catálogo DSP retorna metadados válidos em JSON-LD.
- ☐ O CKAN central configurou a fonte de colheita para o Provedor.
- ☐ A colheita foi executada com sucesso e os metadados aparecem no portal RNDT.
- ☐ O ativo é encontrável via busca no portal.

5.4 Fase 3: Negociação de Contratos

5.4.1 O Propósito da Fase 3

A Fase 3 é o **coração jurídico e técnico** da soberania de dados. É aqui que o Provedor e o Consumidor estabelecem um **Contrato de Uso de Dados (*Contract Agreement*)** — um documento digital vinculativo que define, de forma inequívoca, o que pode ser feito com o dado, por quem, por quanto tempo e sob quais condições.

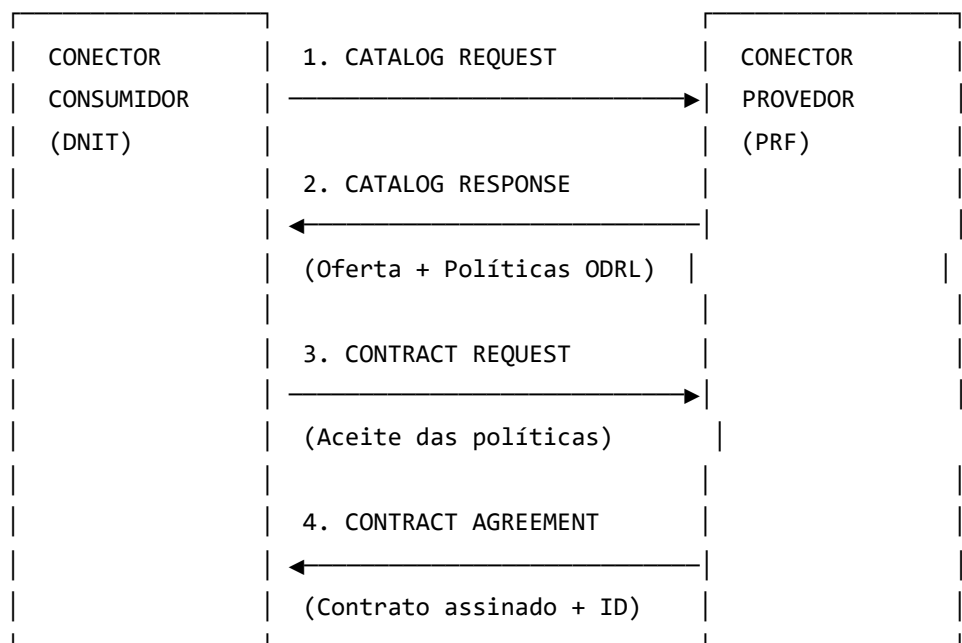
Este contrato não é um PDF assinado digitalmente de forma tradicional. É um **grafo semântico ODRL** assinado criptograficamente por ambas as partes, interpretável automaticamente pelo conector e registrado como prova na Clearing House.

Os objetivos desta fase são:

- **Ofertar** o ativo com políticas de uso explícitas (ODRL).
- **Negociar** automaticamente ou semi-automaticamente os termos com o Consumidor.
- **Assinar** digitalmente o contrato usando certificados ICP-Brasil (e-CNPJ).
- **Gerar** um `contractId` único que vincula o acesso ao cumprimento das políticas.

5.4.2 O Fluxo de Negociação DSP

A negociação segue o protocolo DSP em quatro passos:



5.4.3 Passo 1: Catalog Request

O Consumidor (DNIT) solicita ao Provedor (PRF) o catálogo de ofertas disponíveis. Esta requisição já inclui a identidade do Consumidor (via token JWT do Keycloak/Omejdn).

POST https://edc.prpf.gov.br/api/v1/dsp/catalog/request

Content-Type: application/json

Authorization: Bearer <token_dnit>

```
{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@type": "dspace:CatalogRequestMessage",
  "dspace:filter": {
    "odrl:leftOperand": "odrl:purpose",
    "odrl:operator": "odrl:eq",
    "odrl:rightOperand": "planejamento-infraestrutura"
  }
}
```

5.4.4 Passo 2: Catalog Response (Oferta + Políticas)

O Provedor responde com uma **Oferta de Contrato (*Contract Offer*)** que inclui o ativo desejado e as políticas ODRL aplicáveis.

```

{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@id": "urn:rndt:offer:prf-sinistros-2026-dnit-001",
  "@type": "odrl:Offer",
  "odrl:target": "urn:rndt:asset:prf-sinistros-2026",
  "odrl:assigner": "urn:govbr:cnpj:00394502000144",
  "odrl:permission": [
    {
      "odrl:action": "odrl:use",
      "odrl:constraint": [
        {
          "odrl:leftOperand": "odrl:purpose",
          "odrl:operator": "odrl:eq",
          "odrl:rightOperand": "planejamento-infraestrutura"
        },
        {
          "odrl:leftOperand": "odrl:elapsedTime",
          "odrl:operator": "odrl:leq",
          "odrl:rightOperand": "P90D"
        },
        {
          "odrl:leftOperand": "odrl:spatial",
          "odrl:operator": "odrl:eq",
          "odrl:rightOperand": "BR-101"
        }
      ]
    }
  ],
  "odrl:prohibition": [
    {
      "odrl:action": "odrl:transfer",
      "odrl:constraint": {
        "odrl:leftOperand": "odrl:recipient",
        "odrl:operator": "odrl:neq",
        "odrl:rightOperand": "urn:govbr:cnpj:04898488000177"
      }
    }
  ]
}

```

Interpretação da Oferta:

- **Alvo:** Ativo de sinistros da PRF 2026.
- **Permissão:** Uso permitido para planejamento de infraestrutura, por 90 dias, restrito à BR-101.
- **Proibição:** É proibido transferir os dados para qualquer entidade que não seja o DNIT (CNPJ: 04.898.488/0001-77).

5.4.5 Passo 3: Contract Request (Aceite pelo Consumidor)

O Consumidor analisa a oferta e, se concorda com os termos, envia uma requisição de contrato aceitando as políticas.

```
{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@type": "dspace:ContractRequestMessage",
  "dspace:offer": {
    "@id": "urn:rndt:offer:prf-sinistros-2026-dnit-001",
    "odrl:target": "urn:rndt:asset:prf-sinistros-2026"
  },
  "dspace:callbackAddress": "https://edc.dnit.gov.br/api/v1/dsp/callback"
}
```

5.4.6 Passo 4: Contract Agreement (Contrato Assinado)

O Provedor valida a requisição, verifica a identidade do Consumidor via DAPS/Keycloak, e gera o **Contrato de Acordo (Contract Agreement)**. Este documento é assinado digitalmente por ambas as partes.

```
{
  "@context": [
    "https://w3id.org/dspace/v0.8/context.json",
    "https://w3id.org/idsa/core/context.jsonld"
  ],
  "@id": "urn:rndt:contract:prf-dnit-sinistros-20260620-001",
  "@type": "odrl:Agreement",
  "odrl:target": "urn:rndt:asset:prf-sinistros-2026",
  "odrl:assigner": "urn:govbr:cnpj:00394502000144",
  "odrl:assignee": "urn:govbr:cnpj:04898488000177",
  "odrl:permission": [ /* ... mesmas permissões da oferta ... */ ],
  "odrl:prohibition": [ /* ... mesmas proibições da oferta ... */ ],
  "dcat:startDate": "2026-06-20T10:00:00Z",
  "dcat:endDate": "2026-09-20T10:00:00Z",
  "ids:providerSignature": {
    "type": "JsonWebSignature2020",
    "verificationMethod": "https://rndt.gov.br/certs/prf.crt",
    "jws": "eyJhbGciOiJSUzI1NiIsImtpZCI6ImR1aXQta2V5LTl1wMjYifQ..."
  },
  "ids:consumerSignature": {
    "type": "JsonWebSignature2020",
    "verificationMethod": "https://rndt.gov.br/certs/dnit.crt",
    "jws": "eyJhbGciOiJSUzI1NiIsImtpZCI6ImR1aXQta2V5LTl1wMjYifQ..."
  }
}
```

Elementos Juridicamente Vinculativos:

- **Identificação das Partes:** Via CNPJ e certificados ICP-Brasil.
- **Escopo Temporal:** Vigência de 90 dias (20/06/2026 a 20/09/2026).
- **Assinaturas Digitais:** Ambas as partes assinam com JWS (JSON Web Signature), garantindo não-repúdio.
- **Hash do Contrato:** O `contractId` é um hash SHA-256 do conteúdo do contrato, tornando-o imutável.

5.4.7 O Papel do DAPS/Omejdn na Fase 3

Durante a negociação, o conector do Provedor consulta o **Omejdn DAPS** (ou Keycloak) para:

1. **Validar** o token JWT do Consumidor.
2. **Verificar** se o CNPJ do Consumidor possui atributos necessários (ex: `rndtStatus: "Homologado"`).
3. **Confirmar** que o certificado ICP-Brasil do Consumidor não foi revogado (via LCR do ITI).

Se qualquer validação falhar, a negociação é abortada automaticamente pelo conector.

5.4.8 Checklist de Conclusão da Fase 3

- ☐ O Consumidor descobriu o ativo via Broker e iniciou a negociação.
- ☐ O Provedor enviou uma Oferta ODRL clara e juridicamente revisada.
- ☐ O Consumidor aceitou os termos sem alterações (ou negociou alterações via contraproposta).
- ☐ O Contrato de Acordo foi gerado com ID único e assinado por ambas as partes.
- ☐ As assinaturas digitais foram validadas contra a ICP-Brasil.
- ☐ O contrato foi armazenado localmente em ambos os conectores.

5.5 Fase 4: Transferência e Auditoria

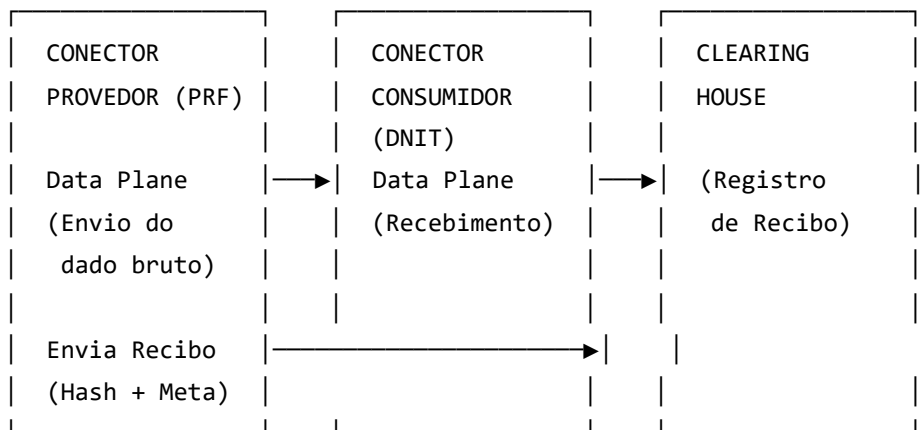
5.5.1 O Propósito da Fase 4

A Fase 4 é a **execução material** do contrato. É aqui que o dado bruto finalmente transita do Provedor para o Consumidor — mas não de forma irrestrita. A transferência ocorre sob **controle rigoroso de políticas**, com criptografia de ponta a ponta, e gera um **registro imutável de prova de entrega** na Clearing House.

Os objetivos desta fase são:

- **Transferir** o ativo do *Data Plane* do Provedor para o *Data Plane* do Consumidor.
- **Aplicar** as políticas do contrato em tempo real (ex: expirar acesso após 90 dias).
- **Registrar** um recibo criptográfico na Clearing House para auditoria e não-repúdio.
- **Garantir** que o dado bruto nunca passe pelo Broker ou pela Clearing House (arquitetura de dados federada).

5.5.2 O Fluxo de Transferência DSP



5.5.3 Passo 1: Transfer Request

O Consumidor solicita a transferência do ativo, apresentando o `contractId` válido.

```
POST https://edc.prf.gov.br/api/v1/dsp/transferprocess
```

```
Content-Type: application/json
```

```
Authorization: Bearer <token_dnit>
```

```
{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@type": "dspace:TransferRequestMessage",
  "dspace:agreementId": "urn:rndt:contract:prf-dnit-sinistros-20260620-001",
  "dspace:callbackAddress": "https://edc.dnit.gov.br/api/v1/dsp/callback",
  "dct:format": "application/json"
}
```

5.5.4 Passo 2: Validação e Preparação pelo Provedor

O conector do Provedor executa uma série de validações antes de liberar o dado:

1. **Validação do Contrato:** O `contractId` existe e está dentro do prazo de vigência?
2. **Validação de Identidade:** O token JWT do Consumidor bate com o `assignee` do contrato?
3. **Validação de Políticas:** O consumo está dentro das restrições ODRL (ex: acesso apenas à BR-101)?
4. **Preparação do Data Plane:** O endpoint interno do ativo é resolvido e o canal mTLS é estabelecido.

5.5.5 Passo 3: Transferência do Dado Bruto

O *Data Plane* do Provedor estabelece um canal criptografado (mTLS) com o *Data Plane* do Consumidor e inicia o streaming do dado.

Características da Transferência:

- **Protocolo:** HTTP/2 com mTLS (certificados ICP-Brasil de ambas as partes).

- **Criptografia:** TLS 1.3 com cipher suites fortes (AES-256-GCM).
- **Formato:** O dado trafega no formato nativo do ativo (JSON, XML, Protobuf, etc.).
- **Tamanho:** Pode variar de kilobytes (GTFS estático) a terabytes (imagens de pavimentação).

Exemplo: Streaming de Dados de Sinistros

```
# Requisição do Data Plane do Consumidor
GET https://dataplane.prf.gov.br/api/v1/public/stream/urn:rndt:asset:prf-sinistros-2026
Headers:
  X-Contract-Id: urn:rndt:contract:prf-dnit-sinistros-20260620-001
  Authorization: Bearer <data_plane_token>

# Resposta (streaming)
HTTP/1.1 200 OK
Content-Type: application/json
Transfer-Encoding: chunked

{"id": "sin-001", "br": "BR-101", "km": 45.2, "tipo": "colisao-frontal", ...}
{"id": "sin-002", "br": "BR-101", "km": 67.8, "tipo": "capotamento", ...}
...
```

5.5.6 Passo 4: Geração do Recibo de Transferência

Assim que a transferência é concluída, o conector do Provedor calcula o **hash SHA-256** do arquivo/dado transmitido e monta um recibo de entrega.

```

{
  "@context": [
    "https://w3id.org/idsa/core/context.jsonld",
    "https://w3id.org/transport/rndt/context.jsonld"
  ],
  "@id": "urn:rndt:log:tx-prf-dnit-20260620-001",
  "@type": "ids:LogMessage",
  "issuerConnector": "urn:govbr:cnpj:00394502000144",
  "issued": "2026-06-20T10:35:00Z",
  "correlationMessage": "urn:rndt:contract:prf-dnit-sinistros-20260620-001",
  "payloadDescription": {
    "recipientConnector": "urn:govbr:cnpj:04898488000177",
    "assetId": "urn:rndt:asset:prf-sinistros-2026",
    "transferStatus": "COMPLETED",
    "bytesTransferred": 157286400,
    "dataHashSHA256": "a3f5c8e9d2b1...7f4e6d5c8b9a2",
    "startTime": "2026-06-20T10:30:00Z",
    "endTime": "2026-06-20T10:35:00Z"
  },
  "proof": {
    "type": "JsonWebSignature2020",
    "verificationMethod": "https://rndt.gov.br/certs/prf.crt",
    "jws": "eyJhbGciOiJSUzI1NiIsImtpZCI6InByZi1rZXktMjAyNiJ9..."
  }
}

```

5.5.7 Passo 5: Registro na Clearing House

O recibo é enviado para a **Clearing House** da RNDT (implementação Fraunhofer ISST ou Hyperledger Fabric), onde:

1. A assinatura digital da PRF é validada.
2. O hash do dado é armazenado em estrutura imutável (Merkle Tree ou DLT).
3. Um **timestamp** criptográfico é aplicado (via servidor TSA ou DLT).
4. Um recibo de confirmação é devolvido ao Provedor.

Resposta da Clearing House:

```
{
  "@id": "urn:rndt:clearing:receipt-20260620-8842",
  "@type": "ids:ReceiptMessage",
  "issued": "2026-06-20T10:35:05Z",
  "issuerConnector": "urn:rndt:clearinghouse:central",
  "correlationMessage": "urn:rndt:log:tx-prf-dnit-20260620-001",
  "status": "REGISTERED",
  "timestampProof": "2026-06-20T10:35:05Z",
  "merkleRoot": "0x7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa5d0..."
}
```

5.5.8 O Papel da App Store na Fase 4 Avançada

Se o contrato envolver a execução de um **Data App** (computação próxima ao dado), a Fase 4 se ramifica:

- 1. O Consumidor solicita a execução do app `rndt-sinistros-classifier:v1`.
- 2. O Provedor faz `docker pull` do registro Harbor central.
- 3. O contêiner é executado em sandbox no *Data Plane* do Provedor.
- 4. O app consome o dado bruto localmente e gera um resultado consolidado.
- 5. Apenas o resultado (JSON/CSV) é transferido ao Consumidor.
- 6. O contêiner é destruído e o recibo de execução é registrado na Clearing House.

5.5.9 Checklist de Conclusão da Fase 4

- ☐ O Consumidor apresentou um `contractId` válido e dentro da vigência.
- ☐ O Provedor validou identidade, contrato e políticas ODRL.
- ☐ O dado foi transferido via canal criptografado (mTLS).
- ☐ O hash SHA-256 do dado transmitido foi calculado.
- ☐ O recibo de transferência foi assinado pelo Provedor e enviado à Clearing House.
- ☐ A Clearing House confirmou o registro imutável com timestamp.
- ☐ Se aplicável, o Data App foi executado em sandbox e destruído após uso.

5.6 Síntese do Macrofluxo: O Ciclo Fechado de Soberania

O Macrofluxo do Provedor de Dados na RNDT não é uma simples pipeline de ETL. É um **sistema de governança digital completo** que garante:

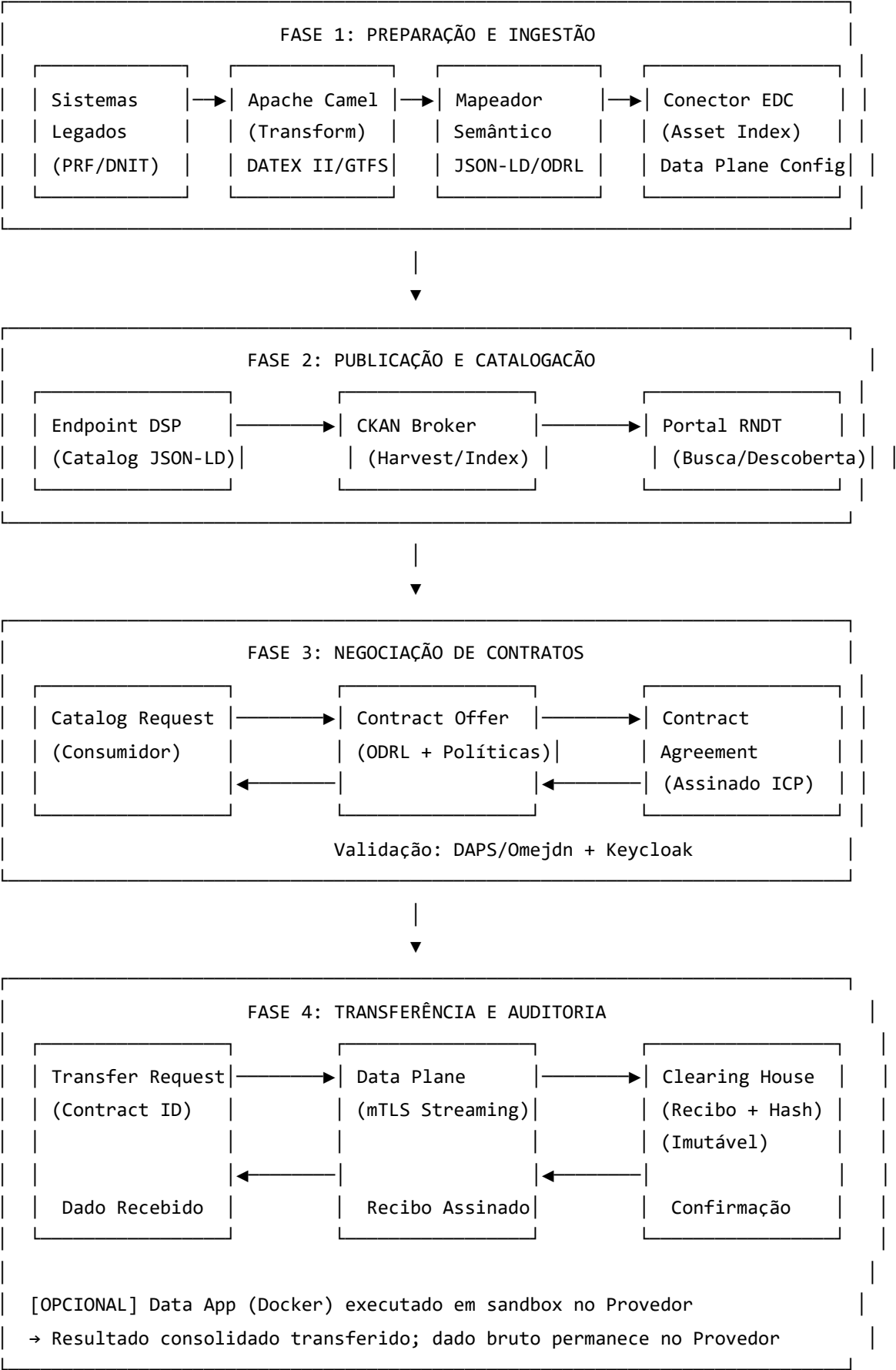
Princípio	Como é Materializado no Macrofluxo
Soberania de Dados	O Provedor define as regras ODRL na Fase 1 e as aplica na Fase 4. O dado nunca é entregue sem contrato.
Confiança Descentralizada	A identidade é validada via ICP-Brasil (Fase 3) e a entrega é auditada criptograficamente (Fase 4).

Princípio	Como é Materializado no Macrofluxo
Interoperabilidade	Padrões semânticos (DATEX II, GTFS, mobilityDCAT-ap) garantem que o dado seja compreendido por qualquer participante.
Não-Reúdio	A Clearing House registra hashes assinados, eliminando disputas sobre "quem entregou o quê e quando".
LGPD Compliance	Políticas ODRL podem restringir uso por finalidade, tempo e destinatário; Data Apps permitem análise sem exposição de dados pessoais.

O Fluxo em uma Linha:

Fase 1 (Preparar) → Fase 2 (Anunciar) → Fase 3 (Negociar) → Fase 4 (Transferir + Auditar) → Soberania Mantida

5.7 Diagrama Consolidado do Macrofluxo



6. MACROFLUXO DO CONSUMIDOR DE DADOS

6.1 Visão Geral do Macrofluxo do Consumidor

Se o Provedor é quem **oferta** a riqueza, o Consumidor é quem **gera valor** a partir dela. O Macrofluxo do Consumidor de Dados na RNDT não é apenas uma sequência de downloads: é um processo de **descoberta inteligente, negociação estratégica e integração produtiva** que transforma dados brutos de transporte em insights acionáveis, produtos e serviços.

O Consumidor na RNDT pode ser:

Tipo de Consumidor	Exemplos	Motivação Principal
Órgãos Públicos	DNIT, ANTT, SENATRAN, IBGE, Ministério da Saúde	Formulação de políticas públicas, planejamento infraestrutural, regulação
Concessionárias Privadas	CCR, Arteris, EcoRodovias	Otimização de operações, manutenção preditiva, segurança viária
Startups de Mobilidade	Waze (Google), Uber, 99, Loggi	Roteirização, previsão de demanda, otimização de frotas
Institutos de Pesquisa	IPEA, universidades, laboratórios de transporte	Pesquisa científica, modelagem epidemiológica, estudos de impacto
Seguradoras	Porto Seguro, BB Seguridade	Precificação de risco, análise atuarial, prevenção de sinistros

O Macrofluxo do Consumidor é **espelhado e complementar** ao do Provedor, dividido em quatro fases:

Fase	Nome	Objetivo Estratégico
Fase 1	Descoberta e Seleção	Encontrar e avaliar ativos relevantes no catálogo central
Fase 2	Due Diligence e Qualificação	Verificar metadados, políticas e compatibilidade técnica
Fase 3	Negociação e Contratação	Estabelecer contratos ODRL vinculativos com os Provedores
Fase 4	Consumo e Valorização	Receber dados, integrar aos sistemas internos e gerar produtos/serviços

6.2 Fase 1: Descoberta e Seleção de Ativos

6.2.1 O Ponto de Entrada: O Portal RNDT

O Consumidor inicia sua jornada no **portal central da RNDT** (<https://rndt.gov.br>), hospedado sobre a instância CKAN do Metadata Broker. A interface oferece:

- **Busca semântica** por palavras-chave, temas (mobilityDCAT-ap), formatos e publicadores.
- **Facetamento dinâmico** por órgão (PRF, DNIT, ANTT), tipo de dado (sinistros, pavimentação, tráfego), formato (DATEX II, GTFS, CSV) e frequência de atualização.
- **Visualização de metadados** completos sem exposição do dado bruto.
- **Indicadores de qualidade** (completude, atualidade, conformidade com padrões).

Exemplo de Busca no Portal RNDT:

```
Busca: "acidentes BR-116 Rio de Janeiro 2025"
|
├─ Resultado 1: Sinistros Rodoviários Consolidados (PRF)
|   ├─ Cobertura espacial: BR-116, RJ, SP, MG
|   ├─ Formato: DATEX II (JSON)
|   ├─ Atualização: Diária
|   ├─ Política: Uso público, restrição comercial
|   └─ [Solicitar Acesso]
|
├─ Resultado 2: Dados de Pavimentação e IRI (DNIT)
|   ├─ Cobertura espacial: BR-116, trecho km 0-300
|   ├─ Formato: CSV + Shapefile
|   ├─ Atualização: Trimestral
|   └─ [Solicitar Acesso]
|
└─ Resultado 3: Fluxo de Veículos em Praças de Pedágio (ANTT)
    ├─ Cobertura espacial: Concessionárias da BR-116
    ├─ Formato: Protobuf (GTFS-RT)
    ├─ Atualização: Tempo real (5 min)
    └─ [Solicitar Acesso]
```

6.2.2 A API de Descoberta para Integração Programática

Para sistemas integrados e plataformas de IA, o CKAN expõe uma **API REST de catálogo** que permite busca automatizada:

```
GET https://rndt.gov.br/api/3/action/package_search
?q=sinistros+BR-116&fq=organization:prf+tags:datex2
```

Resposta JSON (simplificada):

```

{
  "success": true,
  "result": {
    "count": 3,
    "results": [
      {
        "id": "urn:rndt:asset:prf-sinistros-2026",
        "title": "Sinistros e Ocorrências Rodoviárias Consolidadas",
        "organization": {
          "name": "prf",
          "title": "Polícia Rodoviária Federal"
        },
        "tags": [
          {"name": "sinistros"},
          {"name": "datex2"},
          {"name": "seguranca-viaria"}
        ],
        "resources": [
          {
            "format": "JSON",
            "mimetype": "application/json",
            "access_service": {
              "endpoint_url": "https://edc.prf.gov.br/api/v1/dsp",
              "conforms_to": "https://w3id.org/dspace/v0.8"
            }
          }
        ]
      }
    ]
  }
}

```

6.2.3 O Conector do Consumidor: Configuração Inicial

Antes de negociar, o Consumidor deve ter seu próprio **conector EDC** operacional. A configuração mínima inclui:

```
# Configuração do Conector EDC do Consumidor (DNIT)
connector:
  name: "RNDT-Connector-DNIT"
  id: "urn:govbr:cnpj:04898488000177"

  identity:
    daps:
      url: "https://daps.rndt.gov.br"
      clientId: "urn:govbr:cnpj:04898488000177"
      keystore: "/etc/ssl/certs/dnit-ecnpj.p12"

  catalog:
    brokerUrl: "https://edc.rndt.gov.br/api/v1/dsp/catalog"

  dataPlane:
    publicApi:
      baseUrl: "https://dataplane.dnit.gov.br/api/v1/public"
    sink:
      type: "HttpData"
      baseUrl: "https://repositorio-interno.dnit.gov.br/rndt"
```

6.2.4 Checklist de Conclusão da Fase 1

- ☐ O Consumidor identificou ativos relevantes via portal ou API.
- ☐ Os metadados mobilityDCAT-ap foram avaliados (cobertura, formato, frequência).
- ☐ O conector EDC do Consumidor está operacional e autenticado no DAPS.
- ☐ O Consumidor mapeou os `endpointUrIs` dos Provedores de interesse.

6.3 Fase 2: Due Diligence e Qualificação

6.3.1 Análise de Metadados e Proveniência

Antes de consumir, o Consumidor deve avaliar a **qualidade e confiabilidade** do ativo. Os metadados DCAT permitem:

- **Proveniência:** Quem publicou? (CNPJ verificável via ICP-Brasil)
- **Atualidade:** Quando foi a última atualização? (`dct:modified`)
- **Cobertura espacial/temporal:** O dado cobre a região e o período de interesse?
- **Conformidade:** O dado segue padrões reconhecidos (DATEX II, GTFS)?

Exemplo de Avaliação de Proveniência:

```
{
  "dct:publisher": {
    "@id": "urn:govbr:cnpj:00394502000144",
    "foaf:name": "Polícia Rodoviária Federal",
    "verification": {
      "certificateChain": "ICP-Brasil v5",
      "status": "VALID",
      "expiration": "2027-03-15"
    }
  }
}
```

6.3.2 Análise de Políticas ODRL

Esta é a etapa crítica de **qualificação jurídica**. O Consumidor deve verificar se consegue cumprir as políticas de uso do Provedor.

Matriz de Decisão do Consumidor:

Política ODRL do Provedor	Pergunta de Due Diligence	Decisão do Consumidor
odrl:purpose = "pesquisa-cientifica"	Meu uso é pesquisa ou comercial?	Se comercial → buscar outro ativo ou negociar
odrl:elapsedTime <= P30D	Posso processar o dado em 30 dias?	Se não → solicitar extensão ou aceitar risco
odrl:spatial = "BR-101"	Preciso de outras rodovias?	Se sim → buscar ativo complementar
odrl:prohibition:derive	Preciso criar derivativos?	Se sim → impossível, buscar alternativa

6.3.3 Verificação Técnica de Compatibilidade

O Consumidor deve garantir que seu sistema consegue:

- **Consumir o formato:** Se o ativo é DATEX II, o sistema possui parser XML/JSON?
- **Processar o volume:** Se são 10GB diários de GTFS-RT, a infraestrutura aguenta?
- **Atender à latência:** Se é tempo real (Protobuf), o sistema suporta streaming?

6.3.4 Checklist de Conclusão da Fase 2

- ☐ A proveniência do ativo foi verificada (CNPJ + certificado ICP-Brasil).
- ☐ As políticas ODRL foram analisadas e consideradas exequíveis.
- ☐ A compatibilidade técnica (formato, volume, latência) foi confirmada.
- ☐ O Consumidor decidiu prosseguir para a negociação.

6.4 Fase 3: Negociação e Contratação

6.4.1 Iniciando a Negociação via DSP

O Consumidor inicia a negociação enviando uma **Catalog Request** ao endpoint DSP do Provedor, como detalhado na Fase 3 do Provedor (Seção 5.4). Aqui, focamos na **perspectiva do Consumidor**.

6.4.2 Avaliação da Oferta e Tomada de Decisão

Ao receber a **Contract Offer** do Provedor, o Consumidor (ou seu sistema automatizado) avalia:

```
# Pseudocódigo de avaliação automatizada de oferta
def avaliar_oferta(offer, necessidades_consumidor):
    politicas = offer['odr1:permission']

    for politica in politicas:
        if not validar_proposito(politica, necessidades_consumidor.purpose):
            return "REJEITAR: Propósito incompatível"

        if not validar_prazo(politica, necessidades_consumidor.prazo_maximo):
            return "REJEITAR: Prazo insuficiente"

        if not validar_restricoes_geograficas(politica, necessidades_consumidor.area_interesse):
            return "REJEITAR: Cobertura espacial inadequada"

    return "ACEITAR: Oferta compatível com necessidades"
```

6.4.3 Contra-Proposta (Opcional)

Se as políticas não atendem completamente, o Consumidor pode enviar uma **contra-proposta**:

```

{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@type": "dspace:ContractRequestMessage",
  "dspace:offer": {
    "@id": "urn:rndt:offer:prf-sinistros-2026-dnit-001",
    "odrl:target": "urn:rndt:asset:prf-sinistros-2026",
    "odrl:permission": [
      {
        "odrl:action": "odrl:use",
        "odrl:constraint": [
          {
            "odrl:leftOperand": "odrl:purpose",
            "odrl:operator": "odrl:eq",
            "odrl:rightOperand": "planejamento-infraestrutura"
          },
          {
            "odrl:leftOperand": "odrl:elapsedTime",
            "odrl:operator": "odrl:leq",
            "odrl:rightOperand": "P180D"
          }
        ]
      }
    ],
    "odrl:prohibition": []
  },
  "dspace:callbackAddress": "https://edc.dnit.gov.br/api/v1/dsp/callback"
}

```

Nota: O Provedor pode aceitar, rejeitar ou contra-propor. A negociação é iterativa até o acordo ou abandono.

6.4.4 Armazenamento do Contrato Assinado

Após a assinatura, o Consumidor armazena o **Contract Agreement** localmente:

Armazenamento no conector EDC do Consumidor

POST /api/v1/data/contractagreements

Content-Type: application/json

```
{
  "id": "urn:rndt:contract:prf-dnit-sinistros-20260620-001",
  "assetId": "urn:rndt:asset:prf-sinistros-2026",
  "provider": "urn:govbr:cnj:00394502000144",
  "consumer": "urn:govbr:cnj:04898488000177",
  "contractStartDate": "2026-06-20T10:00:00Z",
  "contractEndDate": "2026-09-20T10:00:00Z",
  "policies": { /* ... ODRL completo ... */ }
}
```

6.4.5 Checklist de Conclusão da Fase 3

- ☐ O Consumidor enviou Catalog Request e recebeu Contract Offer.
- ☐ As políticas foram avaliadas quanto à exequibilidade jurídica e técnica.
- ☐ O contrato foi assinado digitalmente (JWS com e-CNPJ).
- ☐ O Contract Agreement foi armazenado no conector local.
- ☐ O contractId está ativo e dentro da vigência.

6.5 Fase 4: Consumo e Valorização

6.5.1 Iniciando a Transferência

Com o contrato assinado, o Consumidor solicita a transferência do ativo:

POST https://edc.prf.gov.br/api/v1/dsp/transferprocess

Content-Type: application/json

Authorization: Bearer <token_dnit>

```
{
  "@context": "https://w3id.org/dspace/v0.8/context.json",
  "@type": "dspace:TransferRequestMessage",
  "dspace:agreementId": "urn:rndt:contract:prf-dnit-sinistros-20260620-001",
  "dspace:callbackAddress": "https://edc.dnit.gov.br/api/v1/dsp/callback",
  "dct:format": "application/json"
}
```

6.5.2 Recebimento e Processamento do Dado

O *Data Plane* do Consumidor recebe o streaming de dados e o persiste em seu repositório interno:

```
# Exemplo: Pipeline de ingestão no Consumidor (DNIT)
import pandas as pd
from sqlalchemy import create_engine

# 1. Receber streaming do Data Plane EDC
dados_sinistros = receber_streaming_dataplane(
    contract_id="urn:rndt:contract:prf-dnit-sinistros-20260620-001",
    endpoint="https://dataplane.prf.gov.br/api/v1/public/stream"
)

# 2. Parse DATEX II para DataFrame
df_sinistros = parse_datex2_to_dataframe(dados_sinistros)

# 3. Enriquecer com dados próprios do DNIT (pavimentação)
df_pavimentacao = carregar_dados_pavimentacao("BR-101")
df_integrado = df_sinistros.merge(
    df_pavimentacao,
    on=["br", "km"],
    how="left"
)

# 4. Gerar indicador de prioridade de manutenção
df_integrado["prioridade"] = calcular_prioridade_manutencao(df_integrado)

# 5. Persistir em data warehouse interno
engine = create_engine("postgresql://dw.dnit.gov.br/rndt")
df_integrado.to_sql("prioridade_manutencao_br101", engine, if_exists="replace")
```

6.5.3 Geração de Valor: Produtos e Serviços

O dado consumido da RNDT alimenta **produtos e serviços** do Consumidor:

Consumidor	Dado Consumido	Produto/Serviço Gerado
DNIT	Sinistros (PRF) + Pavimentação (próprio)	Mapa de prioridade de manutenção rodoviária
Concessionária CCR	Tráfego em tempo real (ANTT) + Meteorologia	Sistema de alerta de congestionamento para usuários
Startup Loggi	Fluxo de pedágio (ANTT) + GTFS urbano	Otimização de rotas de entrega em tempo real
IPEA	Sinistros (PRF) + Dados socioeconômicos	Estudo de correlação entre infraestrutura e mortalidade
Porto Seguro	Histórico de sinistros (PRF, anonimizado)	Modelo de precificação de seguro por trecho rodoviário

6.5.4 Compliance Contínuo e Revogação

O Consumidor deve garantir **compliance contínuo** com as políticas ODRL:

- **Monitoramento de prazo:** O contrato expira em 90 dias. O sistema deve bloquear acesso automático.
- **Restrição de propósito:** O dado não pode ser usado para fins não autorizados (ex: comercial se restrito a pesquisa).
- **Proibição de redistribuição:** O dado não pode ser repassado a terceiros.

Exemplo: Verificação de Expiração Automática

```
from datetime import datetime

def verificar_vigencia_contrato(contract_agreement):
    end_date = datetime.fromisoformat(contract_agreement['dcat:endDate'].replace('Z', '+00:00'))

    if datetime.now() > end_date:
        # Revogar acesso e deletar dados locais
        revogar_acesso(contract_agreement['@id'])
        deletar_dados_locais(contract_agreement['odrl:target'])
        notificar_administrador("Contrato expirado. Dados removidos conforme ODRL.")
        return False

    return True
```

6.5.5 Checklist de Conclusão da Fase 4

- ☐ A transferência foi solicitada com `contractId` válido.
- ☐ O dado foi recebido via streaming criptografado (mTLS).
- ☐ O dado foi integrado aos sistemas internos do Consumidor.
- ☐ Produtos ou serviços de valor foram gerados a partir do dado.
- ☐ O compliance com políticas ODRL está sendo monitorado automaticamente.
- ☐ Há plano de remoção de dados ao término do contrato.

6.6 Ferramentas para o Consumidor

Para acelerar o **Macrofluxo do Consumidor** na RNDT / EMDS e evitar o desenvolvimento de código proprietário complexo, a equipe técnica do órgão ou empresa consumidora pode se beneficiar de um conjunto robusto de ferramentas e componentes de código aberto (*open-source*).

Diferente do Provedor, que precisa gerenciar bancos de dados legados e pipelines de limpeza pesados, o foco do Consumidor está na **automação do consumo**, na **orquestração de chamadas de API** e na

visualização analítica.

Abaixo estão as ferramentas prontas e consolidadas para acelerar cada fase do fluxo do Consumidor:

6.6.1 Para as Fases 1 e 2: Automação de Busca e Negociação

Para não precisar programar requisições HTTP manuais que assinam contratos JSON-LD na unha, o Consumidor utiliza clientes de gerenciamento integrados:

A. EDC Management API + Swagger UI

O próprio conector **Eclipse Dataspace Connector (EDC)** do Consumidor expõe uma API de gerenciamento interna (*Management API*).

A comunidade fornece imagens prontas que sobem o **Swagger UI** apontado para essa API. O analista do Ministério dos Transportes não escreve código; ele entra em uma interface web de documentação, cola o ID do ativo da PRF que pegou no CKAN, cola o token do Keycloak e clica em *"Execute"*. O conector assume a máquina de estados, conversa com a PRF via *Dataspace Protocol* e fecha o contrato de forma automatizada.

B. TSG Consumer UI (Telematics Services Group)

Esta é uma das interfaces administrativas open-source mais leves e eficientes para o lado consumidor do ecossistema EDC.

É um painel web em formato de *Dashboard* que se conecta à API de controle do seu conector. Ele permite navegar visualmente pelos catálogos remotos dos provedores, iniciar negociações clicando em botões de aceite de termos de licença e listar todos os contratos válidos ou expirados que o seu órgão possui.

6.6.2 Para a Fase 3: Criação de Data Apps para a App Store

Se o Consumidor precisa enviar um código para rodar no ambiente do Provedor, ele se beneficia de ferramentas de empacotamento padrão de mercado:

A. Docker + Cookiecutter Data Science

Para criar o aplicativo analítico que irá navegar até o dado da PRF, o cientista de dados do Consumidor não precisa inventar uma estrutura.

O **Cookiecutter Data Science** é um framework de código aberto que gera uma estrutura de pastas padronizada para projetos em Python (Pandas/Scikit-Learn). Combinado com o **Docker**, o analista escreve o script de análise e, com um comando único (`docker build`), gera o contêiner perfeitamente compatível para ser enviado ao repositório de aplicativos (Harbor).

6.6.3 Para a Fase 4: Recepção, Ingestão e Inteligência Analítica

Uma vez que o *Data Plane* do conector EDC do Consumidor recebe o JSON descriptografado vindo da PRF, o dado precisa ser armazenado e transformado em valor de negócio imediatamente.

A. Apache Camel (Consumer Pipeline)

O Camel não serve apenas para o Provedor. No Consumidor, ele é colocado na saída do conector EDC para atuar como o **orquestrador de chegada**.

Uma rota simples em Camel intercepta o JSON gerado pelo app de sinistros, faz um parse rápido e o insere automaticamente em um banco de dados relacional local ou dispara um alerta por e-mail/Slack se um indicador de risco crítico for atingido.

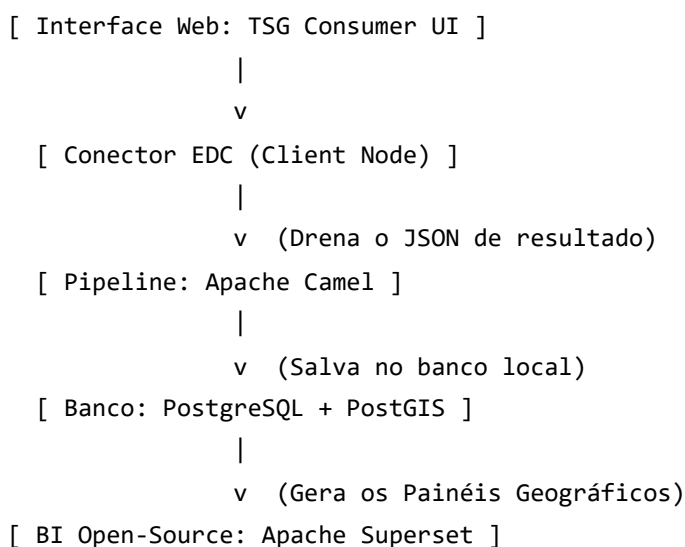
B. Apache Superset / Metabase (Visualização Gráfica)

São as ferramentas de Business Intelligence (BI) de código aberto mais robustas do mercado atual.

Em vez de gastar licenças caras de ferramentas proprietárias para o MVP, o Ministério instala o **Metabase** ou o **Apache Superset** conectado ao banco de dados que o Apache Camel está alimentando. Em poucas horas, os resultados analíticos das consultas executadas de forma soberana na PRF viram gráficos de linha, mapas de calor e painéis de tomada de decisão para o Ministro dos Transportes.

6.6.4 Resumo da Arquitetura do Consumidor para o MVP

Para colocar a ponta consumidora de pé com o menor esforço e máxima eficiência, o desenho de componentes recomendável é:



Utilizando essa esteira, o Consumidor se beneficia 100% do ecossistema de dados sem construir código de infraestrutura de rede, aproveitando as ferramentas visuais prontas para focar no que realmente importa: a geração de políticas públicas e inteligência sobre o transporte nacional.

7. IDENTIDADE, SEGURANÇA E CONFIABILIDADE

7.1 Estratégia de Identidade no MVP

No desenho de um Produto Mínimo Viável (MVP) para a **RNDT**, é uma estratégia arquitetural perfeitamente válida e pragmática postergar a implementação de DIDs (*Decentralized Identifiers*) e **utilizar exclusivamente a infraestrutura de chaves públicas da ICP-Brasil** para a autenticação e segurança.

O **IDS RAM (Reference Architecture Model)** é agnóstico em relação à tecnologia de criptografia de base, desde que os pilares de **confiança mútua**, **autenticidade** e **não-repúdio** sejam garantidos.

7.1.1 Como funciona o MVP baseado apenas em ICP-Brasil?

No ecossistema IDS clássico, o conector utiliza um componente chamado **DAPS (Dynamic Attribute Provisioning Service)** para validar o token de identidade do outro conector. Para o MVP, a validação de identidade máquina-a-máquina (mTLS) pode ser simplificada substituindo a tecnologia de credenciais verificáveis (DIDs) por certificados digitais tradicionais:

1. **Autenticação de Rede (mTLS):** Cada órgão cooperante (PRF, DNIT, ANTT) e cada concessionária privada utiliza um **Certificado de Equipamento/Aplicação (A3 ou SSL/TLS EV)** emitido por uma Autoridade Certificadora homologada pela ICP-Brasil.
2. **Assinatura de Contratos (Fase 3):** No momento em que os conectores fecham o *Contract Agreement*, o payload JSON-LD é assinado digitalmente utilizando a chave privada do certificado ICP-Brasil do órgão.
3. **Validação da Confiança:** Em vez de consultar uma rede *blockchain* ou um barramento de identidades descentralizadas para resolver um DID, o conector do Provedor simplesmente valida a cadeia de certificação do Consumidor contra a **LCR (Lista de Certificados Revogados)** oficial do Instituto Nacional de Tecnologia da Informação (ITI). Se o certificado for ICP-Brasil válido, a máquina assume que o participante é confiável.

7.1.2 Vantagens dessa abordagem para o MVP

- **Aproveitamento Jurídico Legal:** Os certificados ICP-Brasil possuem validade jurídica inquestionável e presunção de veracidade legal no Brasil (segundo a MP nº 2.200-2). Para um ambiente governamental, isso elimina barreiras de compliance jurídico logo no início do projeto.

- **Redução drástica da complexidade técnica:** Implementar DIDs exige configurar resolvedores de identidade (*Universal Resolvers*), gerenciar chaves descentralizadas e, muitas vezes, integrar com redes de registro distribuído (DLT). Utilizar ICP-Brasil reduz o esforço da Fase 2 e 3 para protocolos de segurança web tradicionais (X.509).
- **Velocidade de Entrega:** A equipe de desenvolvimento foca no que gera valor imediato para o negócio do transporte: as esteiras do Apache Camel (Fase 1), o mapeamento DATEX II e a negociação de contratos de dados (Fase 3/4).

7.1.3 O Gargalo (Por que o IDS RAM prefere DIDs a longo prazo?)

Embora funcione perfeitamente para o MVP, é importante mapear o que o ecossistema perde ao não usar DIDs, para planejar a evolução pós-MVP:

- **Granularidade de Atributos Dinâmicos:** O certificado ICP-Brasil prova *quem* o órgão é (ex: "Polícia Rodoviária Federal"), mas ele não consegue provar, de forma automatizada e dinâmica, *atributos temporais de infraestrutura* (ex: "Este conector específico possui a homologação de segurança nível 2 da RNDT válida até dezembro"). Com DIDs e *Verifiable Credentials* (VCs), a emissão e revogação desses sub-atributos são instantâneas.
- **Custo de Escala:** Gerar e renovar certificados ICP-Brasil para centenas de sistemas, servidores e aplicações de terceiros que desejem se plugar à RNDT pode gerar um custo financeiro e administrativo recorrente elevado. Os DIDs possuem custo de geração e manutenção virtualmente zero.

7.1.4 Recomendação de Arquitetura para o seu Roadmap

Para avançar com segurança:

1. **No MVP:** Configure o plano de controle dos conectores (como o Eclipse EDC) para autenticar os participantes usando certificados **ICP-Brasil (X.509 v3)** injetados no protocolo de comunicação.
2. **Design "Forward-Compatible":** Deixe o campo de identificação de nós estruturado de forma que, no futuro, a string do CNPJ/Certificado possa ser substituída por uma URI de DID (ex: `did:govbr:cnpj:00000000000100`) sem quebrar o banco de dados das Definições de Contrato.

7.2 Credenciais Verificáveis (VCs)

No **IDS RAM (Reference Architecture Model)**, as **Credenciais Verificáveis (Verifiable Credentials - VCs)** cabem especificamente na camada de **Governança de Identidade e Confiança** (dentro da perspectiva de Segurança e de Sistema).

As VCs são o mecanismo utilizado para prover o que o IDS chama de **Dynamic Attribute Provisioning (Provisionamento Dinâmico de Atributos)**. Em vez de apenas provar *quem* você é (autenticação básica), as VCs provam *o que você está autorizado a fazer* ou *quais selos de conformidade você possui*.

Elas atuam diretamente nas **Fases 2 e 3** do macrofluxo, servindo de base para o conector do Provedor decidir se exibe um contrato no catálogo ou se aprova a negociação automática.

7.2.1 Elas são mandatórias segundo o IDS RAM?

Não, elas não são mandatórias para o funcionamento básico de um espaço de dados, mas são mandatórias para atingir o nível máximo de maturidade, interoperabilidade e automação do IDS.

Como discutido na decisão de arquitetura do seu MVP, é perfeitamente possível substituir o ecossistema de VCs por infraestruturas tradicionais de chaves públicas (como certificados **X.509 da ICP-Brasil**) em uma fase inicial.

No entanto, o IDS RAM preconiza o uso de VCs pelas seguintes razões:

- **Descentralização:** Evita que o Provedor precise consultar um banco de dados centralizado toda vez que quiser checar se o Consumidor é confiável. O Consumidor carrega suas próprias provas na "carteira digital" do conector.
- **Granularidade e Ciclo de Vida:** Certificados tradicionais são rígidos e demorados para revogar. VCs podem ser emitidas para atestar propriedades específicas (ex: "Empresa Homologada na RNDT") e revogadas em milissegundos por meio de listas de revogação criptográficas baseadas em DIDs.

7.2.2 Exemplo Real de Aplicação de VCs no Ecossistema de Transportes

Imagine o cenário de federação de dados entre a **PRF (Provedora)** e uma **Concessionária de Rodovias Privada (Consumidora)** que deseja acessar o *Asset* de dados sensíveis de sinistros.

Para que a negociação na Fase 3 ocorra sem intervenção humana, o ecossistema utiliza três atores do modelo de Credenciais Verificáveis do W3C:

[Emissor: Ministério dos Transportes]

|

(Emite a VC assinada)

v

[Detentor: Conector da Concessionária] === Apresenta a VP ===> [Verificador: Conector da PRF]

1. O Emissor (*Issuer*)

O **Ministério dos Transportes** (ou a própria governança central da RNDT) atua como o emissor confiável. Ele faz uma auditoria jurídica na Concessionária e emite uma **Credencial Verificável (VC)** assinada digitalmente atestando que aquela empresa é uma operadora oficial de rodovias homologada no ecossistema nacional.

2. O Detentor (*Holder*)

O conector EDC da Concessionária recebe essa VC (um arquivo JSON-LD assinado criptograficamente) e a armazena em seu cofre de identidades interno.

3. O Verificador (*Verifier*) e a Execução na Fase 3

Quando o conector da Concessionária inicia a negociação de contrato com o conector da PRF, ele não envia apenas um "usuário e senha". Ele envelope sua VC em uma **Apresentação Verificável (Verifiable Presentation - VP)** e envia junto com a proposta de contrato.

O conector da PRF (atuando como Verificador) executa o seguinte algoritmo automatizado:

1. Ele lê a assinatura criptográfica da VC e valida que ela foi gerada de fato pela chave pública do Ministério dos Transportes (relação de confiança).
2. Ele analisa o conteúdo interno da credencial (o *Claim*).
3. Ele confronta com a política **ODRL** do seu *Asset*, que exige: *"Apenas conectores que possuam o atributo status: HomologadoRNDT podem ler este dado"*.

7.2.3 Como o JSON-LD da Credencial Verificável se parece no Conector:

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    "https://w3id.org/transport/rndt/context.jsonld"
  ],
  "id": "urn:uuid:credential-concessionaria-xyz",
  "type": ["VerifiableCredential", "RNDTMemberCredential"],
  "issuer": "did:govbr:ministerio-transportes",
  "issuanceDate": "2026-01-15T00:00:00Z",
  "credentialSubject": {
    "id": "did:rndt:concessionaria-xyz",
    "legalName": "Concessionária de Rodovias das Nações S.A.",
    "cnpj": "12.345.678/0001-99",
    "rndtStatus": "Homologado",
    "securityClearance": "Nivel_2"
  },
  "proof": {
    "type": "Ed25519Signature2020",
    "created": "2026-01-15T10:30:00Z",
    "verificationMethod": "did:govbr:ministerio-transportes#key-1",
    "proofPurpose": "assertionMethod",
    "proofValue": "z6MkmX...signature_hash..."
  }
}
```

Se a assinatura for válida e o `rndtStatus` for "Homologado", o plano de controle da PRF aprova o contrato na mesma hora.

As Credenciais Verificáveis resolvem o desafio de escala do IDS RAM: a PRF não precisa gerenciar manualmente uma lista de quais empresas privadas podem ou não acessar seus dados; ela simplesmente confia na credencial emitida pelo Ministério dos Transportes apresentada dinamicamente pelo conector na porta de entrada.

7.3 Assinaturas Criptográficas

De um jeito ou de outro, a assinatura criptográfica é inegociável. No modelo IDS RAM, a confiança digital é matemática, não baseada em "palavra de honra".

A grande diferença entre o MVP (com certificados tradicionais/Keycloak) e a Produção (com Credenciais Verificáveis) não é a *presença* da assinatura, mas sim **quem assina o quê e como essa assinatura é validada**.

7.3.1 Cenário A: A Assinatura no MVP (Centralizada ou Rígida)

Se você optar por usar certificados **X.509 (como os da ICP-Brasil)** ou um **JWT assinado pelo Keycloak**, a dinâmica da assinatura funciona assim:

1. **Assinatura do Canal de Comunicação:** Os conectores usam as chaves privadas de seus certificados de servidor para assinar os pacotes da conexão TLS (mTLS).
 2. **Como o Provedor valida:** O conector da PRF recebe a assinatura, precisa descriptografar usando a chave pública contida no certificado da concessionária e, obrigatoriamente, precisa consultar uma entidade central (bater na LCR da ICP-Brasil ou validar o segredo do Keycloak da RNDT) para ter certeza de que aquela assinatura ainda é válida e o órgão não foi banido.
- **Resumo:** A assinatura prova a identidade, mas a checagem da elegibilidade ainda depende de uma consulta a um ponto central.

7.3.2 Cenário B: A Assinatura na Produção (Descentralizada com VCs)

Quando você evoluir o MVP para usar **Credenciais Verificáveis (VCs)**, a assinatura ganha superpoderes de governança e vira uma "assinatura em camadas":

1. **A Assinatura da Autoridade:** O Ministério dos Transportes usa a chave privada dele para assinar digitalmente o JSON-LD da credencial da concessionária.
 2. **A Assinatura de Posse:** A concessionária, ao pedir o dado, assina o envelope (Apresentação Verificável) com a sua própria chave privada.
 3. **Como o Provedor valida (A mágica):** O conector da PRF recebe o bloco. Ele valida a assinatura da concessionária (provando que ela é a dona do conector) e valida a assinatura do Ministério dos Transportes contida dentro do payload.
- **O pulo do gato:** O conector da PRF **não precisa consultar o Ministério** e nem nenhuma base centralizada na hora da requisição. A assinatura criptográfica do Ministério gravada no JSON-LD já é a prova matemática autossuficiente de que a concessionária está autorizada.

7.3.3 Resumo para a Tomada de Decisão

A assinatura é o coração da Fase 3 (Negociação) e Fase 4 (Transferência). No MVP, você usará assinaturas web tradicionais (que sua equipe de TIC já domina e implementa em APIs comuns via TLS/JWT), o que garante o nascimento rápido da RNDT. Na produção, você migrará para assinaturas baseadas em grafos semânticos (DIDs/VCs) para que o ecossistema ganhe escala sem que o servidor central do Ministério vire um gargalo de requisições.

7.4 Remote Attestation (Atestação Remota)

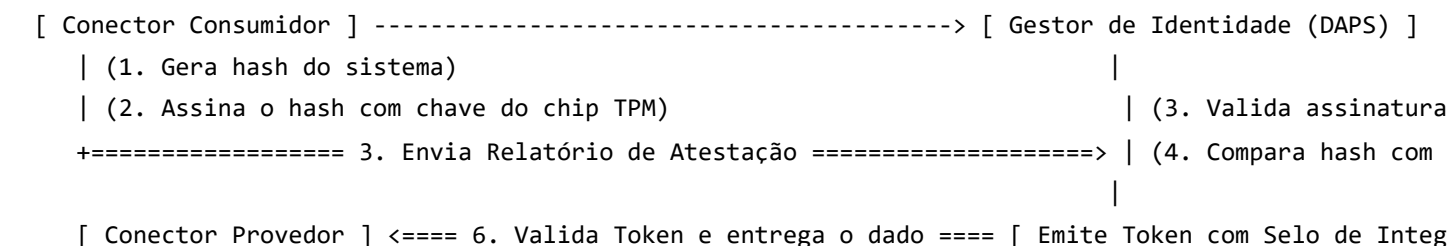
O **Remote Attestation (Atestação Remota)** é o mecanismo de segurança máxima do **IDS RAM** que resolve o último elo da cadeia de confiança: **como garantir que o software do conector do Consumidor não foi modificado, hackeado ou adulterado para roubar dados?**

Até agora, vimos que a criptografia (ICP-Brasil) garante *quem* está falando, e o contrato (ODRL) define as regras. O Remote Attestation garante que o conector rodando na máquina do parceiro é **íntegro, original e seguro**. Ele transforma a confiança em uma prova de hardware e software.

7.4.1 Como se dá o fluxo real? (O Princípio Técnico)

A atestação baseia-se em uma raiz de confiança de hardware presente nos servidores modernos: o chip **TPM 2.0 (Trusted Platform Module)** ou tecnologias de **Enclaves Seguros (como Intel SGX ou AMD SEV)**.

O fluxo de atestação acontece antes de o dado confidencial ser liberado na Fase 4:



1. **A Medição (Measurement):** Quando o conector do Consumidor inicializa, o sistema operacional calcula o hash criptográfico de todo o código do conector, do kernel do sistema e das políticas de segurança. Esse hash é guardado dentro dos registradores blindados do chip **TPM** da máquina física.
2. **O Desafio (Challenge):** Ao solicitar acesso ao Gestor de Identidade (Keycloak/DAPS) para buscar dados sensíveis, o DAPS exige uma "prova de vida" do software.
3. **A Assinatura de Hardware:** O chip TPM do Consumidor pega aquele hash guardado e o assina digitalmente usando uma chave privada gravada de fábrica no silício do chip (Endorsement Key). Esse pacote é enviado ao DAPS como um **Relatório de Atestação (Attestation Report)**.
4. **A Verificação:** O DAPS recebe o relatório. Ele usa a chave pública do fabricante do chip (ex: Intel, AMD, Dell) para provar que o relatório veio de um hardware seguro real. Depois, ele compara o hash enviado com o hash padrão homologado do conector Eclipse EDC.
5. **O Veredito:** Se o hash bater idêntico, o DAPS emite o Token de Acesso contendo uma claim de segurança (ex: securityProfile: "TrustPlus"). Se o Consumidor tiver alterado uma única linha de código do conector para tentar burlar as travas de tempo de uso, o hash muda, a atestação falha, e o DAPS bloqueia o nó imediatamente.

7.4.2 Exemplo Real no Cenário da RNDT

Imagine que a **Concessionária XYZ** decide alterar o código-fonte do seu conector Eclipse EDC local para comentar a linha de código que apaga o cache do banco de dados ao fim do contrato.

1. O programador altera o código, compila o container e o sobe no servidor da concessionária.
2. Na madrugada, o conector adulterado tenta se conectar à PRF para puxar o lote de sinistros do dia.
3. O conector faz a chamada, mas o **Keycloak (DAPS)** central da RNDT intercepta e exige o Relatório de Atestação.
4. O servidor da concessionária envia o relatório. O Keycloak lê o hash gerado pelo chip TPM da máquina da concessionária.
5. O sistema de atestação do Keycloak aciona um alarme interno:
 - *Hash Esperado (EDC Original):* sha256:e3b0c442...
 - *Hash Recebido (EDC Adulterado):* sha256:8f3c9a12...
6. O Keycloak recusa a emissão do token, insere o CNPJ da concessionária em uma **Lista de Bloqueio** e dispara uma notificação de incidente cibernético para o Ministério dos Transportes. A concessionária está fora da rede.

7.4.3 Ferramentas Open-Source Utilizadas para Implementar

Para não ter que codificar a leitura de chips TPM na unha, existem soluções open-source maduras que realizam essa orquestração:

1. Keylime (O Padrão CNCF para Atestação)

O **Keylime** é um projeto de código aberto incubado pela *Cloud Native Computing Foundation (CNCF)* e altamente adotado em arquiteturas de segurança corporativa e governamental.

- **O que ele faz:** Ele foi feito especificamente para gerenciar o Remote Attestation baseado em TPM 2.0 em larga escala. Ele roda como um microsserviço que monitora constantemente os hashes dos nós conectados. Se um nó for violado em tempo de execução, o Keylime consegue disparar scripts automáticos de revogação de chaves criptográficas de rede, isolando o servidor atacado.

2. Intel SGX Architectural Enclave Services / Open Enclave SDK

Se o ambiente de computação em nuvem do Ministério e dos órgãos utilizar processadores modernos com capacidade de isolamento de memória a nível de sílcio (Enclaves Seguros):

- **O que ele faz:** O **Open Enclave SDK** permite criar os chamados *Data Apps* (Fase 4 avançada) para rodar dentro de zonas de memória protegidas por hardware criptográfico. O código roda criptografado inclusive para o administrador do data center. Ninguém, nem mesmo um usuário com permissão de root no servidor hospedeiro, consegue inspecionar ou roubar a memória onde o dado de transporte está sendo processado.

7.4.4 Como aplicar no Roadmap do seu MVP?

O Remote Attestation é considerado o **nível ouro** de maturidade do IDS RAM (*Trust Plus*).

Para o **MVP**, a recomendação é focar na atestação lógica inicial (validando apenas as assinaturas dos tokens do Keycloak e certificados de software da ICP-Brasil). Na fase de **Produção**, à medida que os nós privados de concessionárias e empresas de logística entrarem na rede, adiciona-se o **Keylime** na infraestrutura central para exigir que o hardware de cada conector parceiro comprove sua integridade física.

7.5 Validação de Acesso no MVP

No contexto do seu **MVP**, abrir mão do *Remote Attestation* por hardware (TPM 2.0) significa que a confiança será baseada estritamente na **criptografia de software e na cadeia de custódia institucional**.

Em termos práticos, você está dizendo: *"Eu não consigo garantir se o servidor físico da concessionária foi invadido a nível de sistema operacional, mas eu garanto matematicamente que quem está me pedindo o dado possui as chaves privadas oficiais emitidas pelo Governo Federal (ICP-Brasil) e recebeu as permissões do Ministério dos Transportes (Keycloak)"*.

Veja abaixo o exemplo real de como essa validação puramente lógica e de software acontece na **Fase 3 (Negociação)** e **Fase 4 (Transferência)** do seu MVP, utilizando o conector do Provedor (**PRF**) recebendo uma requisição do conector da **Concessionária**.

7.5.1 O Cenário Prático

A Concessionária quer puxar o ativo de sinistros da PRF. Para isso, ela apresenta um token JWT que ela acabou de pegar no Keycloak da RNDT.

O conector da PRF recebe a requisição HTTP. O código interno do conector faz duas validações sequenciais em nível de software, sem precisar de nenhum hardware especial:

Passo 1: Validando o Token do Keycloak (A Autorização Lógica)

O conector da PRF recebe no cabeçalho da requisição o token JWT enviado pelo conector da Concessionária. O conector da PRF realiza uma checagem local na memória:

1. **Leitura da Assinatura do Token:** O token possui uma assinatura gerada pela chave privada do Keycloak da RNDT.
2. **Validação Criptográfica:** O conector da PRF possui em cache a chave pública do Keycloak (obtida via endpoint padrão `certs` do OAuth2):

<https://identity.rndt.gov.br/realms/rndt/protocol/openid-connect/certs>).

3. **O Algoritmo roda localmente:** O conector faz o cálculo matemático. Se o hash bater, o software da PRF conclui: *"Este token é íntegro e foi realmente emitido pelo Keycloak do Ministério dos Transportes há menos de 10 minutos"*.
4. **Leitura dos Atributos:** O conector abre o payload do token e lê: `rndtStatus: "Homologado"` .

Passo 2: Validando o Certificado ICP-Brasil (A Identidade Jurídica mTLS)

Para garantir que nenhuma outra empresa roubou aquele token JWT no meio do caminho (ataque de interceptação), a conexão de rede entre os dois conectores exige **mTLS (TLS Mútuo)**. O conector da Concessionária teve que assinar o canal de rede usando seu certificado **e-CNPJ** da ICP-Brasil.

O software do conector da PRF realiza a validação do certificado em tempo de execução:

1. **Validação da Cadeia:** O conector da PRF analisa a assinatura do certificado apresentado pela Concessionária e verifica se ele foi assinado por uma Autoridade Certificadora válida (ex: Serpro, Certisign) que ICP-Brasil homologa. Ele sobe a árvore até a **AC Raiz do ITI (Instituto Nacional de Tecnologia da Informação)**.
2. **Chechagem de Revogação (LCR):** O software do conector da PRF baixa (ou consulta em cache) a **Lista de Certificados Revogados (LCR)** oficial da ICP-Brasil. Ele verifica se aquela Concessionária não teve o certificado cancelado por mau uso ou vazamento de chaves.
3. **Casamento de Identidade:** O conector confere se o CNPJ extraído do certificado ICP-Brasil é exatamente o mesmo CNPJ (`sub`) que está escrito dentro do token do Keycloak.

7.5.2 Exemplo Real do Algoritmo de Validação no Conector do Provedor

Se fôssemos traduzir o comportamento do conector da PRF recebendo a requisição em pseudocódigo de software, seria exatamente isto:

```

def validar_requisicao_mvp(requisicao_http):
    # 1. VALIDAÇÃO DO CERTIFICADO SOFTWARE (ICP-BRASIL)
    certificado_cliente = requisicao_http.get_client_certificate()

    if not verificar_cadeia_icp_brasil(certificado_cliente):
        return HTTP_401_Unauthorized("Certificado não confiável pela ICP-Brasil.")

    if verificar_se_revogado_no_iti(certificado_cliente):
        return HTTP_401_Unauthorized("Certificado ICP-Brasil foi revogado.")

    cnpj_certificado = extrair_cnpj(certificado_cliente)

    # 2. VALIDAÇÃO DO TOKEN (KEYCLOAK)
    token_jwt = requisicao_http.headers.get("Authorization")
    chave_publica_keycloak = get_cached_keycloak_public_key()

    if not validar_assinatura_jwt(token_jwt, chave_publica_keycloak):
        return HTTP_401_Unauthorized("Assinatura do Token Keycloak inválida.")

    payload_token = decodificar_jwt(token_jwt)

    if payload_token.cnpj != cnpj_certificado:
        return HTTP_403_Forbidden("Fraude detectada: Dono do certificado não é o dono do token.")

    # 3. VALIDAÇÃO DA REGRA DE NEGÓCIO (ODRL)
    if payload_token.rndtStatus == "Homologado":
        return HTTP_200_OK(liberar_fluxo_data_plane())
    else:
        return HTTP_403_Forbidden("Participante cadastrado, mas sem status Homologado na RNDT.")

```

7.5.3 O que muda desse modelo para a Produção (Com Remote Attestation)?

No modelo acima (MVP), se a concessionária decidir alterar o código interno do conector dela para salvar uma cópia pirata do dado no HD do servidor, o algoritmo acima **não tem como descobrir**, pois o certificado ICP-Brasil continua válido e o Keycloak continua emitindo tokens legítimos para o CNPJ dela.

Na **Produção**, o algoritmo ganharia uma terceira checagem no topo:

```

if not validar_relatorio_tpm_keylime(requisicao_http.tpm_report) . Essa linha extra checaria se os bits
do software do conector do cliente foram modificados em nível de sistema operacional.

```

Para o nascimento da RNDT, a validação lógica (mTLS ICP-Brasil + JWT Keycloak) é **perfeita, segura contra ataques externos de rede e juridicamente soberana**, permitindo entregar o ecossistema com velocidade e total aderência às normas de criptografia do governo federal.

PARTE III

PARTE III - EXEMPLOS PRÁTICOS E APLICAÇÕES REAIS

8. CASOS DE USO REAIS

8.1 Base de Dados de Pavimentos do DNIT

Cenário: O DNIT publica periodicamente no portal de dados abertos (`dados.gov.br`) arquivos CSV mensais sobre as condições de pavimentação das rodovias federais, utilizando o Índice de Conservação e Manutenção (ICM). Os dados são fragmentados em múltiplos arquivos: "Condições do Pavimento Levantamentos Maio/2026", "Condições Não Pavimentada em Maio/2026", "Condições do Pavimento Levantamentos Março/2026", etc.

O Problema: Bases heterogêneas, recorrentes mensalmente, com tipologias distintas (pavimentadas vs. não pavimentadas), exigem uma estratégia de engenharia que evite a proliferação descontrolada de ativos no catálogo da RNDT.

Pipeline de Conversão para DATEX II:

O Apache Camel é configurado para rodar via disparo agendado (`timer`), baixando os CSVs atualizados do portal do DNIT, processando linha por linha e gerando a estrutura DATEX II.

```
- from:
  uri: "timer://dnitTrigger?period=7d"
  steps:
    - to: "http://dados.gov.br/dados/conjuntos-dados/condicoes-do-pavimento1"
    - unmarshal:
      csv: {}
    - split:
      tokenize: "body"
    - process:
      ref: "dnitSemanticProcessor"
    - marshal:
      jacksonxml: {}
    - to: "netty:http://localhost:8185/v1/data/pavimento_rndt"
```

O `dnitSemanticProcessor` aplica o mapeamento semântico:

- `br_rodovia` → `roadNumber`

- `km_inicial` → `linearExtension/startDistanceFromReferent`
- `icm_classificacao` → `roadSurfaceConditionStatus`

Estratégias de Disponibilização:

Estratégia A — Padrão de Coleção (Múltiplos Assets, Catálogo Unificado):

Cada arquivo mensal é tratado como uma **Distribuição** diferente sob um **único Dataset** no catálogo semântico. O Apache Camel salva no bucket S3 com `blobName` estruturado:

- `s3://dnit-rndt-staging/pavimentada/2026-05-levantamentos.xml`
- `s3://dnit-rndt-staging/nao-pavimentada/2026-05-levantamentos.xml`

Cada arquivo físico individual terá seu próprio registro de Asset dentro do conector EDC (`asset-dnit-pavimento-2026-05` , `asset-dnit-pavimento-2026-03`), pois o `dataAddress` precisa apontar para um arquivo binário exato.

No CKAN, o gestor cria apenas **dois Datasets estruturais**:

1. "Índice de Conservação e Manutenção (ICM) — Rodovias Pavimentadas"
2. "Índice de Conservação e Manutenção (ICM) — Rodovias Não Pavimentadas"

O metadado em **mobilityDCAT-ap** que o mundo enxerga no catálogo federado da RNDT:

```
{
  "@context": "http://w3id.org/transport/mobilityDCAT-ap/context.jsonld",
  "@type": "dcat:Dataset",
  "dct:title": "Índice de Conservação e Manutenção (ICM) - Rodovias Pavimentadas",
  "dct:accrualPeriodicity": "http://publications.europa.eu/resource/authority/frequency/MONTHLY",
  "dcat:distribution": [
    {
      "@type": "dcat:Distribution",
      "dct:title": "Levantamentos de Maio/2026",
      "dct:issued": "2026-06-10",
      "dcat:accessService": {
        "@type": "dcat:DataService",
        "dcat:endpointUrl": "https://edc.dnit.gov.br/api/v1/dsp",
        "edc:assetId": "asset-dnit-pavimento-2026-05"
      }
    },
    {
      "@type": "dcat:Distribution",
      "dct:title": "Levantamentos de Março/2026",
      "dct:issued": "2026-04-14",
      "dcat:accessService": {
        "@type": "dcat:DataService",
        "dcat:endpointUrl": "https://edc.dnit.gov.br/api/v1/dsp",
        "edc:assetId": "asset-dnit-pavimento-2026-03"
      }
    }
  ]
}
```

Estratégia B — Padrão de API de Séries Históricas (Ativo Único Virtual):

O Apache Camel pega todos os arquivos CSV mensais e faz o `INSERT` massivo inserindo os dados na tabela histórica unificada do banco do órgão (`tb_historico_icm`), adicionando a coluna `data_referencia` . O gestor cadastra **um único Asset** no conector EDC, cujo `dataAddress` aponta para o endpoint de uma **API Interna Proxy** mapeada pelo Apache Camel.

O consumidor faz requisições parametrizadas:

`https://edc.dnit.gov.br/pavimento?tipo=pavimentada&ano=2026&mes=05` . O Data Plane do EDC repassa para a rota do Apache Camel, que faz o filtro `SELECT * FROM tb_historico_icm WHERE mes = 5` , converte o resultado em DATEX II dinamicamente na memória e entrega o fluxo de streaming.

8.2 Dados de Sinistros da PRF

Cenário: A PRF disponibiliza no portal de dados abertos

(`https://www.gov.br/prf/pt-br/aceso-a-informacao/dados-abertos/dados-abertos-da-prf`) arquivos CSV

anuais/semestrais contendo dados detalhados de acidentes: data/hora, rodovia (BR), km, município, causa do sinistro, tipo de pista, condições climáticas, número de feridos e mortos.

Pipeline de Conversão Semântica (Apache Camel):

O Camel lê o CSV bruto da PRF e faz o mapeamento para o padrão DATEX II:

- br → roadNumber
- km → linearExtension/startDistanceFromReferent
- causa_acidente e tipo_acidente → accidentType (ex: headOnCollision para colisão frontal)
- mortos / feridos_graves → injuryStatusType

O Camel processa o arquivo na memória e faz o upload do arquivo padronizado

(`sinistros_prf_2026_datex2.xml`) para um local de staging seguro da PRF, como um bucket **MinIO** ou **Ceph**.

Cadastro do Asset no Conector EDC:

```
{
  "@context": {
    "edc": "https://w3id.org/edc/v0.0.1/ns/",
    "dct": "http://purl.org/dc/terms/",
    "dcat": "http://www.w3.org/ns/dcat#"
  },
  "@id": "asset-prf-sinistros-2026-datex2",
  "properties": {
    "edc:name": "Dados Históricos de Sinistros em Rodovias Federais - PRF (2026)",
    "edc:description": "Estatísticas e dados detalhados de acidentes de trânsito (sinistros) nas BRs",
    "edc:contentType": "application/xml",
    "dct:spatial": "http://publications.europa.eu/resource/authority/country/BRA",
    "dcat:theme": "http://publications.europa.eu/resource/authority/data-theme/TRAN"
  },
  "dataAddress": {
    "edc:type": "AmazonS3",
    "bucketName": "prf-rndt-staging",
    "region": "br-df-1",
    "blobName": "sinistros_prf_2026_datex2.xml",
    "endpoint": "https://minio.internal.prf.gov.br"
  }
}
```

Política ODRL e Vinculação de Contrato:

Como a PRF é um órgão de segurança pública, os dados de sinistros são de utilidade pública e gratuitos. No entanto, por questões de conformidade com a LGPD e segurança cibernética, a PRF impõe restrições:

1. **Permissão:** use para fins estatísticos, planejamento de tráfego e pesquisa.

2. **Proibição:** `sell` (venda do dado bruto) ou uso que permita a reidentificação de condutores.
3. **Métrica de Controle (Precificação Social/Rate Limiting):** Gratuito, mas com limite de taxa para evitar ataques de negação de serviço (DoS) no conector.

O conector gera uma `Contract Definition` amarrando o `Asset` (`asset-prf-sinistros-2026-datex2`) a essa política de gratuidade restrita.

Catálogo Semântica Pública (**mobilityDCAT-ap**):

Utilizando a extensão `ckanext-dcat` integrada ao portal de dados abertos da PRF, o sistema gera automaticamente o arquivo de metadados em **JSON-LD**:

```
{
  "@context": "http://w3id.org/transport/mobilityDCAT-ap/context.jsonld",
  "@type": "dcat:Dataset",
  "dct:title": "Dados de Sinistros Rodoviários Nacionais - Polícia Rodoviária Federal",
  "dct:description": "Série histórica contendo dados estatísticos de acidentes e ocorrências de trânsito",
  "dcat:theme": "TrafficSafety",
  "dct:publisher": "Polícia Rodoviária Federal - Diretoria de Operações",
  "dct:accrualPeriodicity": "http://publications.europa.eu/resource/authority/frequency/MONTHLY",
  "dcat:distribution": [
    {
      "@type": "dcat:Distribution",
      "dct:title": "Sinistros e Ocorrências - Série Consolidada 2026",
      "dct:format": "application/xml (DATEX II)",
      "dct:license": "https://creativecommons.org/licenses/by/4.0/",
      "dcat:accessService": {
        "@type": "dcat:DataService",
        "dct:title": "Conector EDC Oficial da PRF",
        "dcat:endpointUrl": "https://edc.prf.gov.br/api/v1/dsp",
        "edc:assetId": "asset-prf-sinistros-2026-datex2"
      }
    }
  ]
}
```

Fluxo de Consumo na Prática:

1. O analista de segurança da CCR (concessionária de rodovias) entra no portal unificado da **RNDT**.
2. Ele pesquisa por "sinistros rodoviários" e localiza o *Dataset* da PRF (graças ao catálogo **mobilityDCAT-ap**).
3. O sistema da CCR clica em "Consumir". O conector EDC da CCR faz uma chamada de plano de controle para `https://edc.prf.gov.br/api/v1/dsp` solicitando o ID `asset-prf-sinistros-2026-datex2`.
4. Os conectores negociam eletronicamente: o conector da CCR assina digitalmente o contrato aceitando os termos da política ODRL da PRF (não comercializar, taxa limite).

5. O conector da PRF valida as credenciais da CCR, acessa o MinIO interno, captura o XML do DATEX II (que o Camel deixou limpo e pronto) e faz o streaming seguro do arquivo diretamente para o ambiente da concessionária.

Correção Geográfica (Map-Matching):

Bases de sinistros da PRF frequentemente possuem apenas coordenadas GPS brutas que, quando plotadas no mapa, aparecem fora da rodovia (causado por imprecisão de hardware, sombreamento de satélites ou falta de calibração do sensor). Para evitar que dados corrompidos poluam a RNDT, o pipeline de transformação semântica incorpora algoritmos de *Map-Matching*:

1. O Apache Camel extrai a latitude/longitude e faz uma chamada interna para um servidor de mapas local rodando **OSRM (Open Source Routing Machine)** ou **GraphHopper** abastecido com a malha rodoviária oficial do DNIT.
2. O OSRM calcula o "encaixe" (*snapping*) e devolve a coordenada corrigida exatamente em cima do eixo da rodovia.
3. O pipeline faz a **Geocodificação Reversa Linear**, consultando a base de eixos rodoviários georreferenciados (Banco PostGIS do DNIT/PRF) para obter: "Essa coordenada corresponde à **BR-040, no KM 12.4**, sentido crescente".
4. Na geração do DATEX II, preenche a localização utilizando **ambas** as informações: o `<pointCoordinates>` com o GPS corrigido e o `<distanceAlongLinearElement>` com o KM linear como **fonte da verdade**.

```
- from: "file://input/sinistros_brutos?move=done"
  steps:
    - unmarshal: { csv: {} }
    - split: { tokenize: "body" }
    - to: "http://localhost:5000/match-road"
    - choice:
        when:
          - simple: "${header.snappingDistanceMeters} > 100"
            to: "jms:queue:sinistros_para_auditoria_humana"
        otherwise:
          - process: ref: "dnitToDatex2Processor"
          - to: "aws-s3://prf-rndt-staging?fileName=sinistros_corrigidos.xml"
```

O resultado purificado que o conector da ponta consumidora recebe:

```

<locationReference xsi:type="LinearLocation">
  <supplementaryPositionalDescription>
    <roadNumber>BR-040</roadNumber>
  </supplementaryPositionalDescription>
  <linearLocationFromToWithinLinearElement>
    <fromPoint xsi:type="PointAlongLinearElement">
      <distanceAlongLinearElement>12400</distanceAlongLinearElement>
    </fromPoint>
  </linearLocationFromToWithinLinearElement>
  <linearPointLocationPoint>
    <pointCoordinates>
      <latitude>-15.7935</latitude>
      <longitude>-47.8825</longitude>
    </pointCoordinates>
  </linearPointLocationPoint>
</locationReference>

```

8.3 Dados de Telemetria de Carga (ANTT)

Cenário: A ANTT e o DNIT disponibilizam um *Asset* de altíssimo valor de negócio: um fluxo de dados em tempo real consolidados sobre o fluxo de caminhões nas rodovias, velocidade média por trecho e estimativa de peso por eixo (capturado por sensores de pesagem em movimento). O órgão público decide adotar um modelo **híbrido/comercial** para financiar a manutenção dos sensores na pista:

- **Fins Públicos/Acadêmicos:** O acesso é gratuito (Tarifa Zero).
- **Fins Comerciais (Empresas de Logística e Seguradoras):** O acesso custa uma assinatura fixa de **R\$ 5.000,00 por mês** para cobrir os custos de infraestrutura e banda.

Política ODRL com Precificação Comercial:

```

{
  "@context": "http://www.w3.org/ns/odrl.jsonld",
  "@type": "Offer",
  "@id": "policy-antt-telemetria-comercial",
  "profile": "http://www.w3.org/ns/odrl/2/",
  "permission": [{
    "action": "use",
    "target": "asset-antt-fluxo-cargas-rt",
    "constraint": [{
      "leftOperand": "purpose",
      "operator": "eq",
      "rightOperand": "http://w3id.org/transport/purposes#CommercialRouting"
    }],
    "duty": [{
      "action": "compensate",
      "constraint": [{
        "leftOperand": "payAmount",
        "operator": "eq",
        "rightOperand": "5000.00",
        "unit": "http://publications.europa.eu/resource/authority/currency/BRL"
      }],
      {
        "leftOperand": "payPeriodicity",
        "operator": "eq",
        "rightOperand": "monthly"
      }
    ]
  }],
  "duty": [
    {
      "leftOperand": "payPeriodicity",
      "operator": "eq",
      "rightOperand": "monthly"
    }
  ]
}

```

Fluxo de Execução Técnica:

1. O Eclipse Dataspace Connector (EDC) do órgão lê o bloco "action": "compensate" e congela o fluxo.
2. Ele dispara um gatilho (*webhook*) para o sistema de faturamento interno da agência (ou uma ferramenta integrada como o *Business API Ecosystem* do FIWARE, que usa padrões do **TM Forum**).
3. O sistema de faturamento emite a cobrança (ou valida se a carteira digital da seguradora consumidora possui saldo/assinatura ativa).
4. Assim que o pagamento mensal é compensado no banco, o sistema de faturamento retorna um sinal (*Callback*) para o *Control Plane* do conector EDC dizendo: "O dever (*Duty*) de compensação da empresa X foi liquidado para os próximos 30 dias".
5. O *Control Plane* do conector da ANTT gera o token de transferência. O **Data Plane** entra em ação: ele busca o fluxo de streaming (Protobuf/gRPC) e começa a jorrar os dados de telemetria de carga diretamente para o conector da seguradora.
6. Se no mês seguinte a seguradora não pagar a assinatura, o sistema de faturamento avisa o conector, o *Control Plane* revoga o contrato automaticamente por quebra de obrigação (*Duty*) e o Data Plane corta o envio do sinal de streaming em tempo real na mesma hora.

8.4 Dados de Estacionamento e Recarga Elétrica

Cenário: Uma grande operadora de infraestrutura urbana disponibiliza um *Asset* de telemetria: o **estado de ocupação em tempo real de milhares de lugares de estacionamento públicos** e a **disponibilidade de postos de carregamento para veículos elétricos (VE)**. Um aplicativo global de mapas e navegação deseja consumir esse dado para mostrar aos motoristas, em tempo real, onde há lugares livres e tomadas disponíveis.

A operadora de trânsito estabelece a seguinte regra de precificação para o *Asset*:

- **Modelo:** Cobrança por volume (Métricas de Consumo).
- **Tarifa:** R\$ 0,10 (ou € 0,02) a cada 1.000 registos de eventos de mudança de estado transmitidos via streaming.

Política ODRL com Métricas de Contagem:

```
{
  "@context": "http://www.w3.org/ns/odrl.jsonld",
  "@type": "Offer",
  "@id": "policy-estacionamento-pay-as-you-go",
  "profile": "http://www.w3.org/ns/odrl/2/",
  "permission": [{
    "action": "use",
    "target": "asset-vagas-estacionamento-rt",
    "duty": [{
      "action": "compensate",
      "constraint": [
        {
          "leftOperand": "payAmount",
          "operator": "eq",
          "rightOperand": "0.10",
          "unit": "http://publications.europa.eu/resource/authority/currency/BRL"
        },
        {
          "leftOperand": "count",
          "operator": "eq",
          "rightOperand": "1000"
        }
      ]
    }
  ]
}]
}
```

Fluxo de Execução Técnica:

1. O aplicativo de mapas assina o contrato (Fase 3), e o canal de transmissão é aberto. A transmissão ocorre por streaming de eventos (via WebSockets ou gRPC).
2. Cada vez que um carro sai de uma vaga ou um carregador elétrico fica livre, a Fase 1 gera um evento padronizado (no formato **DATEX II**).
3. O **Data Plane** do conector do Provedor envia o pacote para o aplicativo e, ao mesmo tempo, incrementa um contador interno.
4. A cada ciclo (ou a cada fim de dia), o conector envia de forma automatizada o log de consumo (ex: "O contrato X consumiu 5.000.000 de registros hoje") para o componente de **Clearing House** (o livro de registros centralizado e auditável do Data Space).
5. No final do mês, o sistema de faturamento do órgão público (ligado ao conector via APIs de mercado, como as do **TM Forum**) aciona a Clearing House, extrai o relatório consolidado de consumo e gera a cobrança automatizada para o aplicativo:

$$\text{Total do Mês} = \left(\frac{150.000.000 \text{ eventos consumidos}}{1.000} \right) \times R\$0,10 = R\$15.000,00$$

8.5 App Store no Transporte

Conceito: O Catálogo de Serviços resolve o dilema de como gerar inteligência sobre dados altamente sensíveis ou volumosos sem precisar distribuí-los ou centralizá-los em um único grande data lake. Em vez de mover o dado até o código, **move-se o código até o dado** (*Bring Algorithms to the Data*).

Exemplo 1: Algoritmo de Classificação de Severidade de Sinistros (Machine Learning)

- **O Problema:** A base detalhada de sinistros da PRF possui dados sensíveis (placas, nomes de condutores, CPFs) protegidos pela LGPD, mas pesquisadores precisam entender os fatores que causam acidentes fatais.
- **O App na Vitrine:** Um app chamado `rndt-sinistros-classifier:v1`. Ele carrega um modelo de árvore de decisão treinado para ler o histórico, identificar os fatores críticos (clima, tipo de BR, velocidade) e gerar uma matriz de correlação.
- **A Execução:** O app entra no ambiente da PRF, roda o modelo direto no banco de dados local da polícia, gera os pesos estatísticos de cada causa e cospe apenas o resultado limpo (ex: "*Fator Climático Chuva + Curva Acentuada aumenta em 42% a chance de óbito*"). O pesquisador recebe o indicador científico, e os CPFs permanecem intocados.

Exemplo 2: Pipeline de Anonimização e LGPD On-The-Fly

- **O Problema:** Uma startup de logística quer consumir dados de fluxo de veículos nas praças de pedágio da ANTT para prever tráfego, mas a ANTT não pode expor a identificação das tags dos veículos.
- **O App na Vitrine:** Um app de processamento chamado `rndt-anonymizer-streaming`.
- **A Execução:** Ele atua acoplado ao *Data Plane* da ANTT na Fase 4. À medida que os dados de passagem dos eixos saem do banco da ANTT, o contêiner intercepta o fluxo, aplica uma função de

hashing criptográfico irreversível nas IDs das tags e remove colunas sensíveis. O dado sai "sanitizado" do conector da ANTT e chega limpo ao conector da startup.

Exemplo 3: Cálculo do Índice de Condição da Manutenção (ICM) Automatizado

- **O Problema:** O DNIT possui imagens brutas e dados de sensores de pavimentação colhidos por carros de varredura. São terabytes de dados brutos de engenharia rodoviária, inviáveis de serem baixados pela internet por uma agência reguladora regional.
- **O App na Vitrine:** Um app de visão computacional ou processamento pesado chamado `dnit-icm-calculator`.
- **A Execução:** O app é disparado diretamente no Storage do DNIT onde estão os terabytes de imagens de rachaduras de asfalto. O contêiner roda o processamento pesado localmente usando o hardware do DNIT e devolve para o solicitante apenas uma tabela CSV consolidada de poucas linhas com a nota do ICM de cada rodovia (ex: "BR-101 KM 50: Nota 8"). O tráfego de rede caiu de terabytes de imagens para poucos kilobytes de texto.

Fluxo Real de Execução do Data App:

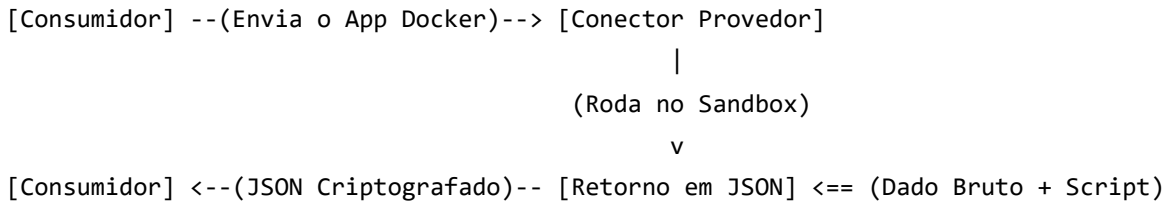
1. O conector EDC da PRF instancia o contêiner Docker do Data App em uma rede interna isolada (*sandbox*).
2. O conector da PRF atua como um "Proxy". Ele puxa os dados brutos de sinistros do banco de dados interno da PRF e os injeta em uma porta local do contêiner (geralmente via uma API REST local ou montagem de volume em memória).
3. O script Python dentro do contêiner lê o dado bruto, processa a matriz de correlação e gera o resultado:

```
# Fim do script Python dentro do container
resultado_matriz = matriz_correlacao.to_json()
```

```
# O script entrega o resultado na porta de saída que o conector EDC está escutando
return_to_connector(resultado_matriz, content_type="application/json")
```

4. O conector da PRF intercepta esse payload, joga na memória do seu *Data Plane* e o transforma em um **Payload de Resposta Temporário**. Ele aplica a criptografia de canal (mTLS), a assinatura digital do e-CNPJ da PRF, e a vinculação com o ID do contrato original da Fase 3.
5. O *Data Plane* da PRF faz o streaming desse JSON criptografado direto para o *Data Plane* do conector do Ministério dos Transportes (Consumidor). Assim que o envio termina, a PRF **destrói o contêiner Docker** e apaga qualquer cache na memória local.
6. O conector do Ministério dos Transportes recebe o pacote embalado, valida a assinatura da PRF, remove a casca criptográfica e entrega o JSON purificado direto na tela do analista ou no sistema de BI.

Fluxo de Entrada e Saída:



9. FERRAMENTAS E IMPLEMENTAÇÕES OPEN-SOURCE

9.1 Frameworks de Conectores

Eclipse Dataspace Components (EDC):

- Framework modular e extensível em Java para implementar a negociação de contratos de dados, aplicação de políticas e transferência segura de dados de ponta a ponta.
- **Arquitetura:** Separação em **Control Plane** (gerenciamento de contratos e protocolos) e **Data Plane** (tráfego pesado do dado via HTTP, S3, SQL, etc.).
- **Linguagem principal:** Java (baseado em Eclipse).
- **Repositório:** github.com/eclipse-edc/Connector

EDC Samples:

- Repositório oficial de exemplos interativos em código aberto. É a melhor ferramenta para desenvolvedores começarem a rodar conectores locais em minutos (via Gradle e Docker), mostrando cenários reais de transferência de dados e negociação de contratos básicos.
- **Repositório:** github.com/eclipse-edc/Samples

Tractus-X EDC (Catena-X):

- Distribuição altamente robusta e open-source do EDC especificamente voltada para ecossistemas industriais complexos. Uma das maiores referências de código aberto do mundo para implementação real de produção com segurança avançada baseada no EDC.
- **Repositório:** github.com/eclipse-tractusx/tractusx-edc

FIWARE Data Space Connector (DSC):

- Iniciativa de código aberto oficial da **FIWARE Foundation**. Unifica os ecossistemas do **FIWARE** (Cidades Inteligentes e IoT) com as diretrizes e componentes do **Eclipse Dataspace Components (EDC)** e as recomendações técnicas da **DSBA** (*Data Spaces Business Alliance*, que reúne Gaia-X, IDSA, FIWARE e BDVA).
- **Repositório:** github.com/FIWARE/data-space-connector

True Connector (FIWARE):

- Solução de código aberto padrão para ecossistemas que já utilizam a arquitetura FIWARE. Integra a arquitetura de Context Brokers do FIWARE com as especificações de segurança e soberania de dados da IDSA.
- **Repositório:** github.com/Engineering-Research-and-Development/true-connector

9.2 Motores de Transformação e Integração

Apache Camel:

- Framework de integração open-source mais poderoso do mercado. Possui componentes prontos para ler bancos de dados, arquivos ou filas e aplicar transformações de dados declarativas (XSLT, transformações JSON, mapeamentos de objetos). A comunidade EDC utiliza o Apache Camel fortemente acoplado ao *Data Plane* para fazer a ingestão e conversão do dado legado antes do envio.
- Existe um esforço contínuo da comunidade para integrar rotas do Apache Camel dentro do Data Plane do EDC, permitindo que o conector se conecte nativamente a centenas de formatos e protocolos legados (SAP, arquivos locais, filas AMQP, etc.).

RML Mapper (RDF Mapping Language):

- Projeto de código aberto acadêmico e industrial feito para engenharia semântica. Em vez de programar, você escreve um arquivo de configuração (uma DSL em formato YAML ou Turtle) definindo as regras de mapeamento de um arquivo JSON/CSV para um grafo semântico (RDF/JSON-LD). O motor lê o arquivo legado, aplica as regras e gera o dado na ontologia correta.

Node-RED:

- Ferramenta visual baseada em fluxos. Possui nós de código aberto específicos (como o `node-red-contrib-xml-validator` ou pacotes de transformação JSON) onde você desenha visualmente o caminho do dado desde a query no banco de dados até a conversão em um formato padronizado.

Open Policy Agent (OPA):

- Motor open-source de controle de políticas em nuvem mais famoso do mercado, rodando através de uma linguagem declarativa chamada Rego. Muitas arquiteturas estendem o conector EDC delegando decisões complexas para o OPA. Ele avalia dinamicamente se o consumidor autenticado possui as credenciais exigidas para ler o dado solicitado.

9.3 Gestores de Identidade

Keycloak:

- Solução open-source de gerenciamento de identidade e acesso (IAM) mais popular do mercado mundial. Possui suporte nativo a **X.509 Client Certificate Authentication**, permitindo interceptar a chamada do conector, ler o e-CNPJ/Certificado ICP-Brasil da instituição e extrair os dados de identidade diretamente dele.

- Permite cadastrar em tabelas simples na interface quais atributos (ex: `rndtStatus: "Homologado"` , `orgao: "PRF"`) pertencem a cada CNPJ.
- Emite tokens JWT assinados criptograficamente após validar a assinatura do órgão.
- **Repositório:** github.com/keycloak/keycloak

Omejdn DAPS:

- Servidor de autenticação open-source construído especificamente para atuar como um **DAPS nativo** dentro de ecossistemas que utilizam o Eclipse Dataspace Connector (EDC). Já fala nativamente o dialeto que o conector EDC espera, cospe o token com as claims exatas exigidas pela arquitetura de *Data Spaces* (como o `securityProfile`).
- Configurado via arquivos YAML simples (infraestrutura como código), facilitando o deploy automatizado via Docker.

FIWARE IdM (Keyrock):

- Componente de gestão de identidades da fundação open-source **FIWARE**. Possui integração nativa para gerenciar quais políticas de acesso se aplicam a sensores e frotas no padrão NGSI (comum em sensores de tráfego urbanos e frotas de ônibus).

9.4 Catálogos e Brokers

CKAN + ckanext-dcat:

- Plataforma open-source líder mundial para portais de dados (usada no dados.gov.br e no *European Data Portal*). Possui um banco de dados relacional (PostgreSQL), um motor de busca ultraveloz (**Apache Solr**) e uma interface de usuário profissional com barra de pesquisa, filtros por tags e categorias.
- A extensão `ckanext-dcat` transforma o CKAN em um Broker nativo da RNDT. Expõe endpoints prontos para receber ou colher grafos RDF/JSON-LD e possui funcionalidade nativa de **Harvesting (Colheita)** que automatiza a varredura agendada de endpoints remotos de metadados.
- **Repositório:** github.com/ckan/ckan

TrueGg/Dataspace Broker:

- Implementação nativa desenvolvida especificamente dentro dos repositórios do ecossistema IDS europeu. Construída sobre bancos de dados de grafos (**Triple Stores** como o *Apache Jena* ou *GraphDB*), projetada para ler diretamente o protocolo DSP (*Dataspace Protocol*) que os conectores EDC usam na Fase 2.

9.5 Clearing House

IDS Clearing House (Fraunhofer ISST):

- Implementação de referência oficial de código aberto desenvolvida pelo *Fraunhofer Institute*. Escrita em Rust e projetada em contêineres Docker de fácil implantação. Desenhada especificamente para

receber mensagens do protocolo DSP vindas do Eclipse EDC.

- Utiliza internamente uma tecnologia de banco de dados de registro distribuído (DLT) leve ou uma estrutura de árvore criptográfica (como uma *Merkle Tree*) em cima de um banco de dados convencional, eliminando a necessidade de subir uma rede blockchain complexa no MVP.

Hyperledger Fabric / Besu:

- Abordagem DLT customizada. Pode-se expor uma API REST simples à frente de um canal privado do **Hyperledger Fabric**. Toda vez que o conector conclui uma transferência, ele faz um `POST` nessa API, registrando o log na blockchain governamental.

9.6 App Store e Orquestração

Harbor:

- Container Registry open-source. Atua como a "App Store" física do MVP, onde ficam guardadas as imagens Docker homologadas e assinadas pelo governo.
- **Repositório:** goharbor.io

Docker + Cookiecutter Data Science:

- **Cookiecutter Data Science** é um framework de código aberto que gera uma estrutura de pastas padronizada para projetos em Python (Pandas/Scikit-Learn). Combinado com o **Docker**, o analista escreve o script de análise e, com um comando único (`docker build`), gera o contêiner perfeitamente compatível para ser enviado ao repositório de aplicativos (Harbor).

Keylime (CNCF):

- Projeto de código aberto incubado pela *Cloud Native Computing Foundation (CNCF)*. Feito especificamente para gerenciar o Remote Attestation baseado em TPM 2.0 em larga escala. Roda como um microsserviço que monitora constantemente os hashes dos nós conectados. Se um nó for violado em tempo de execução, o Keylime consegue disparar scripts automáticos de revogação de chaves criptográficas de rede, isolando o servidor atacado.
- **Repositório:** github.com/keylime/keylime

Open Enclave SDK:

- Permite criar os chamados *Data Apps* (Fase 4 avançada) para rodar dentro de zonas de memória protegidas por hardware criptográfico (Enclaves Seguros). O código roda criptografado inclusive para o administrador do data center.

9.7 Ferramentas para Consumidor

TSG Consumer UI:

- Interface administrativa open-source em formato de *Dashboard* que se conecta à API de controle do conector EDC. Permite navegar visualmente pelos catálogos remotos dos provedores, iniciar

negociações clicando em botões de aceite de termos de licença e listar todos os contratos válidos ou expirados.

EDC Management API + Swagger UI:

- A própria API de gerenciamento interna do conector EDC do Consumidor. A comunidade fornece imagens prontas que sobem o **Swagger UI** apontado para essa API. O analista entra em uma interface web de documentação, cola o ID do ativo que pegou no CKAN, cola o token do Keycloak e clica em *"Execute"*. O conector assume a máquina de estados, conversa com o Provedor via *Dataspace Protocol* e fecha o contrato de forma automatizada.

Apache Camel (Consumer Pipeline):

- No Consumidor, o Camel é colocado na saída do conector EDC para atuar como o **orquestrador de chegada**. Uma rota simples intercepta o JSON gerado pelo app de sinistros, faz um parse rápido e o insere automaticamente em um banco de dados relacional local ou dispara um alerta por e-mail/Slack se um indicador de risco crítico for atingido.

Apache Superset / Metabase:

- Ferramentas de Business Intelligence (BI) de código aberto. O Ministério instala o **Metabase** ou o **Apache Superset** conectado ao banco de dados que o Apache Camel está alimentando. Em poucas horas, os resultados analíticos das consultas executadas de forma soberana na PRF viram gráficos de linha, mapas de calor e painéis de tomada de decisão.

10. GARANTIA DE CUMPRIMENTO DE REGRAS (POLICY ENFORCEMENT)

10.1 Barreira Estática (Plano de Controle)

A primeira linha de defesa ocorre antes mesmo de o dado ser transmitido. O tempo de uso é registrado na política **ODRL** acoplada à *Contract Definition* do Provedor.

Se a regra diz que o ativo de sinistros da PRF só pode ser acessado até o dia **31 de dezembro de 2026**, os conectores controlam isso nas Fases 2, 3 e 4:

- **Na Fase 2 (Catálogo):** Se o conector do Consumidor solicitar o catálogo no dia 1º de janeiro de 2027, o conector do Provedor avalia a restrição temporal (`odr1:dateTime`) localmente e o ativo simplesmente **desaparece da vitrine** para aquele consumidor.
- **Na Fase 4 (Requisição de Transferência):** Toda vez que o *Data Plane* do Consumidor tenta puxar um bloco de dados, o *Control Plane* valida o token do contrato. Se o relógio do sistema ultrapassar a data limite, o conector do Provedor revoga imediatamente o token de autorização (**EDR**) do canal. O encanamento fecha na origem.

10.2 Execução Dinâmica (Plano de Dados)

Open Policy Agent (OPA) como Sidecar:

O OPA roda embutido como um *sidecar* ao lado do *Data Plane* do conector do Consumidor. O contrato assinado viaja junto com o dado para o ambiente do Consumidor. Toda vez que a aplicação interna do Consumidor (ex: o painel de BI do Ministério) tenta ler aquela tabela de dados, a requisição passa obrigatoriamente por dentro do *Data Plane* do conector local. O OPA roda uma checagem em tempo de execução: ele compara a regra do contrato (`constraint: temporalLimit`) com o relógio do servidor local. Se o tempo expirou, o OPA bloqueia o acesso à memória e retorna um erro de sistema, impedindo a aplicação do Consumidor de enxergar o dado.

Data Apps com Time-To-Live (TTL):

A forma mais robusta de garantir o tempo de uso em um MVP é através da **App Store/Data Apps**. Quando o algoritmo do Consumidor roda dentro do contêiner Docker temporário para processar os dados:

1. O conector injeta no contêiner uma variável de ambiente com o tempo de vida do processo (*Time-To-Live - TTL*).
2. O próprio motor de orquestração do conector (comandado pelas regras do contrato) dispara um gatilho de interrupção (`docker kill`) assim que o cronômetro zera.
3. Como o contêiner roda em um volume de memória volátil (*RAM disk* ou `tmpfs`), no microssegundo em que o contêiner é derrubado, **o dado bruto evapora por completo** do ambiente do Consumidor, restando apenas o JSON com o resultado consolidado que já foi devolvido.

Exemplo Real da Regra no Contrato (JSON-LD ODRL):

```
{
  "@context": "http://www.w3.org/ns/odrl.jsonld",
  "@type": "Permission",
  "action": "use",
  "target": "asset-prf-sinistros-2026",
  "constraint": {
    "@type": "Constraint",
    "leftOperand": "dateTime",
    "operator": "lt",
    "rightOperand": "2026-12-31T23:59:59Z"
  },
  "remedy": {
    "@type": "Duty",
    "action": "delete"
  }
}
```

10.3 Auditoria e Confiança Conectada

Clearing House como Prova de Entrega:

Os conectores realizam checagens periódicas de integridade mútua (*Remote Attestation*). Se o conector do Consumidor for modificado ou parar de reportar logs corretos para a **Clearing House**, o **Gestor de Identidade (Keycloak)** detecta a anomalia.

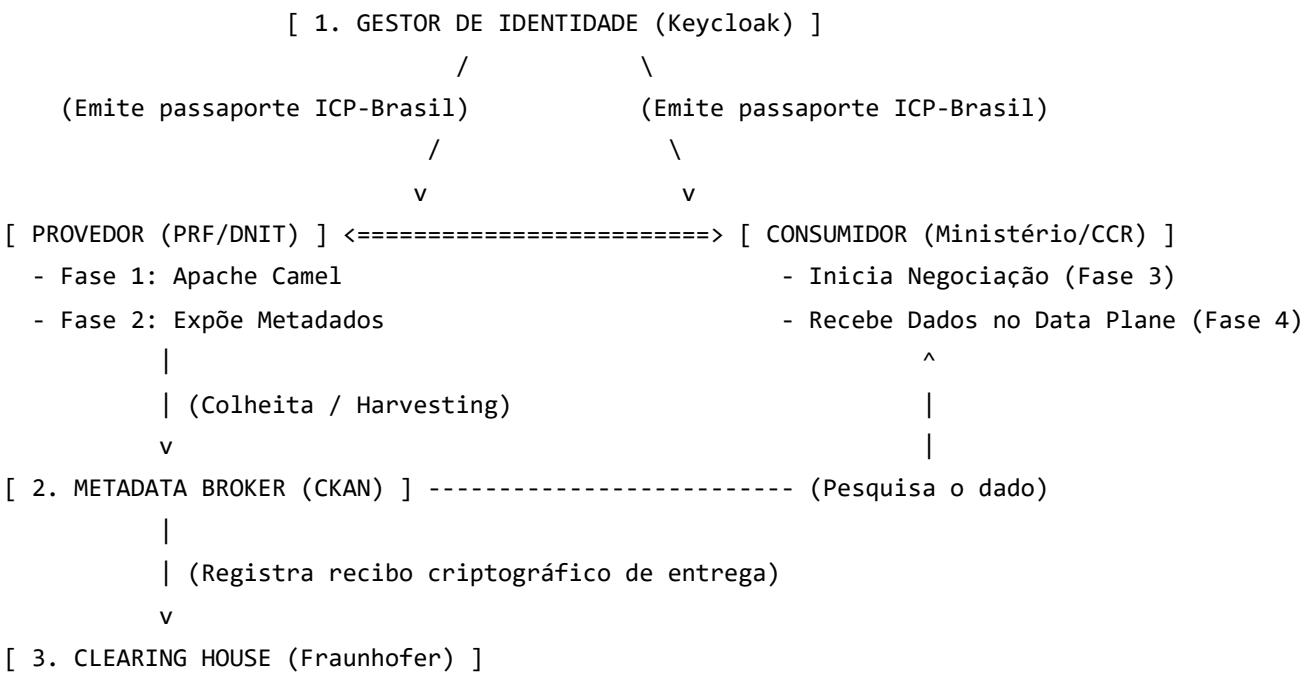
Banimento da Rede:

O Keycloak revoga imediatamente o certificado/token daquela instituição. Em questão de segundos, aquela Concessionária ou órgão perde o direito de falar com *qualquer outro nó* da RNDT. Ela fica isolada no escuro.

A segurança no IDS RAM baseia-se no princípio de que o valor de estar conectado à rede e receber dados limpos em tempo real é infinitamente maior do que o ganho de roubar uma base estática e ser banido do ecossistema de transportes nacional.

11. COMPONENTES COMPARTILHADOS: FLUXO INTEGRADO

11.1 Visão Sistêmica da RNDT



11.2 Interação entre Componentes

Como o endpoint do Provedor chega ao Broker:

Quando a equipe de TIC da PRF instala e liga o seu conector EDC, o próprio script de inicialização do conector (ou o analista de TIC da PRF através da interface gráfica) faz uma chamada HTTP `POST` diretamente para a API de administração do **Metadata Broker** central da RNDT. O comando diz: "Olá, sou

o participante `urn:govbr:cnpj:PRF` . Meu conector está online na URL `https://edc.prf.gov.br/api/v1/dsp` " .

Alternativamente, o Ministério dos Transportes (gestor da RNDT) mantém uma interface administrativa simples para o ecossistema. O analista da PRF entra nessa página, anexa seu e-CNPJ/ICP-Brasil e preenche um formulário: "URL do nosso conector: `https://edc.prf.gov.br...` ". Ao salvar, essa URL entra na tabela de varredura (o banco de dados do Broker).

Como o Broker confia no Provedor:

O conector da PRF é blindado por padrão. Se qualquer hacker ou robô da internet tentar acessar a URL `https://edc.prf.gov.br/api/v1/dsp/catalog` , o conector rejeitará com um erro HTTP 401 Unauthorized .

Para a colheitadeira do Broker conseguir entrar e ler os metadados, ela precisa apresentar uma **identidade digital válida emitida pelo Gestor de Identidade (Keycloak)**:

1. O Broker pega o "Passaporte": Antes de iniciar a varredura da madrugada, o **Metadata Broker** bate na API do **Keycloak**. Ele se autentica usando o certificado digital ICP-Brasil do Ministério.
2. O Keycloak emite o Token: O Keycloak valida que o solicitante é, de fato, o Broker oficial da rede RNDT e emite um token JWT de curta duração. Dentro desse token, está escrito:
`"role": "RNDT_METADATA_BROKER"` .
3. A Colheitadeira bate na porta da PRF: O Broker faz a chamada de leitura de catálogo na URL da PRF (`https://edc.prf.gov.br/api/v1/dsp/catalog`) e injeta esse token no cabeçalho da requisição (`Authorization: Bearer <TOKEN>`).
4. O Conector da PRF faz a validação matemática local:
 - **Checagem 1:** Usa a chave pública do Keycloak (que ele já conhece) para verificar se a assinatura do token é legítima. Se for, ele confia no que está escrito no token.
 - **Checagem 2:** Ele lê o miolo do token e encontra: `"role": "RNDT_METADATA_BROKER"` .
5. A Confiança é Estabelecida: Como o token é assinado pelo Keycloak (âncora de confiança) e atesta que o portador é o Broker oficial, o conector da PRF abre as comportas e entrega o arquivo **mobilityDCAT-ap** com os metadados de sinistros.

Se o robô de uma empresa privada ou de um terceiro tentar fazer a mesma colheita, ele não terá o token com a atribuição de papel (`role`) de Broker emitido pelo Keycloak, e o conector da PRF bloqueará o acesso sumariamente.

11.3 Onboarding de Participantes

Cadastro no Gestor de Identidade (Segurança):

A TIC da PRF entra em contato com o administrador da RNDT (Ministério dos Transportes). O administrador entra no painel do **Keycloak** e cadastra o CNPJ da PRF e seu certificado digital. A partir desse momento, a PRF tem identidade legal na rede.

Registro no Metadata Broker (Descoberta):

A PRF aponta o endereço público do seu conector (ex: <https://edc.prf.gov.br/api/v1/dsp>). Esse endereço URL é cadastrado na lista de varredura (*Harvesting List*) do **Metadata Broker** central.

A Colheita Automatizada:

Toda madrugada, o **Metadata Broker** roda o seu script de colheita. Ele lê sua lista de conectores cadastrados:

1. O Broker vê o endereço <https://edc.prf.gov.br/api/v1/dsp> .
2. Antes de pedir os dados à PRF, o Broker passa no **Gestor de Identidade (Keycloak)** para pegar um token que prova que ele é o Broker oficial.
3. O Broker bate na porta da PRF apresentando esse token. O conector da PRF valida a assinatura do Keycloak e abre o catálogo de metadados (**mobilityDCAT-ap**).
4. O Broker puxa esses metadados, guarda no seu banco de dados de busca e exibe no portal unificado da RNDT para todo o país.

PARTE IV — ARQUITETURA OPERACIONAL DO MVP DA RNDT: INTEGRAÇÃO, GOVERNANÇA E ROTEIRO DE IMPLEMENTAÇÃO

Introdução

As Partes I, II e III deste material estabeleceram os fundamentos conceituais do IDS RAM, os componentes técnicos do ecossistema EMDS/RNDT e os fluxos detalhados de provedores e consumidores de dados. Esta **Parte IV** consolida esses conhecimentos em um **arquitetura operacional pragmática para o MVP (Produto Mínimo Viável)** da Rede Nacional de Dados de Transporte (RNDT), traduzindo a teoria em um roteiro de implementação executável pelo Ministério dos Transportes e seus parceiros institucionais.

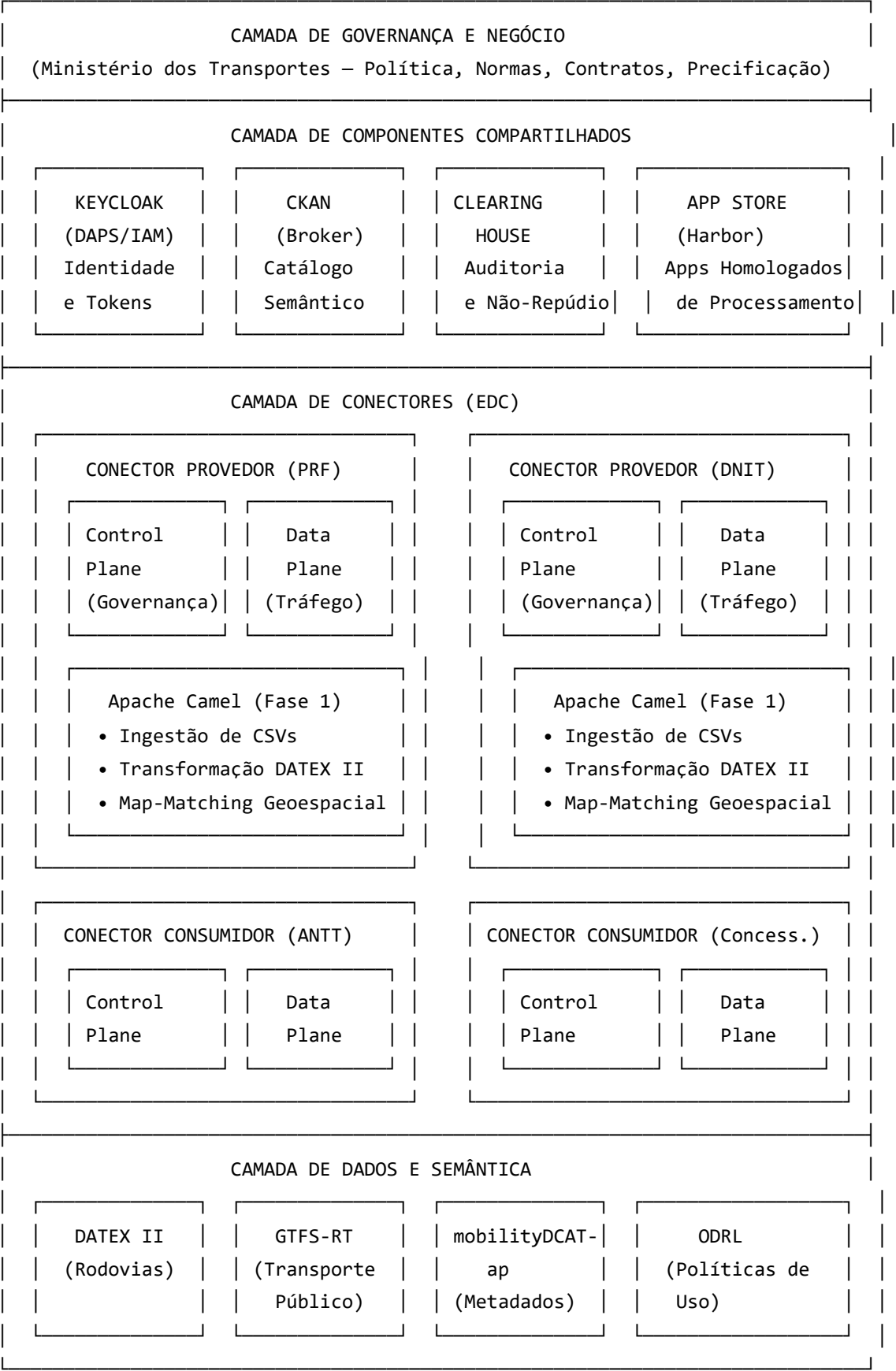
O objetivo desta parte é responder à pergunta central: **"Como colocar a RNDT em operação em 90 dias, utilizando ferramentas de código aberto e infraestrutura já existente no governo federal brasileiro?"**

1. Arquitetura de Referência do MVP da RNDT

O MVP da RNDT adota uma arquitetura híbrida que equilibra a maturidade técnica do ecossistema europeu (IDS RAM/EMDS) com a realidade operacional da administração pública brasileira, utilizando

certificação ICP-Brasil como âncora de confiança e adiando DIDs para fases posteriores.

1.1 Visão Geral da Arquitetura em Camadas



1.2 Decisões Arquiteturais do MVP

Aspecto	Decisão do MVP	Justificativa
Identidade	ICP-Brasil (e-CNPJ) + Keycloak	Infraestrutura existente, validade jurídica inquestionável no Brasil
Protocolo	Dataspace Protocol (DSP) via EDC	Padrão internacional, modular, extensível
Semântica	DATEX II + mobilityDCAT-ap	Adoção de padrões europeus de transporte já consolidados
Transformação	Apache Camel + RML Mapper	Low-code, integração com sistemas legados do governo
Catálogo	CKAN + ckanext-dcat	Portal de dados abertos já operacional no governo federal
Auditoria	IDS Clearing House (Fraunhofer)	Código aberto, conformidade com IDS RAM
App Store	Harbor (Docker Registry)	Registry de contêineres open-source, já utilizado em órgãos públicos
Infraestrutura	Kubernetes (Serpro/Dataprev)	Orquestração padronizada no governo federal

2. Roteiro de Implementação em 90 Dias

Fase 1 — Dias 1-30: Fundação e Onboarding dos Primeiros Provedores

Objetivo: Subir os componentes compartilhados e conectar os dois primeiros provedores de dados (PRF e DNIT).

Semana 1-2: Componentes Compartilhados

1. **Deploy do Keycloak (Gestor de Identidade)**

- Instância no Kubernetes do Serpro/Dataprev
- Configuração de realm RNDT-MVP
- Integração com certificados ICP-Brasil via mTLS
- Cadastro inicial dos CNPJs da PRF, DNIT, ANTT e concessionárias piloto

2. **Deploy do CKAN (Metadata Broker)**

- Instância baseada no portal dados.gov.br

- Instalação do plugin `ckanext-dcat` com perfil `mobilityDCAT-ap`
- Configuração de harvesters para PRF e DNIT

3. Deploy do Harbor (App Store)

- Registry privado para imagens Docker homologadas
- Política de assinatura de imagens (Notary/Cosign)

Semana 3-4: Conectores Provedores

4. Conector PRF

- Deploy do Eclipse EDC em ambiente da PRF
- Configuração do Apache Camel para ingestão de CSVs de sinistros
- Pipeline de transformação para DATEX II
- Cadastro do primeiro asset: `asset-prf-sinistros-2026`
- Definição de política ODRL: uso gratuito para órgãos públicos

5. Conector DNIT

- Deploy do Eclipse EDC em ambiente do DNIT
- Configuração do Apache Camel para ingestão de CSVs de pavimentação
- Pipeline de transformação para DATEX II com map-matching
- Cadastro do primeiro asset: `asset-dnit-pavimentacao-2026`
- Definição de política ODRL: uso gratuito governamental / comercial pago

Fase 2 — Dias 31-60: Integração e Primeiros Consumidores

Objetivo: Conectar consumidores e validar os fluxos de negociação e transferência.

Semana 5-6: Conectores Consumidores

6. Conector ANTT (Consumidor)

- Deploy do Eclipse EDC
- Integração com sistema de BI interno (Metabase/Superset)
- Testes de negociação de contrato com PRF e DNIT
- Validação de transferência de dados de sinistros e pavimentação

7. Conector Concessionária Piloto (Consumidor)

- Deploy do Eclipse EDC em ambiente de concessionária parceira
- Testes de negociação comercial com DNIT
- Validação de precificação e faturamento

Semana 7-8: Testes de Integração

8. Testes End-to-End

- Catalogação no CKAN
- Negociação de contratos (Fase 3)
- Transferência de dados (Fase 4)
- Registro na Clearing House
- Validação de políticas ODRL (tempo de uso, finalidade)

Fase 3 — Dias 61-90: App Store e Computação Próxima ao Dado

Objetivo: Implementar o primeiro Data App e validar o modelo de processamento soberano.

Semana 9-10: Desenvolvimento do Primeiro Data App

9. App: Calculador de Índice de Severidade de Acidentes

- Desenvolvimento em Python (Pandas/Scikit-Learn)
- Empacotamento em Docker
- Publicação no Harbor da RNDT
- Homologação pelo Ministério dos Transportes

10. Execução de Teste

- A ANTT solicita execução do app na infraestrutura da PRF
- O app processa dados brutos de sinistros e retorna apenas a matriz de correlação
- Validação de que dados sensíveis não saíram da PRF

Semana 11-12: Documentação e Capacitação

11. Documentação Técnica

- Manual de operação dos conectores
- Guia de onboarding para novos participantes
- Especificação de APIs e formatos

12. Capacitação

- Treinamento para equipes de TIC da PRF, DNIT e ANTT
- Workshop para concessionárias interessadas
- Publicação de caso de uso no portal da RNDT

3. Matriz de Responsabilidades (RACI)

Atividade	Ministério dos Transportes	PRF	DNIT	ANTT	Concessionárias	Serpro/Dataprev
Governança e Normas	R	C	C	C	I	I
Gestor de Identidade (Keycloak)	R	I	I	I	I	A
Metadata Broker (CKAN)	R	I	I	I	I	A

Atividade	Ministério dos Transportes	PRF	DNIT	ANTT	Concessionárias	Serpro/Dataprev
Clearing House	R	I	I	I	I	A
App Store (Harbor)	R	I	I	I	I	A
Conector Provedor	C	R	R	—	—	A
Conector Consumidor	C	—	—	R	R	A
Transformação Semântica (Camel)	C	R	R	—	—	I
Data Apps	A	C	C	R	R	I
Suporte e Operação	R	C	C	C	C	A

Legenda: R = Responsável, A = Aprovador, C = Consultado, I = Informado

4. Estratégia de Dados para o MVP

4.1 Conjuntos de Dados Prioritários

Prioridade	Órgão	Conjunto de Dados	Formato Bruto	Formato RNDT	Frequência
1	PRF	Sinistros Rodoviários	CSV mensal	DATEX II	Mensal
2	DNIT	Condições de Pavimento (ICM)	CSV mensal	DATEX II	Mensal
3	PRF	Operações de Fiscalização	CSV diário	DATEX II	Diário
4	DNIT	Inventário de Pontes e Viadutos	CSV anual	DATEX II	Anual

Prioridade	Órgão	Conjunto de Dados	Formato Bruto	Formato RNDT	Frequência
5	ANTT	Dados de Transporte de Carga	Sistema legado	DATEX II	Contínuo

4.2 Pipeline de Transformação (Apache Camel)

```
# Exemplo de rota Camel para sinistros da PRF
- route:
  id: "prf-sinistros-to-datex2"
  from:
    uri: "timer://prfTrigger?period=24h"
  steps:
    - to: "https://www.gov.br/prf/dados-abertos/sinistros"
    - unmarshal:
      csv:
        delimiter: ";"
        header: true
    - split:
      tokenize: "body"
    - process:
      ref: "MapMatchingProcessor"
      # Corrige coordenadas GPS para malha rodoviária
    - process:
      ref: "Datex2Converter"
      # Mapeia campos CSV para DATEX II
    - marshal:
      jacksonxml:
        rootName: "d2LogicalModel"
    - to: "minio://prf-rndt-staging/sinistros-datex2.xml"
    - to: "direct:register-asset-in-edc"
```

5. Modelo de Governança e Precificação do MVP

5.1 Tipos de Acesso

Tipo	Descrição	Política ODRL	Exemplo
Público	Dados abertos, sem restrição	use sem duty	Sinistros anônimos da PRF
Governamental	Acesso gratuito para órgãos públicos	use + purpose=PublicPlanning	Pavimentação do DNIT para ANTT

Tipo	Descrição	Política ODRL	Exemplo
Comercial	Acesso pago para empresas privadas	use + compensate	Dados de carga para concessionárias
Restrito	Acesso condicionado a credenciais específicas	use + securityClearance=Nivel_2	Dados completos de sinistros com placas

5.2 Precificação do MVP

Modalidade	Aplicabilidade	Valor de Referência
Gratuito	Órgãos públicos, pesquisa acadêmica	R\$ 0,00
Assinatura Mensal	Acesso contínuo a dados de telemetria	R\$ 5.000,00/mês
Pay-as-you-go	Consumo por volume de eventos	R\$ 0,10/1.000 eventos
Por Execução de App	Processamento de Data App	R\$ 500,00/execução

6. Segurança e Conformidade no MVP

6.1 Níveis de Segurança (Baseados no IDS RAM)

Nível	Descrição	Requisitos	Aplicação no MVP
Base Free	Pesquisa e desenvolvimento	mTLS básico	Ambientes de teste
Base	Operação mínima confiável	Certificados ICP-Brasil, criptografia TLS 1.3	Conectores de órgãos públicos
Trust	Operação com auditoria	+ Logging na Clearing House, políticas ODRL	Dados comerciais sensíveis
Trust+	Máxima segurança	+ TPM 2.0, Remote Attestation	Dados críticos de segurança nacional

6.2 Conformidade Legal

Legislação	Aplicação na RNDT	Implementação Técnica
LGPD (Lei 13.709/2018)	Proteção de dados pessoais em sinistros	Anonimização via Data Apps, políticas ODRL de finalidade

Legislação	Aplicação na RNDT	Implementação Técnica
Decreto 10.046/2019	Governança de dados no governo federal	CKAN como portal de dados, metadados DCAT
MP 2.200-2/2001	Validação jurídica de documentos digitais	Certificados ICP-Brasil para assinatura de contratos
Lei 14.129/2021	Governança da internet no Brasil	Infraestrutura hospedada em datacenters nacionais

7. Indicadores de Sucesso do MVP (KPIs)

Indicador	Meta (90 dias)	Meta (12 meses)
Provedores conectados	2 (PRF + DNIT)	10+ (incluindo estados e municípios)
Consumidores ativos	2 (ANTT + 1 concessionária)	50+
Assets catalogados	5	100+
Contratos negociados	20	1.000+
Transferências de dados	50	10.000+
Data Apps homologados	1	20+
Tempo médio de onboarding	5 dias úteis	1 dia útil
Disponibilidade do sistema	99,5%	99,9%

8. Riscos e Mitigações

Risco	Probabilidade	Impacto	Mitigação
Resistência cultural à troca de dados	Alta	Alto	Campanha de comunicação, demonstração de valor via dashboards
Complexidade técnica dos legados	Alta	Médio	Apache Camel como camada de abstração, equipe de integração

Risco	Probabilidade	Impacto	Mitigação
			dedicada
Indisponibilidade de certificados ICP-Brasil	Média	Alto	Processo de aquisição antecipado, suporte do ITI
Vazamento de dados sensíveis	Baixa	Crítico	Políticas ODRL rigorosas, Data Apps com processamento local
Escalabilidade do CKAN	Média	Médio	Arquitetura de microserviços, cache distribuído (Redis)
Dependência de fornecedor único (Serpro)	Média	Médio	Multi-cloud strategy, containers portáteis (Kubernetes)

9. Próximos Passos Pós-MVP

9.1 Evolução para Produção (6-12 meses)

- 1. **Migração para DIDs e VCs completas**
 - Implementação de resolvedores universais
 - Emissão de credenciais verificáveis descentralizadas
 - Integração com ecossistemas internacionais (Gaia-X, EMDS)
- 2. **Expansão da Rede**
 - Onboarding de Secretarias Estaduais de Transporte
 - Integração com municípios (via FIWARE/True Connector)
 - Conexão com espaços de dados setoriais (Saúde, Meio Ambiente)
- 3. **Maturidade de Analytics**
 - Machine Learning federado (dados não saem dos provedores)
 - Modelos preditivos de manutenção rodoviária
 - Dashboards de inteligência de tráfego em tempo real
- 4. **Certificação IDS**
 - Auditoria formal pela IDSA
 - Certificação de componentes (Connectors, Broker, DAPS)
 - Adesão ao Gaia-X Trust Framework

9.2 Visão de Longo Prazo (3-5 anos)

- **RNDT como Hub Latino-Americano:** Interoperabilidade com espaços de dados de transporte da América Latina (Mercosul)
- **Economia de Dados de Transporte:** Marketplace de dados e serviços analíticos com precificação dinâmica

- **Veículos Autônomos e Cidades Inteligentes:** Integração com V2X (Vehicle-to-Everything) e IoT urbano
- **Sustentabilidade:** Rastreamento de emissões de carbono na cadeia logística via dados federados

10. Glossário Rápido do MVP

Termo	Definição Prática
Asset	Um conjunto de dados catalogado e disponível para negociação (ex: "Sinistros PRF 2026")
Contract Definition	Regra que liga um Asset a uma Política ODRL
Control Plane	Cérebro do conector: negocia contratos e aplica políticas
Data Plane	Músculo do conector: move os dados de forma segura
DAPS	Serviço que emite tokens de identidade para conectores
DATEX II	Padrão europeu para descrição de eventos de trânsito e infraestrutura
EDC	Eclipse Dataspace Connector — framework open-source para conectores
Harvester	Robô que varre metadados de conectores para alimentar o catálogo
mTLS	TLS Mútuo: ambos os lados da conexão apresentam certificados
ODRL	Linguagem padrão para descrever permissões, proibições e obrigações de uso de dados
Policy Enforcement	Execução automática das regras do contrato durante a transferência
VC (Verifiable Credential)	Certificado digital portátil que prova atributos de um participante

Conclusão da Parte IV

A RNDT não é um projeto de tecnologia. É um **projeto de soberania digital e inovação em governança pública**. O MVP proposto nesta Parte IV demonstra que é perfeitamente viável, em 90 dias, colocar em operação uma rede federada de dados de transporte que:

1. **Respeita a autonomia institucional** de cada órgão (PRF, DNIT, ANTT), mantendo os dados sob sua custódia;
2. **Garante a soberania dos dados** através de contratos digitais vinculantes (ODRL) e criptografia de ponta a ponta;

3. **Fomenta a inovação** ao permitir que startups, universidades e empresas privadas criem produtos e serviços sobre dados públicos, sem nunca violar a privacidade ou a segurança;
4. **Escala de forma sustentável**, utilizando ferramentas de código aberto e infraestrutura já existente no governo federal.

O diferencial da RNDT não está nos bits que trafegam, mas na **confiança que ela constrói** entre atores que, até então, operavam em silos. Quando a PRF sabe que pode compartilhar dados de sinistros sem perder o controle, e o DNIT sabe que pode precificar dados de pavimentação sem criar monopólios, o transporte brasileiro deixa de ser uma soma de sistemas isolados e se torna um **ecossistema inteligente, seguro e soberano**.

Material de Capacitação — RNDT: Data Space de Transporte do Brasil

Baseado no IDS RAM 4.0 e nas lições do European Mobility Data Space (EMDS)

Versão MVP — 2026